

A Novel Branch-Price-and-Cut Algorithm for the Capacitated Minimum Spanning Tree Problem

Capacitated Minimum Spanning Tree (CMST) problem extends the well-known MST by restricting the number of vertices in each subtree to at most a given capacity Q . CMST has received considerable attention lately because it is a nice example of “easy to state, hard to solve” problem and has wide real-life applications, e.g., in telecommunication, logistics and energy. Several variants of the CMST have been investigated in the literature, and a number of exact methods have been proposed. However, available computational results suggest that different variants of CMST with different objective functions require ad hoc formulations and solution methods. In addition, solving instances with larger Q and more vertices would take relatively long time, sometimes exceeding ten hours. In this paper, we proposed a novel uniform branch-price-and-cut algorithm to address three variants of CMST. Its pricing procedure relies on two special tree structures, *pseudo tree* and *real tree*, obtained from the analysis of capacitated subtrees. The optimal *pseudo tree* is computed via a novel labeling algorithm based on the dichotomy concept, which can produce fewer labels than the existing method. We also introduce a new tree relaxation called *ng-arb*, which is used in a label pruning strategy to accelerate computation. Enhanced by several acceleration strategies and valid inequalities, our algorithm achieves significantly better performance than state-of-the-art methods from the literature and solves a large number of open CMST instances to optimality for the first time.

Key words: capacitated minimum spanning tree; column generation; exact method; dichotomy labeling

1. Introduction

Let $\mathcal{G} = (\mathcal{V}, \mathcal{A})$ be a directed graph with vertex set $\mathcal{V} = \{0\} \cup \mathcal{V}^+$ and arc set $\mathcal{A}$, where vertex 0 denotes the *Root*. Given a positive integer Q as the capacity bound, the Capacitated Minimum Spanning Tree (CMST) problem aims to find a minimum-cost spanning tree for $\mathcal{G}$ rooted at *Root*, such that every subtree hanging from the *Root* contains at most Q vertices.

The bound Q in CMST serves to constrain the flow on arcs in a network topology, e.g., bandwidth limitations in communication networks (Kawatra and Bricker 2000) and current limits in power grids (Luo et al. 2021). By setting $Q \leftarrow +\infty$, the CMST degenerates into the minimum spanning tree problem (MST), which dates back to the early 20th century; pioneering contributions were made by Borůvka (1926), laying the foundation for subsequent research. The wide application of MST across various fields, such as telecommunication (Graham and Hell 2007), wireless sensor networks (Liu 2015), image processing (Danda et al. 2021), and image segmentation (Sevak et al. 2017), has motivated continuous research into its solution algorithms. Prim (1957) and Kruskal (1956) are the two best-known greedy algorithms, which find the optimal solution to MST in

polynomial time. However, this is not the case for CMST and its variants, since they are NP-hard when $2 < Q < \left\lfloor \frac{|\mathcal{V}|}{2} \right\rfloor$ (Papadimitriou 1978).

In the literature, variants of CMST with different features have been addressed, and a wide range of solution methods (both exact and heuristic) have been proposed. The main difference among these variants lies in their objective functions, such as Euclidean distance cost (Uchoa et al. 2008), link cost (Uchoa et al. 2012), and flow cost (Luo et al. 2021). These three costs give rise to three distinct problems, namely, the *CMST with Euclidean distance cost* (CMST-E), the *CMST with link cost* (CMST-L), and the *CMST with flow cost* (CMST-F). In particular, CMST-L and CMST-F are also referred to as the multi-level CMST and the cable routing problem, respectively, in the literature. For brevity, we hereafter refer to these three problems as the three variants of CMST.

The exact methods currently available for these three variants of CMST are based on several different mathematical formulations, such as arc-based, hop-indexed, and tree-based formulations (Gavish 1983, Gouveia and Martins 1999, Uchoa et al. 2008). Currently, branch-and-cut (for the arc-based and hop-indexed formulations) and branch-and-price (for the tree-based formulation) algorithms are two most effective algorithms to solve CMST. However, the literature indicates that instances with larger Q (e.g., $Q \geq 10$) and more vertices ($|\mathcal{V}| \geq 120$) are challenging to solve, sometimes exceeding ten hours.

In this paper, we aim to partially close this research gap. We propose a novel branch-price-and-cut (BPC) algorithm for solving the tree-based formulation. The pricing subproblem of our BPC relies on two special tree structures, *pseudo tree* (PT) and *real tree* (RT), derived from the analysis of capacitated subtrees. We hereafter refer to our BPC as PR-BPC. The PR-BPC can solve instances of various sizes of the three variants of CMST more quickly, significantly outperforming the state-of-the-art exact methods tailored for each single variant. We describe PR-BPC by focusing on the classic CMST-E, and we later show how it can be tailored to solve the other two variants by adapting the pricing procedure. The main contributions of this paper are the following:

(a) We analyze the structure of capacitated subtrees and show that each such subtree can be decomposed into a PT and an RT through a unique core vertex. These two new structures help to identify promising columns in the column generation procedure without incurring long joining times, i.e., the time spent on combining the PT and RT to form feasible columns.

(b) We develop a novel dichotomy-based labeling algorithm for computing non-dominated PTs. This algorithm generates PT labels in dichotomous order, rather than the sequential order used in Luo et al. (2021) and Uchoa et al. (2008), which significantly reduces the number of label combinations and of newly generated labels.

(c) We propose a new tree structure, termed *ng-arb*, which can be efficiently computed and provides a tight bound in the labeling algorithm to prune unnecessary labels.

(d) We illustrate how PR-BPC can be applied to solve the other two variants of CMST (CMST-L and CMST-F) by modifying the labeling procedure.

(e) We provide computational results demonstrating that PR-BPC outperforms the state-of-the-art exact methods for the three variants of CMST.

The paper is organized as follows. Section 2 reviews the main contributions of exact methods for the three variants of CMST. Section 3 formally introduces three variants of CMST and presents their formulations. Section 4 analyzes the features of capacitated subtrees and proposes the PT, RT, and *ng-arb* structures. Section 5 describes PR-BPC in detail for solving CMST-E, including the dichotomy labeling. Section 6 illustrates how PR-BPC can be tailored to solve the other two variants of CMST. A computational analysis demonstrating the effectiveness of PR-BPC and a comparison of its performance with state-of-the-art methods are provided in Section 7. Finally, conclusions are drawn in Section 8.

2. Literature Review

This section reviews the main exact methods proposed for the three variants of CMST. For brevity, we omit heuristic methods, even though they can provide good feasible solutions to accelerate the convergence of exact algorithms.

As a direct extension of the classic MST, CMST-E was first introduced by Esau and Williams (1966) to design a teleprocessing tree network. Since then, this NP-hard problem has received attention from the optimization community for being a nice example of an “easy to state, hard to solve” problem. Chandy and Russell (1972) formulated CMST-E using a single-commodity flow formulation (SFF) and developed a branch-and-bound algorithm to solve it. Two types of variables are needed in this formulation: arc variables indicate whether certain arcs are used in the optimal solution, and flow variables represent the flow of demands on certain arcs. Gavish (1982) also used the SFF to develop a Benders decomposition algorithm, but with poor results. Gavish (1983) subsequently improved the model tightness by deriving a tighter set of coupling constraints that relate flow variables to arc variables; however, the lower bound of the model remains poor, leading to unsatisfactory results.

To improve the lower bound of SFF, Malik and Yu (1993) employed an arc-variable-only model. The capacity constraints of CMST-E are formulated as subtour elimination constraints and progressively added to the model to ensure feasibility. They designed a branch-and-bound method to solve the model, where the lower bound is given by Lagrangian relaxation with tightened constraints. Hall (1996) improved the solution method of Malik and Yu (1993) and devised a cutting-plane approach enhanced by strong cuts that were derived from polyhedral investigation (Araque et al.

1990, Zhang 1994). The experimental results show that about half of the CMST-E instances with $|\mathcal{V}^+| = 50$ and $Q = 8$ were solved to optimality.

There exist several studies that focus on designing more compact model. Gouveia (1995) showed that the two-index arc and flow variables used in SFF can be replaced by a single set of three-index binary variables. In this reformulation, the number of constraints in SFF is reduced from $\mathcal{O}(|\mathcal{V}|^2)$ to $\mathcal{O}(|\mathcal{V}|)$, while the number of variables increases from $\mathcal{O}(|\mathcal{V}|^2)$ to $\mathcal{O}(Q|\mathcal{V}|^2)$. The fewer number of constraints makes it more attractive to use for bounding purposes than the original SFF. Gouveia and Martins (1999) adopted the reformulation method proposed by Gouveia (1995), and generalized the SFF to a new hop-indexed model, which incorporates a new set of valid inequalities, called hop ordering inequalities. These inequalities, as well as the generalized subtour elimination constraints, are added to the initial model by cutting plane manner. Such method yield quite good lower bounds, but lead to very large LPs having $\mathcal{O}(Q|\mathcal{A}|)$ variables and constraints. Therefore, they are only suitable for CMST-E instances with small Q . In order to alleviate the storage problem and solve larger-scale instances, Gouveia and Martins (2000) proposed several levels of aggregation of the original formulation, yielding a hierarchy of hop-indexed models with fewer variables but weaker lower bound than the original hop-indexed model. These models are later enhanced by Gouveia and Martins (2005) through introducing stronger valid inequalities and new separation algorithms. The results show that instances with up to 120 vertices and $Q = 5$ can be proved to optimality.

Gouveia et al. (1990) is the first one to investigate tree-based set partitioning formulations for CMST-E. By an enumeration scheme, instances with $|\mathcal{V}^+| = 40$ and $Q = 3$ were solved to optimality. However, for larger instances, solving this model using a commercial solver is impractical due to the exponential number of variables. Uchoa et al. (2008) addressed this problem by proposing a robust BPC algorithm. They formulated CMST-E as a new set-partitioning model, where the variables are associated with q -arbs, a structure that arises from the relaxation of the capacitated prize-collecting arborescence problem. The robust cuts, such as capacity cuts, are also used to tighten the lower bound. Luo et al. (2021) provided the best results for CMST-E. They proposed a set-partitioning formulation with *elementary subtree* variables, enhanced by capacity (robust) and subset-row (non-robust) cuts. This formulation can provide the tightest lower bound for CMST-E, which is also used in our work. To solve it, the authors devised a BPC algorithm and adopted several acceleration strategies to speed up the computation. Instances with $|\mathcal{V}^+| = 160$ and $Q = 10$ can be solved to optimality, but solving them takes a relatively long time (over two hours).

CMST-L generalizes CMST-E by further considering the link selection decision for each arc. Gamvros et al. (2006) first proposed this problem. They formulated two flow-based mixed integer programming formulations to compute lower bounds and designed several heuristic procedures to

obtain upper bounds. However, research on exact algorithms for CMST-L has been quite limited. To date, only [Uchoa et al. \(2012\)](#) developed a branch-and-cut algorithm to exactly solve CMST-L, where they proposed Fenchel cuts ([Boyd 1994](#)) and a separation procedure for them to accelerate convergence. They show that CMST-L instances with 100 vertices and $Q = 10$ can be exactly solved, with computation times sometimes exceeding 11 hours.

[Luo et al. \(2021\)](#) first proposed CMST-F, which is used to address the cable routing problem arising in the planning of large-scale solar power plants. They formulated this problem as a generalized CMST problem to capture the flow cost of each arc, and then devised a BPC algorithm to solve it. On real-world instances, they show that their BPC can exactly solve instances with up to 108 vertices and $Q = 14$. However, for the most challenging instances, their BPC did not perform well, as solving a single branch-and-bound node alone took more than ten hours. [Wang et al. \(2025\)](#) generalized the problem proposed in [Luo et al. \(2021\)](#) and designed a Benders decomposition algorithm to solve an integrated problem that includes district partitioning, photovoltaic (PV) component location, and cable routing. Compared with the BPC algorithm of [Luo et al. \(2021\)](#), their method achieves a speedup of approximately 3.52 times on small-scale instances with $Q = 8$. However, the method of [Wang et al. \(2025\)](#) is not general but rather customized for instances where the distances between vertices are measured by Hamiltonian length. In this case, CMST-F can be modeled as a special, simpler assignment problem, which is more amenable to solution.

In this paper, we focus on solving the three aforementioned variants of CMST—CMST-E, CMST-L, and CMST-F—more quickly. These problems serve as a basis for modeling and solving other problems with similar features and problem-specific side constraints.

3. Description of Three Variants of CMST and Their Formulations

3.1. Set Partitioning Formulations with Different Cost Structure

As our solution approach is based on column generation (an iterative technology that used to solve branch-and-bound node in BPC), all three types of CMST are modeled using set partitioning formulations. Consequently, we omit the presentation of their arc-based models in this section. Readers interested in these models can refer to Appendix [EC.1](#). Let Λ denote the set of all *elementary subtrees* whose total demand does not exceed Q ; here, "elementary" means that each vertex appears at most once. A subtree $\lambda \in \Lambda$ is described by binary coefficients $\alpha_{i,\lambda}$ that indicate whether subtree λ contains vertex $i \in \mathcal{V}$, and by the cost $C(\lambda)$ associated with subtree λ . With these notations, the general formulation (GF) is the following.

$$z(\text{GF}) = \min \sum_{\lambda \in \Lambda} C(\lambda) x_{\lambda} \tag{1}$$

$$\text{s.t. } \sum_{\lambda \in \Lambda} \alpha_{i,\lambda} x_\lambda = 1, \quad i \in \mathcal{V}^+ \quad (2)$$

$$\text{side constraints} \quad (3)$$

$$x_\lambda \in \{0, 1\}, \quad \forall \lambda \in \Lambda \quad (4)$$

where constraints (2) ensure that each vertex $i \in \mathcal{V}^+$ is covered exactly once. Since the three variants of CMST have different cost functions, we let $C^1(\lambda)$, $C^2(\lambda)$ and $C^3(\lambda)$ denote the objective coefficients (i.e., $C(\lambda)$ in GF) of subtree λ in CMST-E, CMST-L, and CMST-F, respectively. Then, We describe the structure of these functions and the customized side constraints of the three variants of CMST.

Description of CMST-E

CMST-E has the simplest cost structure and its side constraints are empty. In this problem, each arc $(i, j) \in \mathcal{A}$ is assigned to a fixed cost $c_{i,j}$. Let the binary coefficients $\beta_{i,j,\lambda}$ to denote whether (i, j) is used by the subtree λ , the first cost function $C^1(\lambda)$ can be computed as $C^1(\lambda) = \sum_{(i,j) \in \mathcal{A}} \beta_{i,j,\lambda} c_{i,j}$.

Description of CMST-L

The cost structure of CMST-L introduces the link selection costs, depending on the flow on each arc. Within a subtree, each vertex routes one unit of flow to the *Root*. Due to the capacity limit, the flow cannot exceed Q . Given L link types with capacities $0 < v^1 < v^2 < \dots < v^L = Q$ and costs $c_{i,j}^l$ for each arc $(i, j) \in \mathcal{A}$, the cost of using arc (i, j) in subtree λ is the minimum cost among links whose capacity can cover the flow $f_{i,j,\lambda}$ on that arc. Let $f_{i,j,\lambda}$ denote the flow on arc (i, j) in subtree λ . Then the second cost function $C^2(\lambda)$ is defined as $C^2(\lambda) = \sum_{(i,j) \in \mathcal{A}} \beta_{i,j,\lambda} \min_{l \in \{1, 2, \dots, L\}} \{c_{i,j}^l | v^l \geq f_{i,j,\lambda}\}$. Similarly, the side constraints for CMST-L are also empty.

Description of CMST-F

CMST-F is a special case of CMST-L, applied to a large-scale PV system designed to supply usable solar power to the electricity grid. CMST-F determines the partition of the PV arrays by assigning the locations of combiner boxes. Each location of a combiner box can be seen as the root of a subtree. The PV arrays are represented as vertices in the graph $\mathcal{G}$ and are connected via a tree topology to the combiner boxes to collect current. All combiner boxes then feed into the solar inverter (*Root* 0) through a single link for variable direct/alternating current conversion. Since each PV is connected to its assigned combiner box via a dedicated cable, the cable cost on each arc $(i, j) \in \mathcal{A}, j \neq 0$ is proportional to the flow on that arc. For example, if the flow on an arc is 3, this means that three PVs require three cables to be constructed on this arc for current transmission. The actual costs on arcs are parameterized by Euclidean distance. Based on the above description, the third cost function $C^3(\lambda)$ is given by $C^3(\lambda) = \sum_{(i,j) \in \mathcal{A}, j \neq 0} \beta_{i,j,\lambda} f_{i,j,\lambda} c_{i,j} + \sum_{(i,j) \in \mathcal{A}, j=0} \beta_{i,j,\lambda} c_{i,j}$.

Let $\mathcal{T}$ be the set of combiner box types, where a box of type $t \in \mathcal{T}$ has fixed capacity q_t . This implies that the capacity of a subtree in CMST-F must be one of the values in $\{q_t\}_{t \in \mathcal{T}}$. Let $\gamma_{t,\lambda}$

denote the binary parameter that is equal to 1 if the capacity of subtree λ is exactly q_t . CMST-F requires that the number of combiner boxes of each type $t \in \mathcal{T}$ is fixed in advance to m_t , which introduces the following side constraints :

$$\sum_{\lambda \in \Lambda} \gamma_{t,\lambda} x_\lambda = m_t, \quad t \in \mathcal{T} \quad (5)$$

Figure 1 shows an example of a solution to CMST with two subtrees, λ_1 (rooted at vertex 1) and

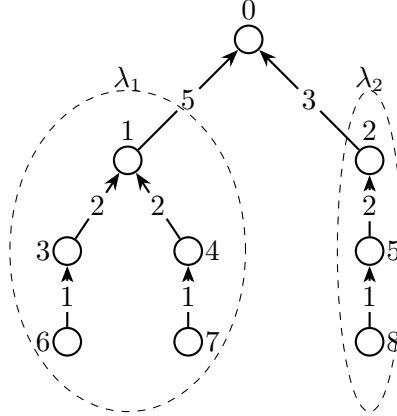


Figure 1 Example of a Spanning Tree with Two Subtrees (λ_1 and λ_2)

λ_2 (rooted at vertex 2), hanging from *Root* 0. Then, the costs corresponding to the three variants of CMST can be evaluated as follows (in the CMST-L case, two link types are available : $l = 1$ with capacity 2 and $l = 2$ with capacity 5):

- CMST-E : $C^1(\lambda_1) + C^1(\lambda_2) = (c_{6,3} + c_{7,4} + c_{3,1} + c_{4,1} + c_{1,0}) + (c_{8,5} + c_{5,2} + c_{2,0})$.
- CMST-L : $C^2(\lambda_1) + C^2(\lambda_2) = (c_{6,3}^1 + c_{7,4}^1 + c_{3,1}^1 + c_{4,1}^1 + c_{1,0}^2) + (c_{8,5}^1 + c_{5,2}^1 + c_{2,0}^2)$.
- CMST-F : $C^1(\lambda_1) + C^1(\lambda_2) = (c_{6,3} + c_{7,4} + 2c_{3,1} + 2c_{4,1} + c_{1,0}) + (c_{8,5} + 2c_{5,2} + c_{2,0})$.

4. Analysis of Capacitated Subtree Structure

This section presents three types of subtree structures. Based on the concept of decomposition, Section 4.1 decomposes a subtree into two structures, namely PT and RT, which help subsequent label setting algorithms (Section 5.1) find optimal *elementary subtrees*. Section 4.2 proposes a novel relaxation of *elementary subtrees*, termed *ng-arb*. This new structure is utilized in later label pruning (Section 5.4.3) to eliminate unnecessary labels and accelerate the label setting process.

4.1. Decomposition of A Capacitated Subtree

4.1.1. Core Vertex of A Subtree Directly pricing *elementary subtrees* is prohibitively time-consuming (Luo et al. 2021). To alleviate the computational burden, we decompose a subtree into multiple subtrees of smaller capacity via a unique vertex, called the *core vertex*.

PROPOSITION 1. *Given a subtree with the capacity $q \leq Q$ hanging from Root 0, we can always find an unique core vertex i to decompose the subtree into a set of subtrees hanging from i with the capacity do not exceed $\lfloor \frac{q}{2} \rfloor$, and one subtree hanging from 0 with the capacity do not exceed $\lceil \frac{q}{2} \rceil$.*

An example of the subtree decomposition in Proposition 1 is shown in Figure 2. The subtree with capacity $q = 7$ is decomposed by core vertex 1 into four subtrees rooted at vertices 1, 2, 3, and 4, respectively. Since the subtree consisting of vertex 1 alone, hanging from Root 0, constitutes a complete tree structure, it is referred to as a *real tree*. The remaining three subtrees rooted at vertices 2, 3, and 1 can be merged via core vertex 1 to form a new subtree that does not include vertex 1. As the merged tree is incomplete (vertex 1 is not activated), it is termed a *pseudo tree*. Within these definitions, it is natural to conclude that any subtree can be obtained by combining one real tree and one pseudo tree via its core vertex.

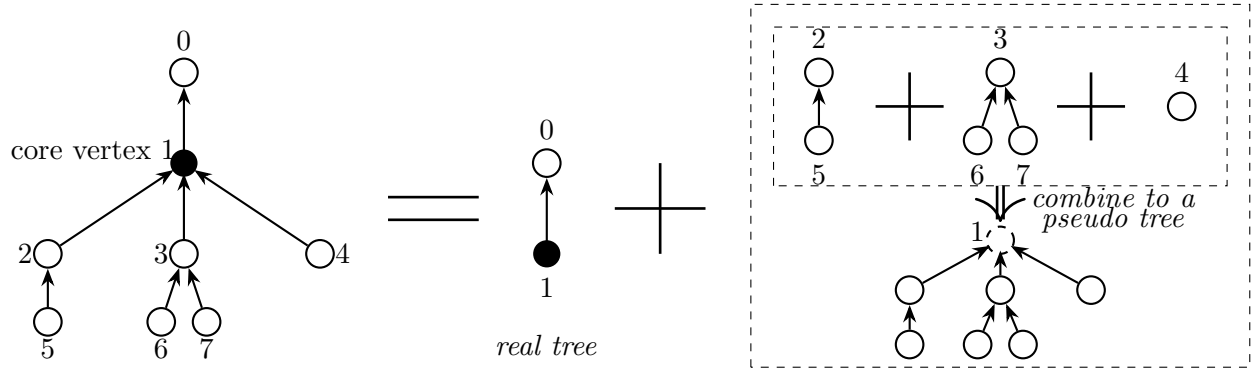


Figure 2 Description of Tree Decomposition, Real Tree and Pseudo Tree

4.1.2. Combination Patterns of Real and Pseudo Trees Although each subtree has a unique core vertex (Proposition 1), there is more than one combination case of PTs and RTs to form the same subtree. Taking the subtree in Figure 2 as an example, besides the combination case presented in that figure, Figure 3 shows the remaining four PT-RT combination cases. However, too many combination cases impose a severe computational burden, because the same subtree is generated repeatedly when PTs and RTs are obtained via joining procedure. Therefore, reducing the number of such cases without sacrificing optimality is essential for accelerating the pricing. To do so, Lemma 1 shows that there are at most $\lceil \frac{q}{2} \rceil$ combination cases.

LEMMA 1. *Considering a subtree with capacity q , there are at most $\lceil \frac{q}{2} \rceil$ possible cases (termed combination patterns) as follows being decomposed a PT and a RT by its core vertex.*

Pattern 0 : $l_{pt} = \lfloor \frac{q}{2} \rfloor$, $l_{rt} = \lceil \frac{q}{2} \rceil$, $\omega_{pt} \geq 1$, and $1 \leq \rho_{pt} \leq \lfloor \frac{q}{2} \rfloor$;

Pattern 1 : $l_{pt} = \lfloor \frac{q}{2} \rfloor + 1, l_{rt} = \lceil \frac{q}{2} \rceil - 1, \omega_{pt} \geq 2$, and $\left\lceil \frac{\lfloor \frac{q}{2} \rfloor + 1}{\lfloor \frac{q}{2} \rfloor} \right\rceil \leq \rho_{pt} \leq \left\lfloor \frac{\lfloor \frac{q}{2} \rfloor + 1}{2} \right\rfloor$;

Pattern 2 : $l_{pt} = \lfloor \frac{q}{2} \rfloor + 2, l_{rt} = \lceil \frac{q}{2} \rceil - 2, \omega_{pt} \geq 3$, and $\left\lceil \frac{\lfloor \frac{q}{2} \rfloor + 2}{\lfloor \frac{q}{2} \rfloor} \right\rceil \leq \rho_{pt} \leq \left\lfloor \frac{\lfloor \frac{q}{2} \rfloor + 2}{3} \right\rfloor$;

...

Pattern f : $l_{pt} = \lfloor \frac{q}{2} \rfloor + f, l_{rt} = \lceil \frac{q}{2} \rceil - f, \omega_{pt} \geq f + 1$, and $\left\lceil \frac{\lfloor \frac{q}{2} \rfloor + f}{\lfloor \frac{q}{2} \rfloor} \right\rceil \leq \rho_{pt} \leq \left\lfloor \frac{\lfloor \frac{q}{2} \rfloor + f}{f+1} \right\rfloor$;

...

Pattern $(\lceil \frac{q}{2} \rceil - 3)$: $l_{pt} = q - 3, l_{rt} = 3, \omega_{pt} \geq \lceil \frac{q}{2} \rceil - 2$, and $\left\lceil \frac{q-3}{\lceil \frac{q}{2} \rceil} \right\rceil \leq \rho_{pt} \leq \left\lfloor \frac{q-3}{\lceil \frac{q}{2} \rceil - 2} \right\rfloor$;

Pattern $(\lceil \frac{q}{2} \rceil - 2)$: $l_{pt} = q - 2, l_{rt} = 2, \omega_{pt} \geq \lceil \frac{q}{2} \rceil - 1$, and $\left\lceil \frac{q-2}{\lceil \frac{q}{2} \rceil} \right\rceil \leq \rho_{pt} \leq \left\lfloor \frac{q-2}{\lceil \frac{q}{2} \rceil - 1} \right\rfloor$;

Pattern $(\lceil \frac{q}{2} \rceil - 1)$: $l_{pt} = q - 1, l_{rt} = 1, \omega_{pt} \geq \lceil \frac{q}{2} \rceil$, and $\left\lceil \frac{q-1}{\lceil \frac{q}{2} \rceil} \right\rceil \leq \rho_{pt} \leq \left\lfloor \frac{q-1}{\lceil \frac{q}{2} \rceil} \right\rfloor$;

where l_{pt} and l_{rt} represents the capacity of PT and RT, respectively, ω_{pt} denotes the minimum capacity of subtrees in PT hanging from the core vertex and ρ_{pt} is the number of these subtrees in PT.

In Pattern f ($1 \leq f \leq \lceil \frac{q}{2} \rceil - 1$), the capacity of subtrees in PT hanging from the core vertex must be greater than or equal to $f + 1$, i.e., $\omega_{pt} \geq f + 1$; otherwise, we can transform this pattern into one of the foregoing patterns by moving one subtree from the PT to the RT. For example, if there exists a subtree of PT with capacity $m \leq f$, it can be removed to RT, and the Pattern f will be transformed to the Pattern $f - m$ where $l_{pt} = \lfloor \frac{q}{2} \rfloor + f - m \geq \lfloor \frac{q}{2} \rfloor$ and $l_{rt} = \lceil \frac{q}{2} \rceil - f + m \leq \lceil \frac{q}{2} \rceil$. Moreover, the capacity range of the subtrees of PT in Pattern f is $f + 1 \leq \omega_{pt} \leq \lfloor \frac{q}{2} \rfloor$, leading to the subtrees' number range $\left\lceil \frac{l_{pt}}{\lfloor \frac{q}{2} \rfloor} \right\rceil = \left\lceil \frac{\lfloor \frac{q}{2} \rfloor + f}{\lfloor \frac{q}{2} \rfloor} \right\rceil \leq \rho_{pt} \leq \left\lfloor \frac{\lfloor \frac{q}{2} \rfloor + f}{f+1} \right\rfloor = \left\lfloor \frac{l_{pt}}{f+1} \right\rfloor$. By this way, the PT-RT combination cases shown in Figure 2 and 3 are shrunk from 5 to 2, only Case 2 (Figure 3b) and Case 4 (Figure 3d) can be considered and both of these two cases are transformed from the combination case in Figure 2. Specifically, Case 2 is transformed by removing one subtrees from PT to RT (rooted at 3 hanging from 1), while Case 4 does so by removing two subtrees (rooted at 2 and 4, respectively, hanging from 1). An example of combination pattern in lemma 1 is shown in EC.3.1.

By lemma 1, we derive the following proposition to constrain the generation of PTs and RTs.

PROPOSITION 2. *To form a subtree with capacity q , we only need to consider the following two types of structures*

1. RT with capacity from 1 to $\lfloor \frac{q}{2} \rfloor$.
2. PT with capacity q' from 1 to $q - 1$. Particularly, if $q' > \lfloor \frac{q}{2} \rfloor$, the capacity of subtree in this PT hanging from the core vertex must be larger than or equal to $q' - \lfloor \frac{q}{2} \rfloor + 1$ and be less than or equal to $\lfloor \frac{q}{2} \rfloor$, PT must contain two or more subtrees hanging from the core vertex.

4.2. A New Capacitated Subtree Relaxation : Ng-arb

Ng-arb is a new relaxation of the *elementary subtree*, inspired by the well-known *ng-route* structure (Baldacci et al. 2011) used in the vehicle routing problem. It hangs from *Root* 0, has total demand

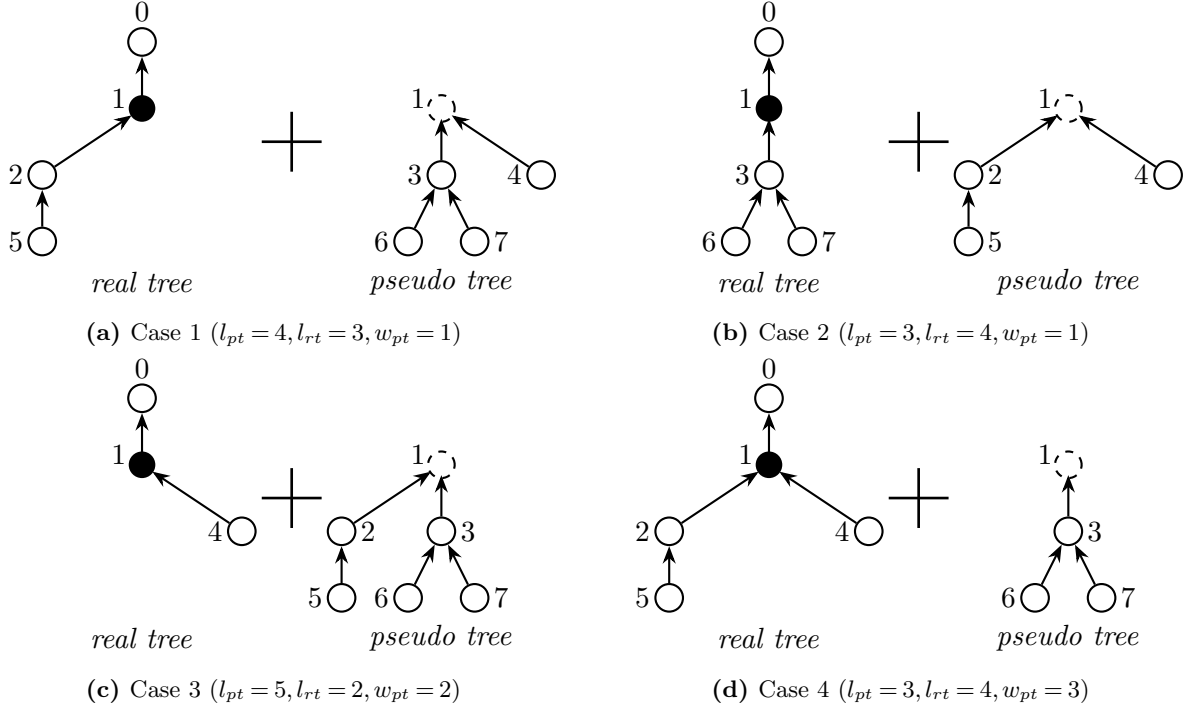


Figure 3 Four Cases of PT-RT combinations

at most Q , but allows some vertices (and even arcs) to appear more than once when certain neighborhood conditions are satisfied. For each vertex $i \in \mathcal{V}^+$, let $N_i \subseteq \mathcal{V}^+$ be the *ng-set* of i , defining its neighborhood. This set may consist of the $|N_i|$ closest vertices to i (this cardinality is chosen a priori) and also include i itself. Define $P_{\lambda_{ng}}(\bar{i}, \bar{j})$ as the path from $\bar{i}$ to $\bar{j}$ in the *ng-arb* λ_{ng} . Here, $\bar{i}$ and $\bar{j}$ represent the positions of vertices in the *ng-arb* rather than the vertices themselves. Let $\phi_{\lambda_{ng}}(\bar{j})$ be a function that returns the vertex at the $\bar{j}$ -th position in the *ng-arb*. Then, we define

DEFINITION 1. A *ng-arbs* rooted at a vertex $i \neq 0$ can be :

1. The vertex i alone, in this case, the *ng-arb* demand is 1.
2. k other *ng-arbs* rooted at distinct vertices $v_1, \dots, v_k$ hangs from i by arcs (v_j, i) for $j = 1, \dots, k$.

The demand of this *ng-arb* is 1 plus the demand of its k *sub-ng-arbs* and must not exceed Q .

3. For any path $P_{\lambda_{ng}}(\bar{i}, \bar{j})$ in the *ng-arb* λ_{ng} where its start and end positions corresponds to the same vertex $i \in \mathcal{V}^+$, i.e., $\phi_{\lambda_{ng}}(\bar{i}) = \phi_{\lambda_{ng}}(\bar{j}) = i$, there must exist one position $\bar{p}$ in this path such that $\phi_{\lambda_{ng}}(\bar{p}) \notin N_i$.

Note that without condition 3, the *ng-arb* degrades into the *q-arb* structure, which was used by [Uchoa et al. \(2008\)](#) as columns in their BPC algorithm. [EC.4](#) gives an example illustrating a *q-arb* that is not an *ng-arb*.

For a subtree λ rooted at vertex i (indexed by $\bar{i}$), we associate with it a *ng-state* $\Pi(\lambda) \subseteq N_i$ containing vertex i itself and every vertex j (indexed by $\bar{j}$) such that j belongs to all the *ng-sets* of vertices in the path $P_\lambda(\bar{j}, \bar{i})$. Let $\Phi(\lambda)$ denote the set of position indices in subtree λ . Then,

$$\Pi(\lambda) = \{\phi(\bar{j}), \bar{j} \in \Phi(\lambda) : \phi(\bar{j}) \in \bigcap_{\bar{k} \in P_\lambda(\bar{j}, \bar{i})} N_{\phi(\bar{k})}\} \cup \{i\} \quad (6)$$

For example, let $\lambda = (1, 2, 3)$ be a subtree rooted at vertex 1, associated with two arcs: $(3, 1)$ and $(2, 1)$, and let $N_1 = \{1, 2\}$, $N_2 = \{2, 3\}$, $N_3 = \{3, 1\}$. As there are no duplicate vertices in this subtree, we omit the mapping function $\phi()$ (Here, $\phi(1) = 1, \phi(2) = 2, \phi(3) = 3$). Then, $2 \in \bigcap_{k \in P_\lambda(2, 1)} N_k = N_2 \cap N_1$ and $3 \notin \bigcap_{k \in P_\lambda(3, 1)} N_k = N_3 \cap N_1$; from expression (6), we have $\Pi(\lambda) = \{1, 2\}$.

We define a special *ng-arb* (NG, i, j, q) covering exactly q vertices in set $\mathcal{V}^+$, rooting at vertex i , with *ng-state* NG , and such that $h \leq j$ for every child vertex $h \in \mathcal{V}^+$ of the root i . To ensure the neighborhood condition, i.e., 3 of Definition 1, the following constraints must be satisfied.

$$i \notin \bigcup_{\kappa=1}^k \Pi(\lambda_\kappa) \quad \Pi(\lambda_i) \cap \Pi(\lambda_j) = \emptyset, \forall i, j \in \{1, \dots, k\}, i \neq j \quad NG = \bigcup_{\kappa=1}^k \Pi(\lambda_\kappa) \cap N_i \quad (7)$$

where $\{\lambda_\kappa \mid \kappa = 1, \dots, k\}$ denotes k *sub-ng-arbs* of (NG, i, q) . The first two constraints ensure that (NG, i, q) is a *ng-arb*, and the last constraint shows how to update the *ng-state* using *sub-ng-arbs*. Let $f(NG, i, j, q)$ be the minimum cost of an *ng-arb* (NG, i, j, q) , computed as follows:

$$f(NG, i, j, q) = \min \begin{cases} f(NG, i, j-1, q) & (i) \\ f(NG', j, |\mathcal{V}^+|, q-1) + c_{j,i} : \forall NG' \in N_j \text{ s.t. } NG' \ni j, NG' \not\ni i \text{ and } NG' \cap N_i = NG & (ii) \\ \min\{f(NG_1, i, j-1, q-q') + c_{j,i} + f(NG_2, j, |\mathcal{V}^+|, q') : 1 \leq q' \leq q-1, \forall NG_1 \in N_i, \\ NG_2 \in N_j \text{ s.t. } NG_1 \ni i, NG_2 \ni j, NG_1 \cap NG_2 = \emptyset \text{ and } NG_1 \cup (NG_2 \cap N_i) = NG & (iii) \end{cases} \quad (8)$$

where the initialization $f(\{i\}, i, j, 1) = 0, \forall i \in \mathcal{V}^+, j \in \mathcal{V}^+$ are required. In the recursion (8), (i) represents the case where the vertex j has no outgoing arc to the root of *ng-arb* (NG, i, j, q) . In this case, all the capacity q of (NG, i, j, q) is used by $(NG, i, j-1, q)$. On the other hand, (ii) represents the case where the vertices numbered from 1 to $j-1$ have no outgoing arcs to the *ng-arb* (NG, i, j, q) . In this case, all the capacity $q-1$ transferred through the outgoing arcs to the root of (NG, i, j, q) is used by the *sub-ng-arb* rooted at j , $(NG', j, |\mathcal{V}^+|, q-1)$. Note that $NG' \in N_j$ presents all possible *ng-state* of the *sub-ng-arb*. In the following constraints, $NG' \ni j$ and $NG' \not\ni i$ respectively guarantee that $(NG', j, |\mathcal{V}^+|, q-1)$ and (NG, i, j, q) are *ng-arbs*, and $NG' \cap N_i = NG$ ensures the correctness of state update. Finally, (iii) represents the case where the capacity q of (NG, i, j, q) is combined by $(NG_1, i, j-1, q-q')$ and the *sub-ng-arb* rooted at j . Note that $NG_1 \in N_i, NG_2 \in N_i$ presents all possible *ng-state* of the two *sub-ng-arbs*. In the following constraints, $NG_1 \ni i, NG_2 \ni j$ and $NG_1 \cap NG_2 = \emptyset$ respectively guarantee that $(NG_1, i, j-1, q-q'), (NG_2, j, |\mathcal{V}^+|, q')$ and (NG, i, j, q) are *ng-arbs*, and $NG_1 \cup (NG_2 \cap N_i) = NG$ ensures the correctness of state update.

The *ng-arb* provides a balance between computational effort and solution quality and is employed in Section 5.4.3 to remove unnecessary labels.

5. PT-RT-based Branch-price-and-cut Algorithm

This section describes the components of the full PR-BPC algorithm we implemented. Following the classic BPC framework, three fundamental components, *pricing*, *branching*, and *cut separation*, must be carefully designed. Among these, the pricing process is the most time-consuming part; therefore, most of the acceleration strategies in PR-BPC focus on this component. Specifically, Section 5.1 describes the pricing subproblem of GF and shows how to obtain PTs by dichotomy labeling and RTs by backward labeling, and how to generate *elementary subtrees* by joining these two structures. Section 5.2 and Section 5.3 present the branching and cut separation components, respectively. Section 5.4 describes four acceleration strategies, all of which are used to speed up the pricing process. Finally, Section 5.5 shows the external upper bound procedure for the three variants of CMST. Note that the PR-BPC described in this section is customized for CMST-E. The modifications of PR-BPC for solving CMST-L and CMST-F are detailed in Section 6.

5.1. Solving the Pricing Subproblem

The goal of the pricing subproblem is to find a feasible *elementary subtree* with the smallest reduced cost. Let $\{\mu_i\}_{i \in \mathcal{V}^+}$ be the dual values of Constraints (2). The pricing subproblem is as follows:

$$\begin{aligned} [PP_1] \quad & \min_{\lambda} \sum_{(i,j) \in \mathcal{A}} \beta_{i,j,\lambda} c_{i,j} - \sum_{i \in \mathcal{V}^+} \alpha_{i,\lambda} \mu_i \\ \text{s.t.} \quad & \lambda \text{ is a feasible elementary subtree} \end{aligned} \quad (9)$$

To solve PP_1 , we can run the recursion (8) associated with *ng-set* $N_i = \mathcal{V}^+$ for all $i \in \mathcal{V}^+$. Under this setting, any *ng-arb* is an *elementary subtree*. Luo et al. (2021) implemented this by designing a label setting algorithm. They maintain a label set $\mathbb{L}(i, j, q)$ storing all non-dominated subtrees rooted at vertex i , covering q vertices, with the restriction that the child vertices of i have indices not larger than j . Then, the optimal elementary subtree can be found in the final non-dominated subtree set $\bigcup_{i \in \mathcal{V}^+, q \in \{1, \dots, Q\}} \mathbb{L}(i, |\mathcal{V}^+|, q)$. Figure 4 shows the label transfer process ((i) of recursion (8)) of this labeling algorithm for vertex i and capacity q . As the set $\mathbb{L}$ is constructed in a vertex sequence manner, i.e., $\mathbb{L}(i, j, q)$ can only be constructed after $\mathbb{L}(i, j-1, q)$ is formed, we refer to this algorithm as *sequential labeling* in the following.

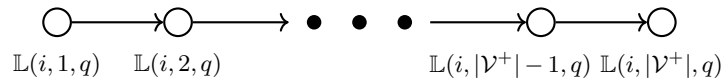


Figure 4 Sequence labeling for vertex i and capacity q

However, in our preliminary experiments, we found that the set $\mathbb{L}(i, j, q), j < |\mathcal{V}^+|$ of the *sequential labeling* always collects too many labels, most of which are unnecessary. These labels generate bad

labels with larger capacity through (iii) of recursion (8), which may be dominated by other labels in the later stage of *sequential labeling*. The reason for this situation can be captured by a new concept called *lifetime*, which means the number of transfer sets a label passes through from its generation to its domination. For example, if a label in $\mathbb{L}(i, 1, q)$ is generated by (ii) of recursion (8), and is dominated (i.e., removed) in set $\mathbb{L}(i, 8, q)$ by other labels in this set, the *lifetime* of this label is 8. Obviously, the average *lifetime* of labels in *sequential labeling* is $\mathcal{O}(|\mathcal{V}^+|)$, which is too large and thus leads to an excessive number of labels in the set $\mathbb{L}(i, j, q), j < |\mathcal{V}^+|$.

5.1.1. Dichotomy Labeling for Pesodu Tree In this subsection, we design a *dichotomy labeling* algorithm to generate PTs. Here, each label represents a stage associated with a PT, and new labels are created by dichotomy extension. The concept of dichotomy stipulates extension rules for labels, ensuring that the generated PTs (labels) are *q-arbs* (excluding their root vertices) even if the elementary condition is relaxed. A dominance criterion is applied to eliminate PTs that cannot lead to an optimal solution. Note that as the core vertex of the optimal *elementary subtree* cannot be determined a priori, PTs rooted at every vertex $i \in \mathcal{V}^+$ must be extended.

Let $P_i = (\bar{c}(P_i), q(P_i), r(P_i), V(P_i))$ denote a label that represents a PT rooted at vertex i , where

- $\bar{c}(P_i)$ is the reduced cost of the PT;
- $q(P_i)$ is the number of vertices the PT covers (i is not counted);
- $r(P_i)$ is the minimum capacity of the subtrees in the PT hanging from the core vertex i .
- $V(P_i)$ is the set of vertices covered by the PT (i is not included).

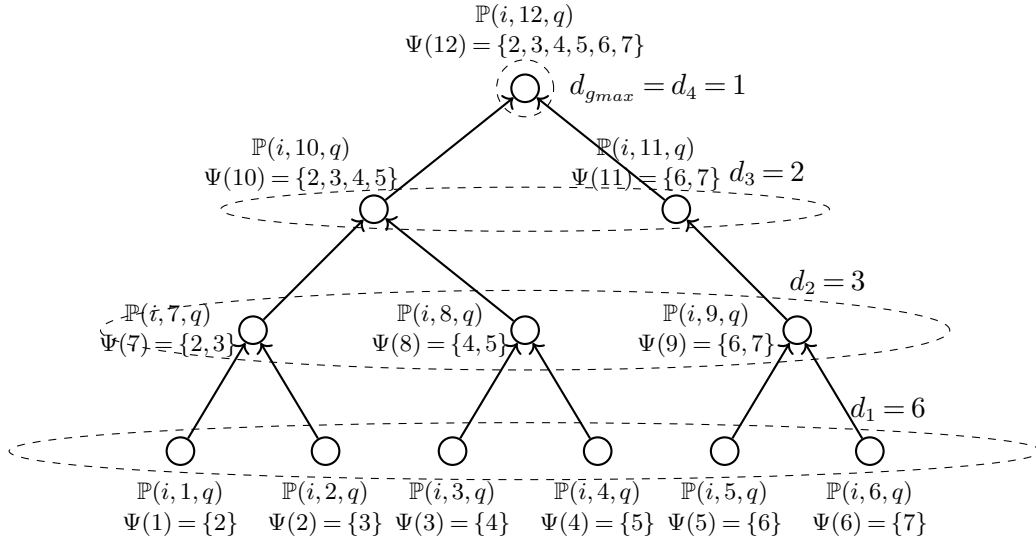


Figure 5 A binary tree with vertex i , capacity q , and $|\mathcal{V}^+| = 7$

The label extension rule depends on a binary tree structure to store corresponding labels. To describe this tree, we let $g_{max} = \lceil \log_2(|\mathcal{V}^+| - 1) \rceil + 1$ be its depth, and define the integers $d_1 =$

$|\mathcal{V}^+| - 1, d_2 = \lceil \frac{d_1}{2} \rceil, \dots, d_{g-1} = \lceil \frac{d_g}{2} \rceil, \dots, d_{g_{max}} = 1$ as the number of vertices in each layer, where d_1 corresponds to the leaf nodes of the deepest layer and $d_{g_{max}}$ corresponds to the root node of the binary tree. This storage structure is replicated and allocated to every possible core vertex $i \in \mathcal{V}^+$. Then, each tree node is regarded as a pool of PT labels, represented by $\mathbb{P}(i, h, q)$, where i is the root of the associated PT, $h = 1, 2, \dots, M = \sum_{g=1}^{g_{max}} d_g$ is the indicated number of tree nodes, and $q = 0, 1, \dots, Q - 1$ is the capacity of the PT. Figure 5 gives an example of binary tree with vertex i , capacity q and $|\mathcal{V}^+| = 7$. Let $\Psi(h)$ be the set of indicated numbers, each of which corresponds to the leaf nodes of the subtree rooted at h in this binary tree (e.g., $\Psi(7) = \{1, 2\}$, $\Psi(10) = \{1, 2, 3, 4\}$ in Figure 5). Then we let the set $\mathbb{P}(i, h, q)$ contain only the PT labels rooted at i , covering q vertices, with the condition that the child vertices of i must be selected from $\Psi(h)$. To achieve this, the following two parts are consecutively performed:

Part 1. *Initializing the deepest layer* ($h \in \{1, \dots, 6\}$ in Figure 5). In this case, we have $\Psi(h) = \{h\}$. The PTs in $\mathbb{P}(i, h, q)$ can have exactly one subtree rooted at h and hanging from i , which are generated from the existing PT labels at the topmost layer $\mathbb{P}(j, M, q - 1)$ associated with each vertex $j \in \mathcal{V}^+ \setminus \{i\}$ and capacity $q - 1$. Note that M represents the indicated number of the root node in the binary tree, so we have $\Psi(M) = \mathcal{V}^+ \setminus \{i\}$. Therefore, the set $\mathbb{P}(j, M, q - 1)$ contains all the non-dominated PT labels rooted at j , with capacity $q - 1$. This part corresponds to (ii) of recursion (8). Figure 6a shows the label generation process.

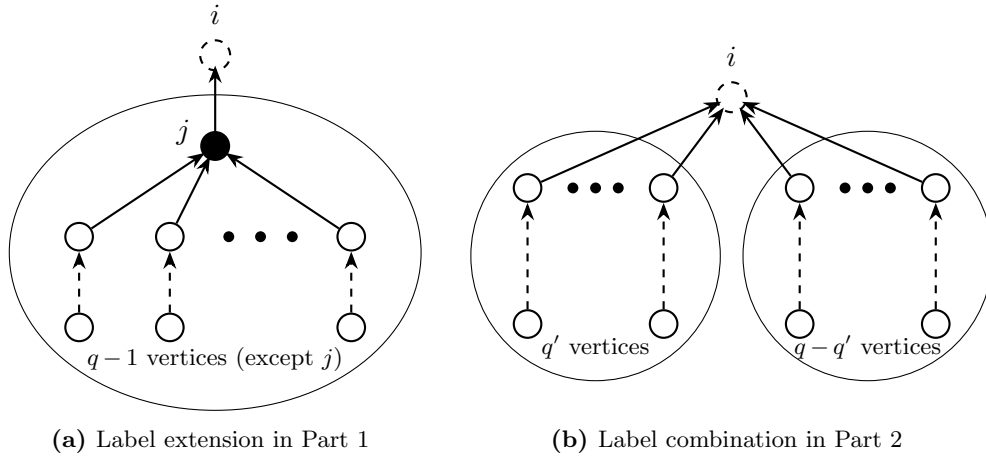


Figure 6 Two Ways of Label Generation in Dichotomy Labeling

Part 2. *Constructing the remaining layers* ($h \in \{7, \dots, 12\}$ in Figure 5). In this case, $\Psi(h)$ has multiple elements, the PTs in $\mathbb{P}(i, h, q)$ are generated by the following two steps:

Step 1. PT labels in $\mathbb{P}(i, h, q)$ are transferred from the one or two child nodes of h in the binary tree. For example, part of the labels in $\mathbb{P}(i, 10, q)$ in Figure 5 are transferred from $\mathbb{P}(i, 7, q)$ and $\mathbb{P}(i, 8, q)$. This step corresponds to (i) of recursion (8).

Step 2. PT labels in $\mathbb{P}(i, h, q)$ are newly generated and combined from PTs in the two child nodes of h in the binary tree with smaller capacity. For example, part of the labels in $\mathbb{P}(i, 10, q)$ in Figure 5 are combined from labels in $\mathbb{P}(i, 7, q - q')$ and $\mathbb{P}(i, 8, q')$ with $1 \leq q' \leq q - 1$. This step corresponds to (iii) of recursion (8). Figure 6b shows the label generation process.

Algorithm 1 Dichotomy labeling procedure

```

1:  $\mathbb{P}(i, h, q) \leftarrow \emptyset, \forall i \in \mathcal{V}^+, h = 1, 2, \dots, M, q = 0, 1, \dots, \lfloor \frac{Q}{2} \rfloor$ 
2: Create a initial PT label  $P_i = (0, 0, 0, \{0\}_{S \in \bar{\Gamma}_2}, \emptyset)$  and add it to  $\mathbb{P}(i, M, 0)$  for each  $i \in \mathcal{V}^+$ 
3: for  $q = 1, 2, \dots, Q - 1$  do
4:   for  $i \in \mathcal{V}^+$  do
5:     // Part 1 : Initializing the deepest layer
6:     for  $h = 1, 2, \dots, |\mathcal{V}^+| - 1$  do
7:       Select the  $h$ -th vertex  $j$  in set  $\mathcal{V} \setminus \{i\}$ 
8:       for  $P_j \in \mathbb{P}(j, M, q - 1)$  do
9:         if  $i \notin V(P_j)$  then
10:          Extend  $P_j$  through arc  $(j, i)$  to create a new PT label  $P_i$ 
11:          if  $P_i$  is not dominated then
12:            Add  $P_i$  to  $\mathbb{P}(i, h, q)$ 
13:       // Part 2 : Constructing the remaining layers
14:       Initialize starting point  $st \leftarrow 0$ 
15:       for  $g = 2, \dots, g_{max}$  do
16:         for  $h = 1, \dots, d_g$  do
17:           // Step 1
18:           if  $2h - 1 = d_{g-1}$  then
19:             Add all the nondominated PT labels in  $\mathbb{P}(i, st + d_{g-1}, q)$  to  $\mathbb{P}(i, st + d_{g-1} + h, q)$ 
20:           else
21:             Add all the nondominated PT labels in  $\mathbb{P}(i, st + 2h - 1, q)$  to  $\mathbb{P}(i, st + d_{g-1} + h, q)$ 
22:             Add all the nondominated PT labels in  $\mathbb{P}(i, st + 2h, q)$  to  $\mathbb{P}(i, st + d_{g-1} + h, q)$ 
23:           // Step 2
24:           for  $q' = 1, \dots, q - 1$  do
25:             for  $P_i^1 \in \mathbb{P}(i, st + 2h - 1, q')$  do
26:               for  $P_i^2 \in \mathbb{P}(i, st + 2h, q - q')$  do
27:                 if  $V(P_i^1) \cap V(P_i^2) = \emptyset$  then
28:                   Join  $P_i^1$  and  $P_i^2$  together to create a new PT label  $P_i$ 
29:                   if  $P_i$  is not dominated then
30:                     Add  $P_i$  to  $\mathbb{P}(i, st + d_{g-1} + h, q)$ 
31:           Update  $st \leftarrow st + d_{g-1}$ 

```

The pseudo code of *dichotomy labeling* is described in Algorithm 1, which enumerates all the PTs in ascending order of q . The algorithm starts with a set of initially empty PT labels. For each vertex $i \in \mathcal{V}^+$ and for each capacity q from 1 to $\lfloor \frac{Q}{2} \rfloor$, the above two parts are performed to construct the PT label set $\mathbb{P}$. Note that the dichotomy extension is implemented according to the integers d_g , $g = 2, \dots, g_{max}$, and the update of the variable st (Line 31 of Algorithm 1). Section EC.3.2 gives an example to show how the binary tree in Figure 5 is constructed by Algorithm 1.

Next, we describe the extension functions in the labeling algorithm. The extension functions used in Line 10 of Part 1 in Algorithm 1 are as follows:

$$\bar{c}(P_i) = \bar{c}(P_j) + c_{j,i} - \mu_j \quad q(P_i) = q(P_j) + 1 \quad r(P_i) = q(P_j) + 1 \quad V(P_i) = V(P_j) \cup \{j\} \quad (10)$$

The extension functions used in Line 28 of Part 2 in Algorithm 1 are as follows :

$$\bar{c}(P_i) = \bar{c}(P_i^1) + \bar{c}(P_i^2) \quad q(P_i) = q(P_i^1) + q(P_i^2) \quad r(P_i) = \min\{P_i^1, P_i^2\} \quad V(P_i) = V(P_i^1) \cup V(P_i^2) \quad (11)$$

To speed up the *dichotomy labeling*, we apply a dominance criterion to identify and eliminate dominated labels that cannot lead to the optimal PT.

DOMINANCE 1. In CMST-E case, given two PT labels $P_i^1 = (\bar{c}(P_i^1), q(P_i^1), r(P_i^1), V(P_i^1))$ and $P_i^2 = (\bar{c}(P_i^2), q(P_i^2), r(P_i^2), V(P_i^2))$ associated with the same root vertex i and the same indicated number, P_i^1 can dominate P_i^2 if

$$\bar{c}(P_i^1) \leq \bar{c}(P_i^2) \quad q(P_i^1) \leq q(P_i^2) \quad V(P_i^1) \subseteq V(P_i^2) \quad (12)$$

Note that the *dichotomy labeling* extends PT labels with capacity from 1 to $\lfloor \frac{Q}{2} \rfloor$. PTs with larger capacity, i.e., $\lfloor \frac{Q}{2} \rfloor < q < Q$, will be generated in the subsequent joining procedure by using the attribute $r(P)$ (following part 2 of Proposition 2).

Finally, we close this section by stating that the average *lifetime* of labels in *dichotomy labeling* depends on the depth of the binary tree; therefore, we have :

PROPOSITION 3. *The average lifetime of labels in dichotomy labeling is $\mathcal{O}(\log_2(|\mathcal{V}^+|))$, which is much smaller than that in sequence labeling, $\mathcal{O}(|\mathcal{V}^+|)$.*

5.1.2. Backward Labeling for Real Tree The backward labeling algorithm relies on the PT labels generated by the *dichotomy labeling*. A RT label R_i represents a RT that has the following three features: (i) capacity $q \leq \lfloor \frac{Q}{2} \rfloor$ (according to part 1 of Proposition 2); (ii) hanging from the *Root* 0; and (iii) the core vertex i . Note that in this labeling algorithm, new RT labels can only be created by joining the existing PTs and RTs through core vertices or directly extending the PTs to *Root*.

We formally define a RT label associated with a core vertex i as $R_i = (\bar{c}(R_i), q(R_i), V(R_i))$, where

- $\bar{c}(R_i)$ is the reduced cost of the RT;
- $q(R_i)$ is the number of vertices the RT covers;
- $V(R_i)$ is the set of vertices covered by the RT.

Define $\mathbb{R}(i, q)$ as a pool of RT labels where i is the core vertex and q is the capacity. The details of the backward labeling algorithm are presented in Algorithm 2, which enumerates all the RTs in ascending order of capacity q .

Algorithm 2 Backward labeling procedure

```

1:  $\mathbb{R}(i, q) \leftarrow \emptyset, \forall i \in \mathcal{V}^+, q = 1, \dots, \lceil \frac{Q}{2} \rceil$ 
2: for  $q = 1, \dots, \lceil \frac{Q}{2} \rceil$  do
3:   for  $i \in \mathcal{V}^+$  do
4:     //Part 1 : Generating RTs labels directly using PTs labels
5:     for  $P_i \in \mathbb{P}(i, M, q-1)$  do
6:       Extend PT label  $P_i$  through arc  $(i, 0)$  to create a new RT label  $R_i$ 
7:       Add  $R_i$  to  $\mathbb{R}(i, q)$ 
8:     // Part 2 : Generating RTs labels by combing PTs and RTs
9:     for  $q' = 0, \dots, q-2$  do
10:      for  $P_i \in \mathbb{P}(i, M, q')$  do
11:        for  $j \in \mathcal{V}^+ \setminus \{i\}$  do
12:          for  $R_j \in \mathbb{R}(j, q - (q' + 1))$  do
13:            if  $V(P_i \cup \{i\}) \cap V(R_j) = \emptyset$  then
14:              Combine  $P_i$  and  $R_j$  together through  $(i, j)$  to create a new RT label  $R_i$ 
15:              if  $R_i$  is not dominated then
16:                Add  $R_i$  to  $\mathbb{R}(i, q)$ 

```

The new RT labels generated by the backward labeling algorithm include activating the root vertex of existing PTs and extending them to *Root* 0 or to existing RTs. Specifically, the former extension functions in Line 6 are as follows :

$$\bar{c}(R_i) = \bar{c}(P_i) + c_{i,0} - \mu_i \quad q(R_i) = q(P_i) + 1 \quad V(R_i) = V(P_i) \cup \{i\} \quad (13)$$

The later extension functions in Line 14 are as follows :

$$\bar{c}(R_i) = \bar{c}(P_i) + \bar{c}(R_j) - \mu_j + c_{i,j} \quad q(R_i) = q(P_i) + q(R_j) + 1 \quad V(R_i) = V(P_i) \cup V(R_j) \cup \{i\} \quad (14)$$

An example of extension operation in backward labeling can be found in Figure 7. Furthermore, the dominance criterion (Dominance 1) can also be used to accelerate this labeling algorithm.

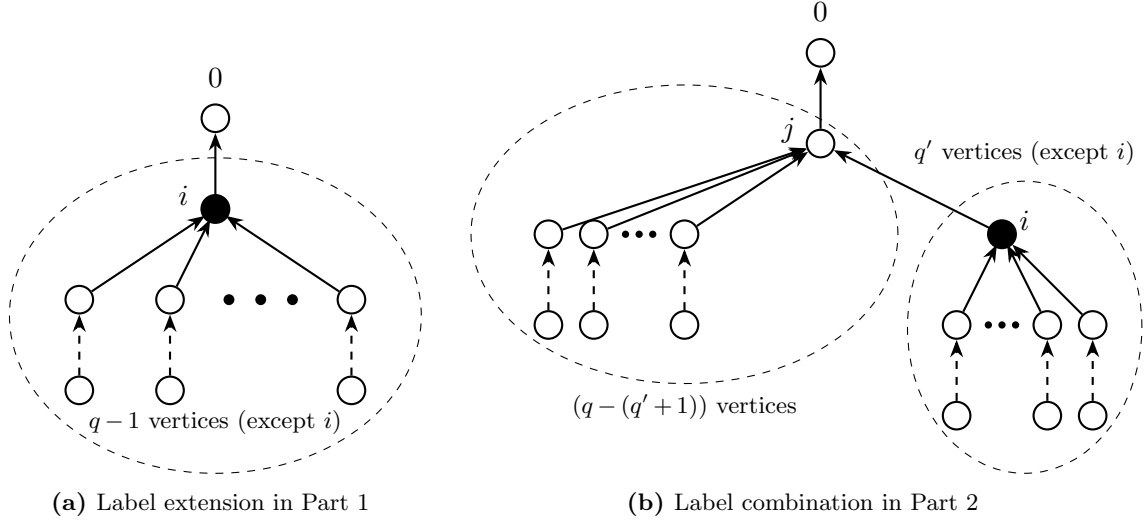


Figure 7 Two Ways of Label Generation in backward Labeling

PROPOSITION 4. *Any non-dominated pseudo-tree and real-tree can be generated from Algorithm 1 and 2, respectively.*

5.1.3. Procedure for Elementary Subtree Now, by the two labeling algorithms described in Section 5.1.1 and Section 5.1.2, we can obtain all the non-dominated PTs and RTs from the set $\bigcup_{i \in \mathcal{V}^+, q \in \{1, \dots, \lfloor \frac{Q}{2} \rfloor\}} \mathbb{P}(i, M, q)$ and $\bigcup_{i \in \mathcal{V}^+, q \in \{1, \dots, \lceil \frac{Q}{2} \rceil\}} \mathbb{R}(i, q)$, respectively. Note that the joined *elementary subtree* has a maximum capacity of Q , so any PT with larger capacity ($q > \lfloor \frac{Q}{2} \rfloor$) must have at least two subtrees hanging from the core vertex (part 2 of Proposition 2), in other words, these PTs must be generated through combination operation. Algorithm 3 shows this procedure.

Given all the PTs and RTs required by Proposition 2, we join them by the core vertex $i \in \mathcal{V}^+$ to obtain *elementary subtrees*. Define a label $E_i = (\bar{c}(E_i), q(E_i))$ to collect the joined *elementary subtrees* with core vertex i , and let $\mathbb{E}_i$ be its pool, where $\bar{c}(E_i)$ denotes its reduced cost and $q(E_i)$ represents its capacity. Algorithm 4 presents the details of the joining procedure.

The extension functions to create new labels in Line 8 are as follows :

$$\bar{c}(E_i) = \bar{c}(P_i) + \bar{c}(R_i) \quad q(E_i) = q(P_i) + q(R_i) \quad (15)$$

Only the *elementary subtrees* with capacity from $\lfloor \frac{Q}{2} \rfloor + 2$ to Q need to be generated in Algorithm 4. For those subtrees whose capacity is less than $\lfloor \frac{Q}{2} \rfloor + 2$, they can be directly obtained from the PT labels with capacity $0 \leq q \leq \lfloor \frac{Q}{2} \rfloor$ by activating their root vertices and connecting them to the Root 0. Finally, we can obtain the optimal *elementary subtree* from the label E_i in $\bigcup_{i \in \mathcal{V}^+} \mathbb{E}_i$ that minimizes $\bar{c}(E_i)$.

Algorithm 3 Procedure to generate PTs with larger capacity

```

1:  $\mathbb{P}(i, M, q) \leftarrow \emptyset, \forall i \in \mathcal{V}^+, q = \lfloor \frac{Q}{2} \rfloor + 1, \dots, Q - 1$ 
2: for  $q = \lfloor \frac{Q}{2} \rfloor + 1, \dots, Q - 1$  do
3:   for  $i \in \mathcal{V}^+$  do
4:     Initialize starting point  $st \leftarrow 0$ 
5:     for  $g = 2, \dots, g_{max}$  do
6:       if 1 then
7:         for  $h = 1, \dots, d_g$  do
8:           for  $q' = q - \lfloor \frac{Q}{2} \rfloor + 1, \dots, \lfloor \frac{Q}{2} \rfloor - 1$  do
9:             for  $P_i^1 \in \mathbb{P}(i, st + 2h - 1, q')$  do
10:            for  $P_i^2 \in \mathbb{P}(i, st + 2h, q - q')$  do
11:              if  $V(P_i^1) \cap V(P_i^2) = \emptyset \wedge r(P_i^1) \geq q' - \lfloor \frac{Q}{2} \rfloor + 1 \wedge r(P_i^2) \geq q - q' - \lfloor \frac{Q}{2} \rfloor + 1$  then
12:                Join  $P_i^1$  and  $P_i^2$  together to create a new PT label  $P_i$ 
13:                Add  $P_i$  to  $\mathbb{P}(i, M, q)$ 
14:            Update  $st \leftarrow st + d_{g-1}$ 

```

Algorithm 4 PT-RT joining procedure

```

1:  $\mathbb{E}_i, \leftarrow \emptyset \forall i \in \mathcal{V}^+$ 
2: for  $i \in \mathcal{V}^+$  do
3:   for  $q = \lfloor \frac{Q}{2} \rfloor + 2, \dots, Q$  do
4:     for  $q' = \lfloor \frac{q}{2} \rfloor, \dots, Q - 1$  do
5:       for  $P_i \in \mathbb{P}(i, M, q')$  do
6:         for  $R_i \in \mathbb{R}(j, q - q')$  do
7:           if  $V(P_i) \cap V(R_i) = \emptyset \wedge r(P_i) \geq q' - \lfloor \frac{q}{2} \rfloor + 1$  then
8:             Combine  $P_i$  and  $R_i$  together to create a elementary subtree label  $E_i$ 
9:             Add  $E_i$  to  $\mathbb{E}_i$ 

```

5.2. Branching Strategies

here We design two branching rules to generate child nodes in the branch-and-bound tree, both of which are robust and cannot increase the complexity of solving the pricing subproblem. The first rule in our PR-BPC concerns the number of *elementary subtrees* hanging from *Root* 0. Specifically, for a given branch-and-bound node, if the value $\sum_{\lambda \in \Lambda} \bar{x}_\lambda$ in the optimal solution $\bar{\mathbf{x}}$ is not integral, the disjunction $\sum_{\lambda \in \Lambda} x_\lambda \leq \lfloor \sum_{\lambda \in \Lambda} \bar{x}_\lambda \rfloor$ and $\sum_{\lambda \in \Lambda} x_\lambda \geq \lceil \sum_{\lambda \in \Lambda} \bar{x}_\lambda \rceil$ are imposed for the two child nodes, respectively.

The second rule branches on arcs. Define a binary decision variable $\bar{y}_{i,j}$ that is equal to 1 if arc $(i, j) \in \mathcal{A}$ is used in the solution and 0 otherwise. For one branch-and-bound node with optimal solution $\bar{\mathbf{x}}$, $\bar{y}_{i,j}$ can be obtained by setting $\bar{y}_{i,j} = \sum_{\lambda \in \Lambda} \beta_{i,j,\lambda} \bar{x}_\lambda$. If all the $\bar{\mathbf{y}}$ in the optimal solution

are integral, it automatically leads to an integral solution. Otherwise, the solution must be further explored by selecting a fractional $\bar{y}_{i,j}$ to branch on. For an arc (i^*, j^*) with a fractional $\bar{y}_{i^*, j^*}$, we create two child nodes by fixing $\bar{y}_{i^*, j^*} = 1$, implying that arc (i^*, j^*) must be used, and $\bar{y}_{i^*, j^*} = 0$, implying that arc (i^*, j^*) is forbidden. In the former case, we delete from $\mathcal{A}$ all arcs $(i^*, h^*) \in \mathcal{A}$ with $h^* \neq j^*$. In the latter case, we delete arc (i^*, j^*) from $\mathcal{A}$. To select which arcs to branch on, we adopt a strong branching strategy proposed by [Uchoa et al. \(2008\)](#) and [Ralphs et al. \(2003\)](#), which has been proven to work very well on CMST-E ([Luo et al. 2021](#)).

5.3. Valid Inequalities

The linear programming (LP) relaxation of the GF model provides a lower bound for CMST. To further strengthen the lower bound, we introduce the following two families of valid inequalities, namely the rounded capacity inequality (RCI) and the subset-row inequality (SRI), which are frequently applied in BPC algorithms for various applications, e.g., vehicle-routing problems ([Pecin et al. 2017](#)) and CMST-E ([Uchoa et al. 2008](#), [Luo et al. 2021](#)).

According to the classification proposed in [de Aragao and Uchoa \(2003\)](#), the RCI is one type of *robust cuts*, as their dual values can be translated into costs in the pricing subproblem. Therefore, the structure and the size of that subproblem remain unaltered, regardless of the number of such cuts added. For any set $\mathcal{S} \subseteq \mathcal{V}^+$, we let $\delta^+(\mathcal{S}) = \{(i, j) \in \mathcal{A} \mid i \in \mathcal{S}, j \in \mathcal{V} \setminus \mathcal{S}\}$ be the set of arcs leaving vertices in set $\mathcal{S}$. The RCI is defined as follows:

$$\sum_{(i,j) \in \delta^+(\mathcal{S})} \sum_{\lambda \in \Lambda} \beta_{i,j,\lambda} x_\lambda \geq \left\lceil \frac{|\mathcal{S}|}{Q} \right\rceil, \quad \forall \mathcal{S} \in \Gamma_1 \quad (16)$$

where each element of Γ_1 corresponds to an individual RCI. To separate RCIs and obtain the set Γ_1 , we sequentially use three heuristics, namely the partial enumeration scheme proposed by [Archetti et al. \(2011\)](#), the shrinking heuristic proposed by [Desaulniers \(2010\)](#), and the route-based connected component heuristic proposed by [Ralphs et al. \(2003\)](#).

As one type of *non-robust cuts*, SRI can change the size of the pricing subproblem; each additional cut makes it harder. For any set $\mathcal{S} \subseteq \mathcal{V}^+$ and an integer p ($1 < p \leq |\mathcal{S}|$), the SRI is defined as follows:

$$\sum_{\lambda \in \Lambda} \left\lfloor \frac{1}{p} \sum_{i \in \mathcal{S}} \alpha_{i,\lambda} \right\rfloor x_\lambda \leq \left\lfloor \frac{|\mathcal{S}|}{p} \right\rfloor, \quad \forall \mathcal{S} \in \Gamma_2 \quad (17)$$

where each element of Γ_2 corresponds to an individual SRI. To control the impact of SRI in the pricing, we only consider the SRIs with $p = 2$, $|\mathcal{S}| = 3$ and $p = 2$, $|\mathcal{S}| = 5$. As p and $|\mathcal{S}|$ are small numbers, the set Γ_2 and the corresponding SRIs can be obtained by enumeration.

5.4. Accelerating Strategies for Pricing

5.4.1. Heuristic Pricing In the initial stage of the column generation procedure, the number of columns in the restricted master problem is often very small. To quickly search for high-quality columns and stabilize the dual values, we first execute a heuristic *dichotomy labeling* procedure to generate PTs with capacities ranging from 1 to $Q - 1$ before implementing the exact labeling algorithm. The dominance rule of this heuristic drops the impact of elementary elements $V(P_i)$ from the definition of P_i , retaining at most one label with the lowest reduced cost in each set $\mathbb{P}$. Therefore, this heuristic can be performed very efficiently.

5.4.2. Arc Elimination We use a well-known **arc elimination rule** (Gouveia and Lopes 2005) to remove arcs in $\mathcal{A}$ that must not be included in the optimal solution. For any arc $(i, j) \in \mathcal{A}$ with $i \neq 0, j \neq 0$, if $c_{i,j} > c_{i,0}$, arc (i, j) can be removed safely. **Suppose that there exist a feasible solution $\mathbf{x}^*$ containing arc (i^*, j^*) such that $(i^*, j^*) > (i^*, 0)$. We can modify $\mathbf{x}^*$ to a solution with smaller cost by deleting arc (i^*, j^*) , and add the arc $(i^*, 0)$ to form a new rooted-subtree with vertex i^* alone. Therefore, solution $\mathbf{x}^*$ must be non-optimal.** However, this rule is not valid for CMST-F as directly connecting the vertex i^* to the *Root* 0 may violate the side constraints (5). **We show that this rule is effective for both CMST-E and CMST-L instances, removing on average 33.56% and 47.07% of their arcs, respectively.**

5.4.3. Label Pruning To further accelerate the labeling algorithm, we adopt a pruning strategy to eliminate redundant labels that cannot lead to optimal solutions. Let $\Delta^{pt}(i, q)$ denote a lower bound on the reduced cost of PT labels rooted at vertex i with capacity q . Similarly, let $\Delta^{rt}(i, q)$ be a lower bound on the reduced cost of RT labels with core vertex i and capacity q . A PT label P_i (respectively, RT label R_i) can be safely discarded if $\bar{c}(P_i) + \Delta^{rt}(i, q') > 0$ for all $q' \in \{Q - q(P_i), \dots, Q\}$ (respectively, $\bar{c}(R_i) + \Delta^{pt}(i, q') > 0$ for all $q' \in \{Q - q(R_i), \dots, Q\}$). Obviously, the effectiveness of the pruning strategy heavily depends on the lower bounds $\Delta^{pt}(i, q)$ and $\Delta^{rt}(i, q)$. The stronger the lower bounds are, the more labels will be pruned; however, the time spent computing these bounds also increases.

The *ng-arb* structure described in Section 4.2 achieve an excellent balance between solution speed and lower bound quality, which is more effective than the *q-arb* pruning proposed in Luo et al. (2021). So, we use the *ng-arb* to compute the bound $\Delta^{pt}(i, q)$ and $\Delta^{rt}(i, q)$. Specifically, the lower bound $\Delta^{pt}(i, q)$ can be computed using the recursion (8) together with the dual values $\{\mu\}_{i \in \mathcal{V}^+}$, shown as follows ($c_{i,j}$ in the recursion is replaced by $\bar{c}_{i,j} = c_{i,j} - \mu_i$) :

$$\Delta^{pt}(i, q) = \min_{NG \subseteq N_i, NG \ni i} \{f(NG, i, |\mathcal{V}^+|, q + 1)\} + \mu_i \quad (18)$$

Similarly, $\Delta^{rt}(i, q)$ can be computed by backward *ng-arb*, which is similar to the definition of RTs. Let $f^{-1}(NG, i, j, q)$ be the cost of the least cost *backward ng-tree* (NG, i, j, q) . The recursion for it is described in Section EC.5.1. Then, we have :

$$\Delta^{rt}(i, q) = \min_{NG \subseteq N_i, NG \ni i} \{f^{-1}(NG, i, |\mathcal{V}^+|, q)\} \quad (19)$$

Compared with the *ng-arb* pruning, *q-arb* requires significantly less time, which is typically negligible. Therefore, we run *q-arb* pruning first to generate weaker lower bounds, and then use them to accelerate the *ng-arb* computation. Furthermore, the bound quality of $\Delta^{rt}(i, q)$ and $\Delta^{pt}(i, q)$ can be further improved by considering the penalizations of SRIs, but this requires more computational effort. To trade off bound quality against computational effort, only the $\tau\%$ of SRIs with the smallest dual values will be activated.

5.4.4. Decremental State-Space Relaxation The decremental state-space relaxation (DSSR) is an iterative technique introduced by Boland et al. (2006) and Righini and Salani (2008), used to accelerate a label setting algorithm. Denote by $\mathcal{I}$ the set of vertices that a subtree cannot cover more than once. The label-setting algorithm with DSSR first fully relaxes the *elementary* requirement and solves the pricing problem, i.e., it drops the attributes $V(P_i)$ and $V(R_i)$, and imposes $\mathcal{I} = \emptyset$. If in the least-cost subtree each vertex appears at most once, we obtain an optimal *elementary subtree* for the original pricing problem; then the algorithm terminates. Otherwise, we solve the pricing problem again after adding to $\mathcal{I}$ the set of vertices that have been covered more than once. Namely, in the next iteration, more vertices are forbidden to appear more than once in the least-cost subtree. This process repeats until an optimal solution of the pricing problem is found. The set $\mathcal{I}$ enlarges in every iteration, which ensures that the procedure eventually terminates.

5.5. Upper Bound Calculation

We generate an upper bound for the three variants of CMST to prune nodes in the branch-and-bound tree. Specifically, for CMST-E and CMST-F, we employ the memetic algorithm proposed by Luo et al. (2021) as an external upper bound procedure. For CMST-L, we use a simple column-based heuristic, which collects the columns obtained at the root node of the branch-and-bound tree and feeds them directly to the integer programming (IP) model GF . The optimal cost of this model serves as an upper bound.

6. Tailoring the PR-BPC to Solve the Other Two Variants of CMST

As mentioned in Section 3.1, CMST-E can be extended to other two variants by changing the cost structure and adding side constraints. The differences in cost structures of subtrees have a minor effect on *dichotomy labeling* but a significant effect on backward labeling. In this section, we customize dedicated pricing subproblems for CMST-L and CMST-F respectively, and present detailed modifications to the backward labeling algorithm of pricing.

6.1. Modification of PR-BPC for CMST-L

6.1.1. Modification on Pricing Problem Recall that in CMST-L, there exist L different links with capacities v^l , $l = 1, 2, \dots, L$, such that $0 < v^1 < v^2 < \dots < v^L = Q$, which are available to be installed on each arc $(i, j) \in \mathcal{A}$ at a cost $c_{i,j}^l$. The pricing problem PP_2 for CMST-L consists of finding a least-cost spanning tree and the best link assignment, which is as follows :

$$[PP_2] \min_{\lambda} \sum_{(i,j) \in \mathcal{A}} \beta_{i,j,\lambda} \min_{l \in \{1,2,\dots,L\}} \{c_{i,j}^l | v^l \geq f_{i,j}\} - \sum_{i \in \mathcal{V}^+} \alpha_{i,\lambda} \mu_i \quad (20)$$

s.t. λ is a feasible *elementary subtree*

where $f_{i,j}$ represents the amount of flow on arc (i, j) . Compared with PP_1 (9), only the first term of PP_2 (20) is changed. For the optimal *elementary subtree*, the best link for each arc is the minimum-cost link whose capacity is sufficient to cover the flow on that arc.

6.1.2. Modification on Labeling Algorithm *Dichotomy* and backward labeling algorithms require modifications to adapt to changes in the cost structure of CMST-L. Specifically, because the extension in *dichotomy* labeling is performed at the root of a PT label, the amount of flow on the extended arc can be predetermined (see Figure 6). Therefore, only small changes to the reduced cost extension functions in (10) are needed :

$$\bar{c}(P_i) = \bar{c}(P_j) + \min_{l \in \{1,2,\dots,L\}} \{c_{j,i}^l | v^l \geq q(P_j) + 1\} - \mu_j \quad (21)$$

The Dominance 1 is also valid for the modified *dichotomy* labeling algorithm to accelerate computation. However, for the backward labeling, an RT label is created by combining PT labels and existing RT labels. If the amount of flow into the RT via the core vertex cannot be determined in advance, the reduced cost for that RT cannot be correctly calculated. Thus, in the CMST-L case, we add an element $w(P_i)$ to the label definition, meaning the amount of extra flow into the core vertex i . Let $\mathbb{R}(i, q, w)$ denote the pool of RT labels with core vertex i , capacity q , and w extra flows. Algorithm 5 presents the detailed procedure for backward labeling in CMST-L, which enumerates all RTs in descending order of w . The minimum value of w in Line 1 ($\left\lfloor \frac{\lfloor \frac{Q}{2} \rfloor + 2}{2} \right\rfloor$) indicates the

minimum capacity of a PT (minimum extra flow) required to combine the *elementary subtree* with capacity $\lfloor \frac{Q}{2} \rfloor + 2$. The value $\lfloor \frac{Q}{2} \rfloor + 2$ is the smallest capacity of an *elementary subtree* that needs to be generated by joining PTs and RTs. In addition, Proposition 5 shows that for different values of w , the capacity of an RT is limited to the set $\mathcal{S}_w = [\max\{1, \lfloor \frac{Q}{2} \rfloor + 2 - w\}, \min\{Q - w, w + 1\}] \cap \mathbb{Z}$. Therefore, Line 5 of Algorithm 5 stipulates that the capacity must be selected from this set. An example of the construction of set $\mathcal{S}_w$ is given in EC.3.3.

Algorithm 5 Backward labeling algorithm for CMST-L

```

1:  $\mathbb{R}(i, q, w) \leftarrow \emptyset, \forall i \in \mathcal{V}^+, q = 1, \dots, \lceil \frac{Q}{2} \rceil, w = \lfloor \frac{\lfloor \frac{Q}{2} \rfloor + 2}{2} \rfloor, \dots, Q - 1$ 
2: for  $w = Q - 1, \dots, \lfloor \frac{\lfloor \frac{Q}{2} \rfloor + 2}{2} \rfloor$  do
3:   for  $i \in \mathcal{V}^+$  do
4:     //Part 1 : Generating RT labels directly using PT labels
5:     for  $q \in \mathcal{S}_w$  do
6:       for  $P_i \in \mathbb{P}(i, M, q - 1)$  do
7:         Extend PT label  $P_i$  through arc  $(i, 0)$  to create a new RT label  $R_i$ 
8:         Add  $R_i$  to  $\mathbb{R}(i, q, w)$ 
9:     // Part 2 : Generating RT labels by combining PT and RT
10:    for  $w' = 1, \dots, Q - w - 1$  do
11:      if  $q - w' \in \mathcal{S}_{w+w'}$  then
12:        for  $P_i \in \mathbb{P}(i, M, w' - 1)$  do
13:          for  $j \in \mathcal{V}^+ \setminus \{i\}$  do
14:            for  $R_j \in \mathbb{R}(j, q - w', w + w')$  do
15:              if  $V(P_i \cup \{i\}) \cap V(R_j) = \emptyset$  then
16:                Combine  $P_i$  and  $R_j$  together through  $(i, j)$  to create a new RT label  $R_i$ 
17:                if  $R_i$  is not dominated then
18:                  Add  $R_i$  to  $\mathbb{R}(i, q, w)$ 

```

PROPOSITION 5. In CMST-F case, given a fixed extra flow w , the capacity of the corresponding RTs must be limited in the set $[\max\{1, \lfloor \frac{Q}{2} \rfloor + 2 - w\}, \min\{Q - w, w + 1\}] \cap \mathbb{Z}$.

In addition, the reduced cost extension functions for Line 7 and Line 16 in Algorithm 5 are modified by considering the extra flows w , as shown in (22) and (23), respectively.

$$\bar{c}(R_i) = \bar{c}(P_i) + \min_{l \in \{1, 2, \dots, L\}} \{c_{i,0}^l | v^l \geq q(P_i) + 1 + w\} - \mu_i \quad (22)$$

$$\bar{c}(R_i) = \bar{c}(P_i) + \bar{c}(R_j) + \min_{l \in \{1, 2, \dots, L\}} \{c_{i,j}^l | v^l \geq q(P_i) + 1 + w\} - \mu_i \quad (23)$$

We also derive the following dominance rule to remove non-optimal labels.

DOMINANCE 2. In CMST-L case, given two RT labels $R_i^1 = (\bar{c}(R_i^1), q(R_i^1), w(R_i^1), V(R_i^1))$ and $R_i^2 = (\bar{c}(R_i^2), q(R_i^2), w(R_i^2), V(R_i^2))$ associated with the same root vertex i , R_i^1 can dominate R_i^2 if

$$\bar{c}(R_i^1) \leq \bar{c}(R_i^2) \quad w(R_i^1) = w(R_i^2) \quad V(R_i^1) \subseteq V(R_i^2) \quad (24)$$

Note that the dominance rule $q(R_i^1) \leq q(R_i^2)$ is removed in Dominance 2 because the element $w(R_i)$ determines the capacity of the eventually formed subtree in advance.

6.1.3. Modification on Label Pruning In the CMST-L case, to include the link assignment feature in the *ng-arb* recursion (8), we need to modify the cost of a given arc $c_{i,j}$ by considering the extra flow f : $c_{i,j,f} = \min_{l \in \{1,2,\dots,L\}} \{c_{i,j}^l \mid v^l \geq f\} - \mu_i$. The modified recursion is shown in EC.5.2.

To customize the backward *ng-arb* recursion function (EC.15), we remove the capacity element from the state definition and replace it with extra flow w . In this context, the recursive function $f^{-1}(NG, i, j, w)$ to compute the least-cost backward *ng-arb* is shown in EC.5.3.

Let $\Delta^{rt}(i, w)$ denote the lower bound of the reduced cost of RT labels when the core vertex is i and the extra flow is w . A PT label P_i (respectively, an RT label R_i) can be safely discarded if $\bar{c}(P_i) + \Delta^{rt}(i, q(P_i)) > 0$ (respectively, $\bar{c}(R_i) + \Delta^{rt}(i, q(R_i)) > 0$).

6.2. Modification of PR-BPC for CMST-F

6.2.1. Modification on Pricing Problem Recall that in CMST-F, the flow cost $f_{i,j,\lambda} c_{i,j}$ between PV arrays (i, j) in subtree λ need to be considered. Furthermore, the side constraints (5) ensure that the capacity of *elementary subtrees* in the integer solution must be one of the values in the set $\{q_t\}_{t \in \mathcal{T}}$. Let η_t be the dual value of side constraints (5) for each type $t \in \mathcal{T}$. The pricing problem PP_3 for CMST-F is as follows:

$$\begin{aligned} [PP_3] \quad & \min_{\lambda} \sum_{(i,j) \in \mathcal{A}: j=0} \beta_{i,j,\lambda} c_{i,j} + \sum_{(i,j) \in \mathcal{A}: j \neq 0} \beta_{i,j,\lambda} f_{i,j,\lambda} c_{i,j} - \sum_{i \in \mathcal{V}^+} \alpha_{i,\lambda} \mu_i - \sum_{t \in \mathcal{T}} \gamma_{t,\lambda} \eta_t \\ & \text{s.t. } \lambda \text{ is a feasible elementary subtree with } q_t \text{ vertices, } t \in \mathcal{T} \end{aligned} \quad (25)$$

6.2.2. Modification on Labeling Algorithm The *dichotomy* and backward labeling algorithms for CMST-L can be transferred to CMST-F with only minimal modifications. For the *dichotomy* labeling, the first modification is on the reduced cost extension functions in (10) :

$$\bar{c}(P_i) = \bar{c}(P_j) + c_{j,i} + (q(P_j) + 1)c_{j,i} - \mu_j \quad (26)$$

The second modification is applied to the dominance rule of the *dichotomy* labeling for generating PTs. As the capacity of a subtree is limited to a small set $\{q_t\}_{t \in \mathcal{T}}$, only labels with the same capacity can dominate one another. Therefore, we modify the second term in Dominance 1 to require $q(P_i^1) = q(P_i^2)$.

For the backward labeling, the first modification is applied to the capacity set $\mathcal{S}_w$ (Line 5 and 11 of Algorithm 5) by setting $\mathcal{S}_w = \{q_t - w \mid t \in \mathcal{T}, 1 \leq q_t - w \leq \lceil q_t/2 \rceil\}$, which contains possible RT capacity values in CMST-F when the extra flow is fixed to w . The second modification is on the extension functions. When directly using PT labels to generate RT labels (Line 6-8 in Algorithm 5), the impact of the dual values of side constraints (5) must be incorporated :

$$\bar{c}(R_i) = \bar{c}(P_i) + c_{i,0} - \mu_i - \sum_{t \in \mathcal{T}, q_t = q(P_i) + 1 + w} \eta_t \quad (27)$$

where the last term represents the dual value of type- t side constraints such that q_t is equal to the capacity of the *elementary subtree* that this label can eventually form, i.e., $q_t = q(P_i) + 1 + w$.

Similarly, the extension function in Line 16 in Algorithm 5 is modified as follows :

$$\bar{c}(R_i) = \bar{c}(P_i) + \bar{c}(R_j) + (q(P_i) + 1 + w)c_{i,j} - \mu_i \quad (28)$$

For the label pruning strategy, we use *q-arb* rather than *ng-arb* to prune labels in CMST-F, because *ng-arb* does not perform well in this case. This is due to the fact that the graph $\mathcal{G}$ of CMST-F is extremely sparse, with each vertex having at most four adjacent vertices; thus, *ng-arb* provides only very limited improvement to the bounds $\Delta^{pt}(i, q)$ and $\Delta^{rt}(i, w)$ compared with that provided by *q-arb*, while requiring much more time.

7. Computational Results

In this section, we report the computational results achieved by PR-BPC on the three variants of CMST and, when available, compare its performance with state-of-the-art exact methods. The PR-BPC is implemented in Java using Gurobi 12.0.1's callbacks. We solve the LP models using the simplex algorithm implemented by Gurobi. The tests are performed on a Mac server equipped with a 4.4 GHz CPU and 32 GB RAM. A time limit of 7200 seconds per instance is set. Since the acceleration strategies described in Section 5.4 are not equally effective for all three variants, Table 1 presents, for each variant, which components are enabled and which are disabled. The value of τ and $|N_i|$ for these problems was determined through preliminary experiments.

Table 1 Algorithm Configuration for Three Variants of CMST

Problem	Heuristic Pricing	Arc Elimination	Label Pruning				DSSR
			$q\text{-arb}$	$ng\text{-arb}$	$ N_i $	τ	
CMST-E	✓	✓		✓	5	0	✓
CMST-L	✓	✓		✓	5	50	
CMST-F	✓		✓		-	0	

The first three subsections are devoted to CMST-E, CMST-L, and CMST-F, respectively. In each subsection, we first describe the instances and then present the results achieved by our solution method. The fourth subsection analyzes the strength of three important components of our PR-BPC, namely *dichotomy labeling*, PT-RT joining, and *ng-arb* pruning. Finally, the last subsection is devoted to a comparison with the literature.

Tables 2 to 4 summarize the results by group of instances, and the following general information is indicated: the name of the group of instances (Name), the number of instances (Inst), and the number of instances exactly solved (Solved). We report detailed results of PR-BPC at the root node of the branch-and-bound tree (Root) and for the overall algorithm execution (Total), respectively. For the root node, we collect: the percentage gap between the optimal LP value without any cuts and the best found integer solution (LP gap (%)), and its associated computing time (LP Time); the percentage gap between the optimal LP value with enumerated RCIs and SRIs and the best found integer solution (LPC gap (%)), and its associated computing time (LPC Time); the number of PT labels with capacity $1 \leq q \leq \lfloor \frac{Q}{2} \rfloor$ (No.P1 Label) and with $\lfloor \frac{Q}{2} \rfloor + 1 \leq q \leq Q - 1$ (No.P2 Label), respectively, in the last column generation iteration; the number of RT labels in the last column generation iteration (No.R Label); and the numbers of enumerated RCIs (No.RCI) and SRIs (No.SRI). For the overall algorithm execution, we present: the numbers of enumerated RCIs and SRIs, the number of branch-and-bound nodes visited (No.Node), the time spent in the joining procedure (Joining Time), the time spent in the label pruning procedure (Pr Time), and the overall computing time (IP Time).

All test instances used in this section are available in the online appendix.

7.1. Computational Results on the CMST-E

The benchmark set consists of 54 instances, derived from Luo et al. (2021) and Uchoa et al. (2008). There are three groups of instances (tc, te, and td), each of which is constructed by placing non-root vertices randomly on a 100×100 grid. These groups are distinguished based on the position of the *Root* vertex. The *Root* of tc instances has a central position, coordinates (50, 50); the *Root* of te instances has an eccentric position, coordinates (0, 0). Finally, the *Root* of td instances is distant from the other vertices, coordinates $(-201, -201)$. In addition, the naming of the instances includes the number of vertices and the maximum subtree capacity. For example, "tc-80-5" refers to tc group instance with 80 vertices and maximum capacity $Q = 5$.

Table 2 summarizes computational results on CMST-E. It shows that PR-BPC can solve all 54 instances with an average (maximum) running time of 144.62 (726.29) seconds. Incorporating RCIs and SRIs improves the lower bound considerably, reducing the average gap from 0.69% to 0.08%, at the expense of increasing the average computation time from 2.10 to 28.68 seconds. Furthermore,

Table 2 Detailed Experimental Results for CMST-E

			Root									Total					
Name	Inst	solved	LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	IP Time
tc-80-5	5	5	0.90	0.22	0.12	7.66	116	0	98	84	28	89	44	1	0.01	0.08	34.61
te-80-5	5	5	0.63	0.32	0.08	6.17	165	0	190	92	44	98	64	3	0.01	0.13	33.78
td-80-5	5	5	0.17	0.72	0.01	9.40	205	0	258	90	56	90	56	1	0.01	0.11	35.54
tc-120-5	5	5	1.30	0.36	0.08	22.53	568	0	698	145	136	163	164	4	0.01	0.49	71.24
te-120-5	5	5	0.46	1.35	0.06	22.16	411	0	554	138	104	143	112	3	0.01	0.42	68.20
tc-160-5	1	1	0.66	1.34	0.04	40.25	543	0	887	316	120	316	120	1	0.01	0.78	162.82
te-160-5	1	1	0.40	2.06	0.06	53.63	599	0	833	184	180	202	200	15	0.06	1.54	222.94
tc-80-10	5	5	0.46	1.64	0.08	14.43	3397	2046	2720	47	96	47	96	1	0.08	0.82	118.91
te-80-10	5	5	0.65	2.10	0.03	11.98	5143	817	6133	45	120	45	120	1	0.09	0.93	116.16
td-80-10	5	5	0.28	3.17	0.05	25.29	10225	1299	15071	41	152	47	260	5	5.17	5.98	198.87
tc-120-10	5	5	1.63	3.63	0.21	68.23	12692	8548	19174	127	288	139	408	5	3.54	7.49	289.94
te-120-10	5	5	0.48	4.89	0.09	38.02	7498	747	7448	63	232	66	324	3	0.60	7.71	235.35
tc-160-10	1	1	0.85	7.64	0.09	157.91	15210	2562	46305	123	500	127	560	3	3.46	12.14	684.27
te-160-10	1	1	0.68	10.07	0.12	167.36	17666	1415	90535	117	480	128	560	5	2.84	14.49	726.29
Average	54	54	0.69	2.10	0.08	28.68	4373	1320	7413	95	140	101	179	3	1.00	2.77	144.62

in the $Q = 5$ case, PTs with capacities 3 and 4 are forbidden to be generated in Algorithm 3 due to Proposition 2. Thus, the value of No.P2 Label in this case is shown to be zero.

7.2. Computational Results on the CMST-L

We test PR-BPC on the 450 instances derived from Uchoa et al. (2012). These instances have 50, 100, and 150 vertices, besides the root vertex, randomly distributed in a 40×40 square grid. Three different links are available for every arc: $v^1 = 1$, $v^2 = 3$, and $v^3 = Q = 10$. For each arc $(i, j) \in \mathcal{A}$, the cost $c_{i,j}^1$ of assigning the first link is equal to the Euclidean distance between vertices i and j , and the costs of assigning the second and third links are $c_{i,j}^2 = 2c_{i,j}^1$ and $c_{i,j}^3 = 3c_{i,j}^1$, respectively. There are three classes of instances for each size, according to the location of the root vertex: in the center (c), at the edge (e), and randomly located (r). Therefore, the 450 CMST-L instances can be divided into the following nine series of 50 instances with the same size and class: 50c, 50e, 50r, 100c, 100e, 100r, 150c, 150e, and 150r.

Table 3 Detailed Experimental Results for CMST-L

Root												Total					
Name	Inst	solved	LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	IP Time
50c	50	50	0.64	0.08	0.20	0.58	4605	2249	2780	6	79	13	362	40	0.66	5.15	11.63
50e	50	47	0.46	1.06	0.10	23.86	28610	33104	13678	20	184	47	783	29	69.63	488.39	940.06
50r	50	50	0.54	0.35	0.11	6.71	12349	11732	6554	12	118	19	342	16	21.10	192.75	310.19
100c	50	50	0.51	1.44	0.07	47.09	35282	32312	17891	36	316	46	778	24	34.01	299.67	545.44
100e	50	19	0.43	3.69	0.19	105.36	56850	63919	27916	34	344	94	1686	38	858.95	1293.47	5145.91
100r	50	38	0.42	2.50	0.12	81.40	42889	45478	21080	29	305	83	1606	51	638.08	810.01	2738.18
150c	50	31	0.45	6.32	0.09	314.15	74924	83804	37339	44	471	77	1733	42	710.12	1036.96	3626.07
150e	50	13	0.43	8.38	0.27	292.50	84517	86026	42608	34	472	116	2538	49	951.30	1662.39	5919.14
150r	50	19	0.39	9.81	0.14	245.40	69675	64504	35459	39	445	110	2315	51	964.81	1096.62	5417.66
Average	450	317	0.47	3.73	0.14	124.12	45522	47014	22812	28	304	67	1349	38	472.07	765.05	2739.37

Table 3 shows that 317 out of 450 instances can be solved to optimality within the time limit of 7200 seconds. From the column "IP Time" in the table, it can be seen that "c" instances are

the easiest to solve (1394.38 seconds on average), "r" instances are moderately difficult (2822.01 seconds on average), and "e" instances are the most difficult (4001.70 seconds on average), which is consistent with the results reported in [Uchoa et al. \(2012\)](#). Unlike CMST-E, CMST-L relies more on SRIs to improve the lower bound. The number of SRIs enumerated is 20.13 times that of RCIs. However, too many SRIs significantly slow down the pricing process. Therefore, we set the value of τ to 0.5 to strengthen the bound quality obtained by *ng-arb*, at the expense of longer pruning time (765.05 seconds on average).

7.3. Computational Results on the CMST-F

[Luo et al. \(2021\)](#) introduces 68 real-world instances of CMST-F, with 108 PV arrays (non-root vertices). According to the types of subtrees (combiner boxes) required, these instances are divided into four groups, each consisting of 17 instances. We name each group according to the allowed combiner box capacities; for example, "q-6-8" indicates that only subtrees with capacities 6 and 8 are allowed.

Table 4 Detailed Experimental Results for CMST-F

Root																		Total						
Name	Inst	solved	LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	IP Time							
q-6-8	17	17	0.98	1.22	0.18	5.68	12992	6139	5490	28	194	66	477	28	1.39	0.10	82.43							
q-8-10	17	17	1.16	0.98	0.18	12.46	52083	57893	21046	27	245	52	541	23	14.04	0.09	230.46							
q-10-12	17	17	1.24	7.00	0.11	73.85	180921	375340	69089	20	268	21	365	7	55	0.06	372.23							
q-10-14	17	14	1.01	108.38	0.12	907.18	588690	2059823	188285	8	269	10	336	4	441.46	0.07	3128.02							
Average	68	65	1.10	29.39	0.15	249.79	208671	624799	70977	21	244	37	430	16	127.96	0.08	953.29							

Overall, in Table 4, 65 out of 68 instances can be solved to optimality within the time limit of 7200 seconds. We observe from this table that the values of No.P1 Label, No.P2 Label, and No.R Label increase exponentially as the maximum capacity Q increases from 8 to 14. Similar to CMST-L, SRIs are more effective than RCIs in CMST-F. Furthermore, the value of Pr Time is extremely small (below 0.1 seconds), because *q-arb* is used for label pruning in this problem and can be performed very fast.

7.4. Strength of Algorithm Feature

In this section, we compare the performance of three components, namely *dichotomy labeling*, the joining procedure, and *ng-arb* pruning, with the state-of-the-art method from [Luo et al. \(2021\)](#). The comparison is performed on the most classic CMST-E instances with maximum capacity $Q = 10$.

7.4.1. Dichotomy Labeling vs. Sequence Labeling As shown in Proposition 3, the average *lifetime* of labels in *dichotomy labeling* ($\mathcal{O}(\log_2 |\mathcal{V}^+|)$) is much smaller than that in *sequence labeling* ($\mathcal{O}(|\mathcal{V}^+|)$). Therefore, more labels are retained in $\mathbb{L}$ rather than in $\mathbb{P}$, meaning more PT labels are generated in *sequence labeling*. Table 5 provides experimental validation of this point by solving LP without any cuts at the root node of the branch-and-bound tree. For fairness, we disable the backward labeling and *ng-arb* pruning. We report, for the final column generation iteration, the following: the number of non-dominated labels (No.Nd Label), the number of labels generated by extension from the core vertex (No.Ex Label) (see Figure 6a), the number of labels generated by combination (No.Co Label) (see Figure 6b), the time of label combination (No.Co Count), and the time required to solve the LP (LP Time). Note that the value of No.Co Label is much less than that of No.Co Count. This is due to our pre-validation procedure: if two labels cover the same vertex, or if the sum of their reduced costs can be pruned by the lower bound Δ^{rt} , then they are not combined to form a new label.

Table 5 Detailed Experimental Results for Dichotomy and Sequence Labeling

Name	Inst	Dichotomy Labeling					Sequence Labeling				
		No.Nd Label	No.Ex Label	No.Co Label	No.Co Count	LP Time	No.Nd Label	No.Ex Label	No.Co Label	No.Co Count	LP Time
tc-80-10	5	1171	990	833	6675	0.85	1056	851	781	384987	1.77
te-80-10	5	1007	1033	735	11630	1.21	1134	1277	1095	579476	2.17
td-80-10	5	2458	3944	3628	119240	1.74	2505	4066	3956	2789172	3.08
tc-120-10	5	4449	5938	5158	170396	2.40	4033	5237	6287	5720434	7.79
te-120-10	5	3490	5324	4472	211022	3.52	3208	5695	6080	5850954	7.49
tc-160-10	1	3928	5094	4086	87609	5.61	4654	5890	5499	8589067	18.57
te-160-10	1	3581	6181	5273	135022	7.41	4035	6684	7615	7344413	15.48
Average	27	2607	3608	3093	104350	2.28	2532	3637	3856	3428096	5.39

From Table 5, we see that the two labeling algorithms yield similar values for No.Nd Label and No.Ex Label. This is reasonable because for a given pricing subproblem, the set of non-dominated labels (No.Nd Label) is fixed, stored in $\cup_{i \in \mathcal{V}^+, q \in \{0, \dots, Q-1\}} \mathbb{P}(i, M, q)$ for *dichotomy labeling* and in $\cup_{i \in \mathcal{V}^+, q \in \{1, \dots, Q\}} \mathbb{L}(i, |\mathcal{V}^+|, q)$ for *sequence labeling*. Since No.Ex Label is extended from No.Nd Label, its values in the two labeling algorithms are also similar. The impact of *lifetime* manifests primarily in No.Co Label and No.Co Count. The smaller *lifetime* in *dichotomy labeling* results in fewer labels being retained in set $\mathbb{P}$, leading to a No.Co Count value that is 31.8 times smaller than that of *sequence labeling*. After the pre-validation procedure, No.Co Label of *dichotomy labeling* remains significantly smaller than that of *sequence labeling*, which enables it to achieve a speedup factor of approximately 2.36 in runtime.

7.4.2. PT-RT Joining vs. Classical Joining Our joining procedure (termed as PT-RT joining in this subsection) consists of two parts. The first part involves generating PTs with larger capacity using Algorithm 3, and the second part involves combining all PTs and RTs to form elementary subtrees using Algorithm 4. With the help of capacitated subtree analysis and Proposition

2, PT-RT joining can be executed very efficiently. Luo et al. (2021) also adopted the idea of a joining procedure (termed as classical joining in this subsection) to combine their forward (No.F Label) and backward (No.B Label) labels. However, due to the absence of Proposition 2, their joining process is very time-consuming.

Table 6 Detailed Experimental Results for PT-RT joining and Classical joining

Name	Inst	PT-RT Joining					Classical Joining			
		No.P1 Label	No.P2 Label	No.R Label	Join Time	LP Time	No.F Label	No.B Label	Join Time	LP Time
tc-80-10	5	927	43	955	0.02	0.84	921	945	0.81	2.16
te-80-10	5	975	31	1960	0.01	1.25	927	1946	1.11	2.59
td-80-10	5	2630	137	4610	0.02	1.85	2093	4078	2.33	4.84
tc-120-10	5	3700	185	5828	0.05	2.53	3207	5742	6.75	13.36
te-120-10	5	2874	131	6680	0.05	3.73	2636	7202	7.68	17.03
tc-160-10	1	4163	157	9908	0.15	5.67	2768	7196	13.76	27.52
te-160-10	1	3950	73	5967	0.13	7.91	3133	4758	13.68	25.56
Average	27	2358	107	4298	0.04	2.39	2031	4130	4.48	9.36

We run PT-RT joining and classical joining to solve the LP at the root of the branch-and-bound tree on 27 CMST-E instances. The results are detailed in Table 6. It can be seen that although the numbers of non-dominated labels generated by the two algorithms are similar, PT-RT joining reduces the required time by a factor of 111. In contrast, classical joining accounts for nearly half of its total runtime.

7.4.3. Ng-arb Pruning vs. Q-arb Pruning To demonstrate the superiority of *ng-arb* over *q-arb*, we run our full PR-BPC with *ng-arb* and *q-arb* pruning, respectively, to compute the optimal integer solution of CMST-E, which is assisted by an upper bound provided by the external memetic algorithm. The following information is indicated in Table 7: the maximum number of non-dominated PT labels with capacity $1 \leq q \leq \lfloor \frac{Q}{2} \rfloor$ (Max.P1 Label) and with $\lfloor \frac{Q}{2} \rfloor + 1 \leq q \leq Q - 1$ (Max.P2 Label), respectively, among the column generation iterations, and the maximum number of non-dominated RT labels among the column generation iterations (Max.R Label).

Table 7 Detailed Experimental Results for Ng-arb and Q-arb pruning

Name	Inst	Ng-arb					Q-arb				
		Max.P1 Label	Max.P2 Label	Max.R Label	Pr Time	IP Time	Max.P1 Label	Max.P2 Label	Max.R Label	Pr Time	IP Time
tc-80-10	5	3547	2401	2823	0.83	14.51	3792	2295	4882	0.07	15.78
te-80-10	5	5880	1164	6676	0.99	12.04	6437	1458	14450	0.08	13.63
td-80-10	5	26236	37642	26195	5.80	90.68	191010	1763983	52339	0.12	254.63
tc-120-10	5	24827	63629	26493	7.57	124.01	30739	60025	53239	0.44	166.38
te-120-10	5	9920	1876	12722	7.88	65.85	19403	3557	50668	0.39	137.94
tc-160-10	1	28867	28762	54798	11.77	212.55	39086	16834	91293	0.88	231.23
te-160-10	1	19421	3022	90624	15.10	247.61	45483	21402	74399	1.04	284.53
Average	27	14828	20939	19258	5.27	73.91	49685	340550	38652	0.27	128.06

Compared with *q-arb*, the values of Max.P1 Label, Max.P2 Label, and Max.R Label obtained by *ng-arb* are reduced by factors of 2.32, 15.26, and 1.01, respectively. Furthermore, although *ng-arb* requires more time to compute the bounds Δ^{pt} and Δ^{rt} , the tighter bounds achieve a speedup of 1.73-fold in total computing time.

7.5. Comparison with the Literature

In this section, we compare the performance of PR-BPC with state-of-the-art exact methods from the literature. Specifically, we compare PR-BPC with the methods of Luo et al. (2021) (hereafter Luo21) and Uchoa et al. (2008) (hereafter Uchoa08) on CMST-E, with Uchoa et al. (2012) (hereafter Uchoa12) on CMST-L, and with Luo21 on CMST-F. For the root node and the overall algorithm execution, we respectively report the percentage of instances solved to optimality (Opt (%)), the average optimality gap (Gap %), and the total running time (Time). Note that the root node results are only available for our PR-BPC, and the reported information is based on the LPC value at the root node and the external upper bound procedure.

7.5.1. Comparison with Uchoa et al. (2008) and Luo et al. (2021) on CMST-E Table 8 compares the performance of PR-BPC with Luo21 and Uchoa08 on the 54 CMST-E instances. Both Luo21 and Uchoa08 proposed BPC algorithms for CMST-E; however, Luo21 relies on an *elementary-subtree*-based formulation, whereas Uchoa08 focused on a *q-arb*-based formulation. Luo21 was run on a machine with a 3.10 GHz E5-1607 CPU and 128 GB of RAM, whereas Uchoa08 was run on a machine with a 3 GHz Pentium processor. A dash indicates that the corresponding information is unavailable.

Table 8 Comparison with Luo et al. (2021) and Uchoa et al. (2008) on the CMST-E

Name	Inst	PR-BPC						Luo21			Uchoa08		
		Root			Total			Total			Total		
		Opt (%)	Gap (%)	Time	Opt (%)	Gap (%)	Time	Opt (%)	Gap (%)	Time	Opt (%)	Gap (%)	Time
tc-80-5	5	80	0.06	33.26	100	0	34.61	100	0	57.3	100	0	67.4
te-80-5	5	60	0.05	32.0	100	0	33.8	100	0	59.5	100	0	188.8
td-80-5	5	80	0.01	35.3	100	0	35.5	100	0	55.5	100	0	106.3
tc-120-5	5	40	0.10	65.2	100	0	71.2	100	0	192.3	100	0	311.1
te-120-5	5	60	0.03	64.4	100	0	68.2	100	0	154.7	100	0	1536.3
tc-160-5	1	100	0	162.8	100	0	162.8	100	0	288.9	100	0	280.6
te-160-5	1	0	0.06	176.4	100	0	222.9	100	0	290.0	100	0	938.0
tc-80-10	5	100	0	118.9	100	0	118.9	100	0	372.4	-	-	-
te-80-10	5	100	0	116.2	100	0	116.2	100	0	252.0	100	0	10452.7
td-80-10	5	60	0.04	129.6	100	0	198.9	100	0	1761.7	100	0	45883.8
tc-120-10	5	20	0.21	236.0	100	0	289.9	100	0	4919.5	100	0	176491.0
te-120-10	5	80	0.04	208.5	100	0	235.4	100	0	1200.9	60	-	118833.2
tc-160-10	1	0	0.24	632.8	100	0	684.3	100	0	8,940.8	0	-	303,095.0
te-160-10	1	0	0.12	684.1	100	0	726.3	100	0	5,481.8	0	-	226,346.0
Average	54	64.8	0.06	126.2	100	0	144.6	100	0	1,113.5	92.6	-	46,939.1

Table 8 shows that PR-BPC and Luo21 can solve all 54 instances, whereas Uchoa08 can only solve 92.6% of them. Moreover, PR-BPC demonstrates a significant advantage in computation, achieving speedups of 7.7-fold and 324.6-fold over Luo21 and Uchoa08, respectively. The effectiveness of PR-BPC stems from an optimality gap of 0.06% at the root node; this small value enables 64.8% of the instances to be solved to optimality at the root node without any branching.

7.5.2. Comparison with Uchoa et al. (2012) on CMST-L Table 9 compares the performance of PR-BPC with Uchoa12 on 450 CMST-L instances. In particular, for instances 50c, 50e, 50r, and 100c, Uchoa et al. (2012) adopted their branch-and-cut algorithm to obtain an optimal solution. For the other instances, this algorithm cannot solve them optimally within a reasonable computing time; therefore, they ran the branch-and-cut at the root node to obtain a lower bound and used the greedy randomized adaptive search procedure to compute an upper bound. Uchoa12 was run on a Core 2 Quad 2.5 GHz machine with 4 GB of RAM, running Ubuntu 8.04.

Table 9 Comparison with Uchoa et al. (2012) on the CMST-L

Name	Inst	PR-BPC						Uchoa12		
		Root			Total			Total		
		Opt (%)	Gap (%)	Time	Opt (%)	Gap (%)	Time	Opt (%)	Gap (%)	Time
50c	50	22	0.25	0.6	100	0	11.63	100	0	17.2
50e	50	34	0.16	24.0	94	0.01	940.1	100	0	113.2
50r	50	30	0.16	6.77	100	0	310.19	100	0	46.7
100c	50	14	0.10	47.34	100	0	545.4	100	0	6780.1
100e	50	4	0.25	106.8	38	0.12	5145.91	0	0.45	10562.8
100r	50	14	0.16	81.92	76	0.04	2738.2	0	0.51	4950.8
150c	50	8	0.12	314.9	62	0.04	3626.1	0	0.56	7529.5
150e	50	4	0.30	312.7	26	0.22	5919.1	0	0.53	16620.8
150r	50	0	0.18	247.0	38	0.09	5417.6	0	0.58	8028.4
Average	450	14.4	0.19	126.8	70.4	0.06	2739.4	44.4	0.29	6072.2

Table 9 shows that PR-BPC can solve 70.4% of the 450 instances, whereas Uchoa12 can solve 44.4% of them. Specifically, on two groups of small-scale instances (50e and 50r), PR-BPC performs worse because too many SRIs are enumerated; however, on other medium- and large-scale instances, PR-BPC significantly outperforms Uchoa12. In particular, on instances 100e, 100r, 150c, 150e, and 150r, the optimality gap (0.20%) obtained at the root node of PR-BPC is already much better than that (0.53%) achieved by the complete run of Uchoa12, and the time cost is substantially lower (a 42.69-fold reduction).

7.5.3. Comparison with Luo et al. (2021) on CMST-F Table 10 compares the results achieved by PR-BPC and by Luo21 on the four groups of CMST-F instances. A dash indicates that the corresponding information is unavailable.

Table 10 Comparison with Luo et al. (2021) on the CMST-F

Name	Inst	PR-BPC						Luo21		
		Root			Total			Total		
		Opt (%)	Gap (%)	Time	Opt (%)	Gap (%)	Time	Opt (%)	Gap (%)	Time
q-6-8	17	23.5	0.42	12.0	100	0	82.4	100	0	341.1
q-8-10	17	23.5	0.28	19.8	100	0	230.5	100	0	2956.6
q-10-12	17	35.3	0.19	82.7	100	0	372.2	88.2	-	10206.9
q-10-14	17	52.9	0.23	917.0	82.4	0.04	3128.0	41.1	-	23858.0
Average	68	33.8	0.28	257.9	95.6	0.01	953.3	82.4	-	9340.6

We can see that PR-BPC can solve 95.6% of the 68 instances, whereas Luo21 can solve 82.4% of them. On average, PR-BPC achieves a 9.8-fold speedup over Luo21, an advantage that becomes more pronounced as the maximum subtree capacity Q increases (from $Q = 8$ to $Q = 14$), thanks to the algorithmic features of the pricing procedure.

8. Conclusion

We have presented a novel branch-price-and-cut algorithm based on the concepts of *pseudo tree* and *real tree* to solve the capacitated minimum spanning tree problem. The PR-BPC relies on a mathematical model with an exponential number of *elementary subtree* variables and valid inequalities. Although this model provides a relatively tight lower bound, its pricing procedure is very time-consuming. To address this drawback, we analyze the structure of capacitated subtrees and propose three key components—PT-RT joining, *ng-arb* pruning, and *dichotomy labeling*—along with four additional acceleration strategies to speed up the label setting process. Computational results show that PR-BPC efficiently solves CMST-E instances and can be easily adapted to solve the other two variants of CMST by modifying the pricing procedure. The obtained results significantly outperform those achieved by the best exact methods in the literature.

The bottleneck of PR-BPC is the label explosion in the labeling algorithm when the maximum capacity Q is large and too many non-robust SRIs are enumerated. Future research can be directed toward studying effective robust valid inequalities that do not affect the complexity of the pricing problem. Moreover, because the construction of PTs and RTs relies on the decomposition of subtrees with unit demands, PR-BPC is applied only to CMST instances with unit demand in this work. Exploring new decomposition methods for subtrees with general demand (where each vertex can have an arbitrary positive integer demand) would be a promising direction for solving more general instances.

Electronic Companion

EC.1. Arc-based Models for Three Types of CMST

Let $x_{i,j}^d$ be equal to 1 if d ($1 \leq d \leq Q$) units of flow traverse arc $(i,j) \in \mathcal{A}$, and 0 otherwise. Then, the capacity-indexed formulations for the three CMST problems are as follows:

The CMST-E problem can be modeled as:

$$\min \sum_{(i,j) \in \mathcal{A}} c_{i,j} \sum_{d=1}^Q x_{i,j}^d \quad (\text{EC.1})$$

$$\text{s.t. } \sum_{(i,j) \in \mathcal{A}} \sum_{d=1}^Q x_{i,j}^d = 1, \quad \forall i \in \mathcal{V}^-, \quad (\text{EC.2})$$

$$\sum_{(i,j) \in \mathcal{A}} \sum_{d=1}^Q dx_{i,j}^d - \sum_{(j,i) \in \mathcal{A}} \sum_{d=1}^Q dx_{j,i}^d = 1, \quad \forall i \in \mathcal{V}^-, \quad (\text{EC.3})$$

$$x_{i,j}^d \in \{0, 1\}, \quad \forall (i,j) \in \mathcal{A}, d = 1, \dots, Q. \quad (\text{EC.4})$$

where the objective function (EC.1) aims at minimizing the total Euclidean costs. Constraints (EC.2) ensure that each vertex $i \in \mathcal{V}^-$ has exactly one outgoing arc with a certain amount of flow. Constraints (EC.3) indicate that the difference between the outgoing and incoming flow of each vertex $i \in \mathcal{V}^-$ must be equal to 1 (unit demand). Finally, Constraints (EC.4) stipulate the variable domain.

The CMST-L problem can be modeled as:

$$\min \sum_{(i,j) \in \mathcal{A}} \sum_{d=1}^Q \min_{l \in \{1,2,\dots,L\}} \{c_{i,j}^l | v^l \geq d\} x_{i,j}^d \quad (\text{EC.5})$$

$$\text{s.t. } \sum_{(i,j) \in \mathcal{A}} \sum_{d=1}^Q x_{i,j}^d = 1, \quad \forall i \in \mathcal{V}^-, \quad (\text{EC.6})$$

$$\sum_{(i,j) \in \mathcal{A}} \sum_{d=1}^Q dx_{i,j}^d - \sum_{(j,i) \in \mathcal{A}} \sum_{d=1}^Q dx_{j,i}^d = 1, \quad \forall i \in \mathcal{V}^-, \quad (\text{EC.7})$$

$$x_{i,j}^d \in \{0, 1\}, \quad \forall (i,j) \in \mathcal{A}, d = 1, \dots, Q. \quad (\text{EC.8})$$

where the objective function (EC.5) aims at minimizing the total link costs. The constant $\min_{l \in \{1,2,\dots,L\}} \{c_{i,j}^l | v^l \geq d\}$ of (EC.5) represents the minimum link cost for arc (i,j) when it carries d units of flow. Constraints (EC.6)-(EC.8) correspond to (EC.2)-(EC.4).

The CMST-F problem can be modeled as:

$$\min \sum_{(i,j) \in \mathcal{A}: j=0} c_{i,j} \sum_{d=1}^Q x_{i,j}^d + \sum_{(i,j) \in \mathcal{A}, j \neq 0} \sum_{d=1}^Q dc_{i,j} x_{i,j}^d \quad (\text{EC.9})$$

$$\text{s.t. } \sum_{(i,0) \in \mathcal{A}} x_{i,0}^{qt} = m_t \quad t \in \mathcal{T}, \quad (\text{EC.10})$$

$$x_{i,0}^{qt} = 0, \quad \forall (i,0) \in \mathcal{A}, t \notin \mathcal{T}, \quad (\text{EC.11})$$

$$\sum_{(i,j) \in \mathcal{A}} \sum_{d=1}^Q x_{i,j}^d = 1, \quad \forall i \in \mathcal{V}^-, \quad (\text{EC.12})$$

$$\sum_{(i,j) \in \mathcal{A}} \sum_{d=1}^Q dx_{i,j}^d - \sum_{(j,i) \in \mathcal{A}} \sum_{d=1}^Q dx_{j,i}^d = 1, \quad \forall i \in \mathcal{V}^-, \quad (\text{EC.13})$$

$$x_{i,j}^d \in \{0, 1\}, \quad \forall (i,j) \in \mathcal{A}, d = 1, \dots, Q. \quad (\text{EC.14})$$

where the objective function (EC.9) aims at minimizing the total flow costs. Constraints (EC.10) guarantee that the solution must contain m_t type t rooted-subtrees. Constraints (EC.11) mean that any rooted-subtree of the solution must be one type of $\mathcal{T}$. Constraints (EC.12)-(EC.14) correspond to (EC.2)-(EC.4).

EC.2. Proof of Propositions

EC.2.1. Proof of Proposition 1

Algorithm 6 Procedure to identify the Core Vertex

Input A subtree λ with capacity q .

```

1: Initialization active vertex :  $i \leftarrow 0$ .
2: function CoreSearch ( $i$ )
3: set core vertex  $cv \leftarrow i$ 
4: for each subtree  $\lambda^i$  hanging from  $i$  of subtree  $\lambda$  do
5:   if Tree_Count( $\lambda^i$ ) >  $\lfloor \frac{q}{2} \rfloor$  then
6:      $i \leftarrow \text{Tree\_Root}(\lambda^i)$ 
7:      $cv = \text{CoreSearch}(i)$ .
8: break
return  $cv$ .

```

To show how to find the unique core vertex of a capacitated subtree, we introduce a recursive procedure in Algorithm 6, where **Tree_Count**() and **Tree_Root**() are two subroutines that compute the capacity of a subtree and retrieve its root vertex, respectively. It starts by setting the active vertex to 0, then checks the capacity of the subtree hanging from it (Line 4 of Algorithm 6) and updates the active vertex (Line 6 of Algorithm 6) until the capacity of each subtree does not exceed $\lfloor \frac{q}{2} \rfloor$. Notably, given a vertex i , there exists at most one subtree hanging from it with a capacity greater than $\lfloor \frac{q}{2} \rfloor$; otherwise, $q \geq 2 \lfloor \frac{q}{2} \rfloor + 2 > q$ would lead to a contradiction. Therefore, Lines 6 of Algorithm 6 are executed at most once per recursion.

Figure EC.1 gives an example (a rooted-subtree with capacity 9) of finding core vertex by Algorithm 6. For ease of presentation, we represent a subtree rooted at i hanging from j as λ_i^j . First, we have active vertex 0; since the capacity of the subtree λ_1^0 is $9 > \lfloor \frac{9}{2} \rfloor = 4$, the active vertex cv is thus updated to the root of λ_1^0 , i.e., $cv = 1$ (From Figure EC.1a to Figure EC.1b). Then, we check the subtrees hanging from vertex 1, λ_2^1 and λ_3^1 , and find that their capacities are $6 \geq \lfloor \frac{9}{2} \rfloor = 4$ and $2 \leq \lfloor \frac{9}{2} \rfloor = 4$, respectively. So we continue to update the core vertex cv to the root of λ_2^1 , i.e., $cv = 2$ (From Figure EC.1b to Figure EC.1c). At this point, the subtrees λ_4^2 and λ_5^2 both have capacity less than 4, therefore, we return vertex 2 as the core vertex.

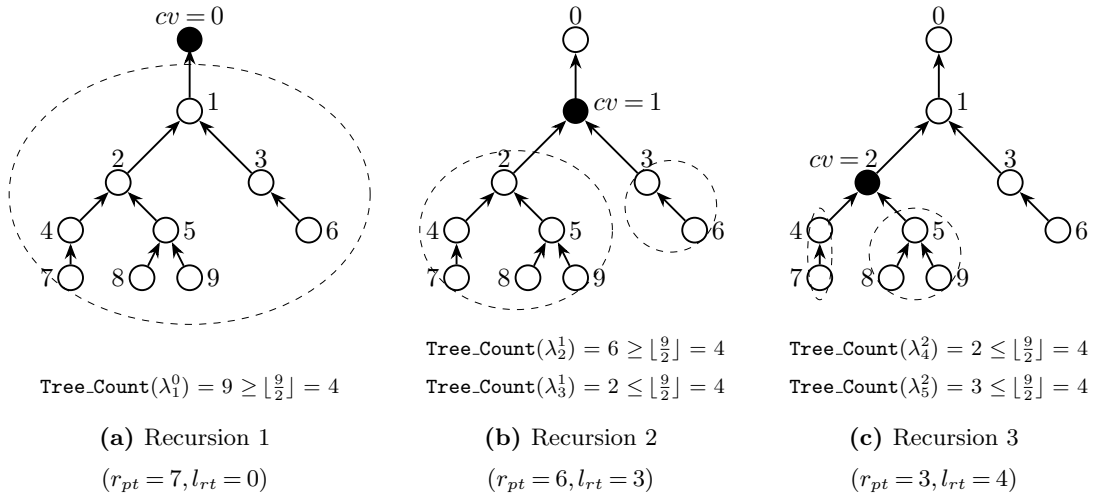


Figure EC.1 Example of Finding Core Vertex by Algorithm 6

As the vertices of a subtree are limited, the core vertex can always be found, and the `Tree.Count()` subroutine ensures that the capacity of the subtrees hanging from the core vertex must not be greater than $\lfloor \frac{q}{2} \rfloor$. We next show that the capacity of the RT will also be bounded. Let i denote the core vertex found by Algorithm 6, and j be its parent vertex. In the recursion when the active vertex is j (e.g., $cv = 1$ in Figure EC.1b), Line 4 of Algorithm 6 indicates that the subtree λ_i^j has capacity larger than $\lfloor \frac{q}{2} \rfloor$ (e.g., $\lambda_2^1 \geq 4$ in Figure EC.1b). Otherwise, j will be selected as the core vertex. In this case, the capacity of the RT must be less than $q - \lfloor \frac{q}{2} \rfloor = \lceil \frac{q}{2} \rceil$. After updating the active vertex from j to i , i belongs to RT, and its capacity plus one satisfies the requirement that it is at most $\lceil \frac{q}{2} \rceil$ for an RT, as stated in Proposition 1. Therefore, the capacities of PTs and subtrees in RTs hanging from the core vertex must be bounded by $\lfloor \frac{q}{2} \rfloor$ and $\lceil \frac{q}{2} \rceil$, respectively.

Regarding the uniqueness of the core vertex, since Lines 6 of Algorithm 6 are executed at most once per recursion, exactly one core vertex can be found by this procedure. If there exist multiple core vertices other than the core vertex cv found by Algorithm 6, there will be three cases:

Case 1 : If there exist a core vertex cv' selected from the subtrees hanging from cv , in this case, since the total capacity of these subtrees (their root vertexes are not included) is less than $\lfloor \frac{q}{2} \rfloor$, the capacity of RT will be larger than $q - \lfloor \frac{q}{2} \rfloor = \lceil \frac{q}{2} \rceil$, which leads to a contradiction. **Example can be seen in Figure EC.2a.**

Case 2 : If there exist a core vertex cv' selected from the vertices on the path from the cv to the root 0, in this case, the **PT-element** including cv must be larger than $\lfloor \frac{q}{2} \rfloor$, which leads to a contradiction. **An example of such PT-element is λ_2^1 in Figure EC.2b.**

Case 3 : **Otherwise**, the capacity of RT will be larger than $\lceil \frac{q}{2} \rceil$, which leads to a contradiction. Note that the **inequality** is derived from the activation of vertex cv' . **For example, in Figure EC.2c,** as the core vertex cv is updated from 1 to 2, the subtree λ_2^1 must have capacity larger than $\lfloor \frac{q}{2} \rfloor$. In other words, λ_2^1 has at least $\lceil \frac{q}{2} \rceil$ capacity. If $cv' = 3$ is selected as the new core vertex, vertex 3 and subtree λ_2^1 will both be included in the RT.

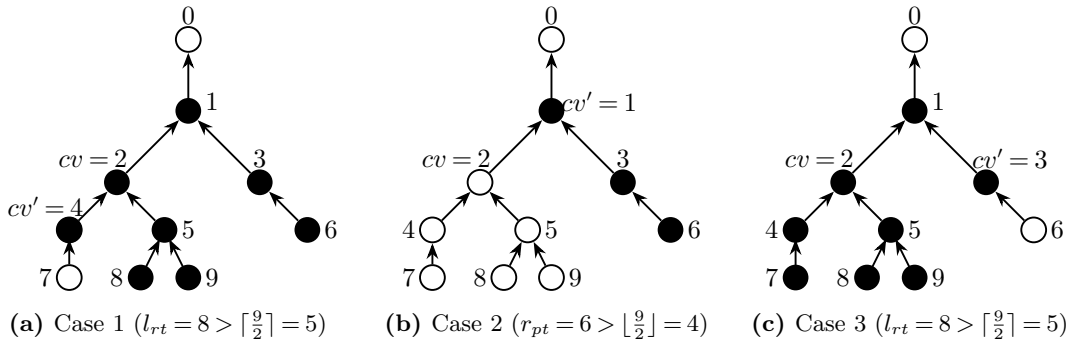


Figure EC.2 sss

In summary, the core vertex for a capacitated subtree is unique.

EC.2.2. Proof of Proposition 4

It is easy to see that Algorithm 1 can generate all non-dominated pseudo-trees. Under the binary tree storage structure, the set $\mathbb{P}(i, M, q)$ contains every non-dominated pseudo-tree whose core vertex is i and whose capacity is at most q .

For the downward labeling, suppose that there exists a non-dominated real-tree label R_i with core vertex i , which is not generated by Algorithm 5. We consider the following two cases :

Case 1 : if i is directly linked to the *Root* 0, this type of real-trees R_i in Algorithm 5 is created by Part 1, which activates the core vertex of pseudo-trees in $\mathbb{P}(i, M, q(R_i) - 1)$ and links it to 0. If R_i is not extended by these pseudo-trees, this means R_i is dominated, which leads to contradiction.

Case 2 : if i is not directly linked to the *Root* 0, we let notation $parent(i)$ to denote the parent vertex of i . Suppose that the subtree of R_i rooted at i has capacity q' , termed as R_i^{sub} , this type of

real-trees R_i in Algorithm 5 is created by Part 2, which activates the core vertex of pseudo-trees in $\mathbb{P}(i, M, q' - 1)$ and link it to real-trees in $\mathbb{R}(\text{parent}(i), q(R_i) - q')$. If R_i^{sub} is not extended by these pseudo-trees, this means R_i^{sub} , also, R_i , is dominated, which leads to contradiction. Otherwise, if the remaining tree $(R_i \setminus R_i^{sub})$ with core vertex $\text{parent}(i)$ is not included in $\mathbb{R}(\text{parent}(i), q(R_i) - q')$, we recursively check it by the above procedure until the core vertex is updated to the vertex directly linked to 0. In this case, we use Case 1 to prove the contradiction.

EC.2.3. Proof of Proposition 5

Recall that the rooted-subtrees needed to be generated by joining have capacity from $\lfloor \frac{Q}{2} \rfloor$ to Q . Therefore, $\lfloor \frac{Q}{2} \rfloor - e$ and $Q - e$ naturally give one type of the lower and upper bound of real-tree capacity when the extra flow is fixed to e .

For the lower limit, according to Lemma 1, when joining occurs, the real-trees have capacity at least 1 ($(\lfloor \frac{q}{2} \rfloor - 1)$). Combine the values $\lfloor \frac{Q}{2} \rfloor - e$ and 1, we obtain the lower bound $\max\{\lfloor \frac{Q}{2} \rfloor - e, 1\}$.

For the upper limit, according to Lemma 1, when joining occurs, the capacity of real-trees is at most one greater than that of pseudo-trees, or, extra flow e (Pattern 0). Combine the values $Q - e$ and $e + 1$, we obtain the upper bound $\min\{Q - e, e + 1\}$.

EC.3. Examples

EC.3.1. An Example of Combination Pattern

This section gives a detailed example of lemma 1. Considering the subtree in Figure 2 with capacity $q = 7$, there are two cases below :

Pattern 0 : $l_{pt} = 3, l_{rt} = 4, \omega_{pt} \geq 1$, and $1 \leq \rho_{pt} \leq 3$

Pattern 1 : $l_{pt} = 4, l_{rt} = 3, \omega_{pt} \geq 2$, and $2 \leq \rho_{pt} \leq 2$

The following cases do exist, since the inequality $2 \leq \rho_{pt} \leq 1$ is contradictory.

Pattern 2 : $l_{pt} = 5, l_{rt} = 2, \omega_{pt} \geq 3$, and $2 \leq \rho_{pt} \leq 1$

Pattern 3 : $l_{pt} = 6, l_{rt} = 1, \omega_{pt} \geq 4$, and $2 \leq \rho_{pt} \leq 1$

We let the set $\mathbb{G}(i)$ collecting the group of subtrees in PT hanging from the core vertex, where i denotes the index of pattern. According to the requirement of ω_{pt} in each pattern, we construct the group $\mathbb{G}(i), i \in \{0, 1\}$ as follows :

$$\mathbb{G}(0) = \{(3), (1, 2), (1, 1, 1)\}.$$

$$\mathbb{G}(1) = \{(4), (2, 2)\}.$$

The Case 2 in Figure 3b corresponds to (1, 2) in $\mathbb{G}(0)$ (Pattern 0), while Case 4 in Figure 3d corresponds to (3) in $\mathbb{G}(0)$ (Pattern 0).

EC.3.2. An Example of Constructing A Storage Binary Tree

We give an example to show how the dichotomy extension in Algorithm 1 works to construct the binary tree shown in Figure 5. As there are 7 vertices, we have $d_1 = 6, d_2 = 3, d_3 = 2, d_4 = 1$ and the depth $g_{max} = \lceil \log_2 6 \rceil + 1 = 4$. For a fixed capacity q and vertex i , Line 6-12 (Part 1) is straightforward to construct leaf node indexed from 1 to 6, so we omit its description. In the following part, st is firstly initialized to 0, and then there are $g_{max} - 1 = 3$ outer iterations, and each outer loop has d_g inner loops that need to be executed :

1-outer loop $st = 0, g = 2, d_g = 3$

1.1-inner loop Line 21-22 (Step 1) : Add all the non-dominated PT labels in $\mathbb{P}(i, 1, q)$ and $\mathbb{P}(i, 2, q)$ to $\mathbb{P}(i, 7, q)$.

Line 24-30 (Step 2) : Combine labels in $\mathbb{P}(i, 1, q')$ and in $\mathbb{P}(i, 2, q - q'), \forall q' \in \{1, \dots, q - 1\}$ to generate new labels, and add these labels to $\mathbb{P}(i, 7, q)$.

1.2-inner loop Line 21-22 (Step 1) : Add all the non-dominated PT labels in $\mathbb{P}(i, 3, q)$ and $\mathbb{P}(i, 4, q)$ to $\mathbb{P}(i, 8, q)$.

Line 24-30 (Step 2) : Combine labels in $\mathbb{P}(i, 3, q')$ and in $\mathbb{P}(i, 4, q - q'), \forall q' \in \{1, \dots, q - 1\}$ to generate new labels, and add these labels to $\mathbb{P}(i, 8, q)$.

1.3-inner loop Line 21-22 (Step 1) : Add all the non-dominated PT labels in $\mathbb{P}(i, 5, q)$ and $\mathbb{P}(i, 6, q)$ to $\mathbb{P}(i, 9, q)$.

Line 24-30 (Step 2) : Combine labels in $\mathbb{P}(i, 5, q')$ and in $\mathbb{P}(i, 6, q - q'), \forall q' \in \{1, \dots, q - 1\}$ to generate new labels, and add these labels to $\mathbb{P}(i, 9, q)$.

2-outer loop $st = 6, g = 3, d_g = 2$

2.1-inner loop Line 21-22 (Step 1) : Add all the non-dominated PT labels in $\mathbb{P}(i, 7, q)$ and $\mathbb{P}(i, 8, q)$ to $\mathbb{P}(i, 10, q)$.

Line 24-30 (Step 2) : Combine labels in $\mathbb{P}(i, 7, q')$ and in $\mathbb{P}(i, 8, q - q'), \forall q' \in \{1, \dots, q - 1\}$ to generate new labels, and add these labels to $\mathbb{P}(i, 10, q)$.

2.2-inner loop Line 18-19 (Step 1) : Add all the non-dominated PT labels in $\mathbb{P}(i, 9, q)$ to $\mathbb{P}(i, 11, q)$.

3-outer loop $st = 9, g = 4, d_g = 1$

3.1-inner loop Line 21-22 (Step 1) : Add all the non-dominated PT labels in $\mathbb{P}(i, 10, q)$ and $\mathbb{P}(i, 11, q)$ to $\mathbb{P}(i, 12, q)$.

Line 24-30 (Step 2) : Combine labels in $\mathbb{P}(i, 10, q')$ and in $\mathbb{P}(i, 11, q - q')$, $\forall q' \in \{1, \dots, q - 1\}$ to generate new labels, and add these labels to $\mathbb{P}(i, 12, q)$.

EC.3.3. An Example of Capacity Range of RT

Assume that we have a maximum capacity $Q = 10$, in this case, the subtree with capacity from $\lfloor \frac{Q}{2} \rfloor + 2 = 7$ to $Q - 1 = 9$ need to be generated through joining. The extra flow range are constrained in $[3, 9] \cap \mathbb{Z}$ according to Line 1 of Algorithm 5. Therefore, for each possible extra flow value $w \in [3, 9] \cap \mathbb{Z}$, we derive its corresponding RT capacity as follows :

Case 1 : if $w = 3$, the RT capacity is constrained in $[4, 4] \cap \mathbb{Z}$.

Case 2 : if $w = 4$, the RT capacity is constrained in $[3, 5] \cap \mathbb{Z}$.

Case 3 : if $w = 5$, the RT capacity is constrained in $[2, 5] \cap \mathbb{Z}$.

Case 4 : if $w = 6$, the RT capacity is constrained in $[1, 4] \cap \mathbb{Z}$.

Case 5 : if $w = 7$, the RT capacity is constrained in $[1, 3] \cap \mathbb{Z}$.

Case 6 : if $w = 8$, the RT capacity is constrained in $[1, 2] \cap \mathbb{Z}$.

Case 7 : if $w = 9$, the RT capacity is constrained in $[1, 1] \cap \mathbb{Z}$.

EC.4. Description of Ng-arb

Given a subtree $\lambda_{ng} (0, 1, 2, 3, 3, 4, 5)$ indexed by $(0, 1, 2, 3, 4, 5, 6)$, with arcs $(3, 2), (4, 2), (5, 3), (2, 1), (3, 1)$ and $(1, 0)$, we therefore have $\phi(0) = 0, \phi(1) = 1, \phi(2) = 2, \phi(3) = 3, \phi(4) = 3, \phi(5) = 4$ and $\phi(6) = 5$. Figure EC.3 shows this subtrees.

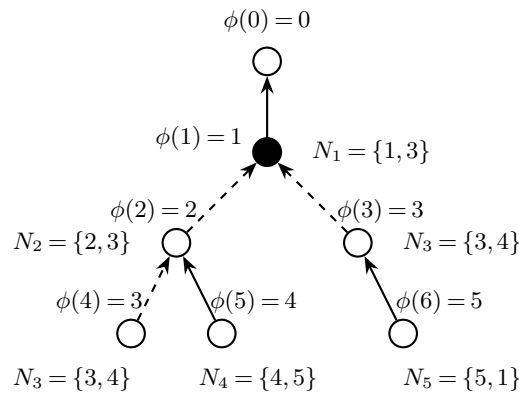


Figure EC.3 Example of non-ng-arb

Let $N_1 = \{1, 3\}, N_2 = \{2, 3\}, N_3 = \{3, 4\}, N_4 = \{4, 5\}, N_5 = \{5, 1\}$. As the vertex 3 appears twice in this q -arb in position 4 and 3, and for every vertex j of the path $P_{\lambda_{ng}}(4, 3) = (3, 2, 1, 3)$, vertex 3 is always included in N_j , such type of q -arbs are forbidden in ng -arb generation.

EC.5. Label Pruning

EC.5.1. Backward Ng-arb Recursion for CMST-E

We first define a backward q -arb $\bar{\lambda}$, hanging from *Root* 0 and is associated with a core vertex i . The vertex set $\Pi(\bar{\lambda})$ computed by expression (6) is also assigned to each backward q -arb. The difference is that the core vertex for a q -arb is always its root, but this is not the case for a backward q -arb. For example, let $\lambda = (1, 2, 3, 0)$ be a backward q -arb with core vertex 3, associated with three arcs : $(3, 1)$, $(2, 1)$ and $(1, 0)$, and let $N_1 = \{1, 2\}$, $N_2 = \{2, 3\}$, $N_3 = \{3, 1\}$. Obviously, the core vertex of this backward q -arb is one of its leaf node. Then, $2 \notin \bigcap_{k \in P_\lambda(2,3)} = N_2 \cap N_1 \cap N_3$ and $1 \in \bigcap_{k \in P_\lambda(1,3)} = N_1 \cap N_3$. Thus, from expression (6), we have $\Pi(\bar{\lambda}) = \{1, 3\}$.

Similar to the extension functions of backward labeling, backward q -arbs are also created by combining existing *backward* q -arbs and q -arbs. Define $\bar{\lambda}_1$ and $\bar{\lambda}_2$ as the *backward* q -arbs and q -arbs, respectively, used to combine $\bar{\lambda}$. A *backward ng-arb* (NG, i, j, q) is defined as a special *backward* q -arb $\bar{\lambda}$ covering a subset of q vertices in set $\mathcal{V}^+$, hangs from *Root* 0 such that $h \leq j$ for every child and parent vertex $h \in \mathcal{V}^+$ of the core vertex i , $NG = \Pi(\bar{\lambda})$ and $\Pi(\bar{\lambda}_1) \cap \Pi(\bar{\lambda}_2) = \emptyset$. Let $f^{-1}(NG, i, j, q)$ be the cost of the least cost *backward ng-tree* (NG, i, j, q) , which can be computed using dynamic programming as follows.

$$f^{-1}(NG, i, j, q) = \begin{cases} f^{-1}(NG, i, j-1, q) \\ \min\{f^{-1}(NG_1, i, j-1, q-q') + c_{j,i} + f(NG_2, j, |\mathcal{V}^+|, q') : 1 \leq q' \leq q-1, \forall NG_1 \in N_i, \\ NG_2 \in N_j \text{ s.t. } NG_1 \ni i, NG_2 \ni j, NG_1 \cup (NG_2 \cap N_i) = NG \text{ and } NG_1 \cap NG_2 = \emptyset \\ \min\{f(NG_1, i, j-1, q-q') + c_{i,j} + f^{-1}(NG_2, j, |\mathcal{V}^+|, q') : 1 \leq q' \leq q-1, \forall NG_1 \in N_i, \\ NG_2 \in N_j \text{ s.t. } NG_1 \ni i, NG_2 \ni j, NG_1 \cup (NG_2 \cap N_i) = NG \text{ and } NG_1 \cap NG_2 = \emptyset \\ f(NG, i, j, q) + c_{i,0} \end{cases} \quad (\text{EC.15})$$

where the initialization $f^{-1}(\{i\}, i, j, 1) = c_{i,0}, \forall i \in \mathcal{V}^+, j \in \mathcal{V}^+$ are required.

EC.5.2. Ng-arb Recursion for CMST-L

The modified recursion function for *ng-arb* in CMST-L case is shown as follows :

$$f(NG, i, j, q) = \min \begin{cases} f(NG, i, j-1, q) \\ f(NG', j, |\mathcal{V}^+|, q-1) + c_{j,i,q-1} : \forall NG' \in N_j \text{ s.t. } NG' \ni j, NG' \not\ni i \text{ and } NG' \cap N_i = NG \\ \min\{f(NG_1, i, j-1, q-q') + c_{j,i,q'} + f(NG_2, j, |\mathcal{V}^+|, q') : 1 \leq q' \leq q-1, \forall NG_1 \in N_i, \\ NG_2 \in N_j \text{ s.t. } NG_1 \ni i, NG_2 \ni j, NG_1 \cap NG_2 = \emptyset \text{ and } NG_1 \cup (NG_2 \cap N_i) = NG \end{cases} \quad (\text{EC.16})$$

EC.5.3. Backward Ng-arb Recursion for CMST-L

The modified recursion function for backward *ng-arb* in CMST-L case is shown as follows :

$$f^{-1}(NG, i, j, w) = \begin{cases} f^{-1}(NG, i, j-1, w) \\ \min\{f^{-1}(NG_1, i, j-1, w+w') + c_{j,i,w'} + f(NG_2, j, |\mathcal{V}^+|, w')\} : 1 \leq w' \leq Q-w-1, \forall NG_1 \in N_i, \\ NG_2 \in N_j \text{ s.t. } NG_1 \ni i, NG_2 \ni j, NG_1 \cup (NG_2 \cap N_i) = NG \text{ and } NG_1 \cap NG_2 = \emptyset \\ \min\{f(NG_1, i, j-1, w') + c_{i,j,w+w'} + f^{-1}(NG_2, j, |\mathcal{V}^+|, w+w')\} : 1 \leq w' \leq Q-w-1, \forall NG_1 \in N_i, \\ NG_2 \in N_j \text{ s.t. } NG_1 \ni i, NG_2 \ni j, NG_1 \cup (NG_2 \cap N_i) = NG \text{ and } NG_1 \cap NG_2 = \emptyset \\ \min\{f(NG, i, j, q) + c_{i,0,w+q}\} : 1 \leq q \leq Q-w \end{cases} \quad (\text{EC.17})$$

where the initialization $f^{-1}(\{i\}, i, j, Q-1) = c_{i,0,Q}, \forall i \in \mathcal{V}^+, j \in \mathcal{V}^+$ are required.

EC.6. Detailed Experimental Results

EC.6.1. Arc Elimination Results

Table EC.1 reports the number of arcs removed (No.Remove) and the percentage of this number relative to the total number of arcs $|\mathcal{A}|$ (P.Remove(%)) for each CMST-E instance. It can be seen that instances of the "tc" group have the most removed arcs. This is because the center position (50, 50) of *Root* 0 makes arcs $(i, 0), i \in \mathcal{V}$ have a relatively small cost, which allows more arcs to be removed by the rule $c_{i,0} < c_{i,j}$. In contrast, the corner position (0, 0) of *Root* 0 in the "te" group instances makes arcs $(i, 0), i \in \mathcal{V}$ have a relatively large cost, which will lead to fewer removed arcs. In addition, because the instances of the "td" group have the *Root* 0 positioned far from the non-root vertex (-201, -201), no arcs will be removed.

Similar findings can be observed in Table EC.2-EC.4 for CMST-L instances. The group "c" with a center *Root* position removes the most arcs, while the group "e" with an edge *Root* position removes the fewest arcs. Furthermore, because the group "r" randomly positions the *Root*, the number of removed arcs exhibits substantial variation across instances with distinct "id".

EC.6.2. Dichotomy labeling vs Order labeling

Table EC.1 Detailed Experimental Results for Eliminating Arcs in CMST-E

No.Vertex	No.Arc	id	tc		te		td	
			No.Remove	P.Remove(%)	No.Remove	P.Remove(%)	No.Remove	P.Remove(%)
80	6320	1	4458	70.54%	1799	28.47%	0	0.00%
		2	4396	69.56%	1509	23.88%	0	0.00%
		3	4462	70.60%	1451	22.96%	0	0.00%
		4	4430	70.09%	1407	22.26%	0	0.00%
		5	4396	69.56%	1739	27.52%	0	0.00%
120	14280	1	10099	70.72%	3178	22.25%	-	-
		2	8005	56.06%	2197	15.39%	-	-
		3	7309	51.18%	2007	14.05%	-	-
		4	7671	53.72%	1731	12.12%	-	-
		5	7947	55.65%	2060	14.43%	-	-
160	15440	1	11497	74.46%	5134	33.25%	-	-

EC.6.3. PT-RT label joining vs Classical label joining**EC.6.4. Ng-arb pruning vs Q-arb pruning**

EC.6.5. Detailed experiments on CMST-E

EC.6.6. Detailed experimental results on CMST-L

EC.6.7. Detailed experimental results on CMST-F

References

- Araque, J. R., Hall, L. A., and Magnanti, T. L. (1990). Capacitated trees, capacitated routing, and associated polyhedra.
- Archetti, C., Bouchard, M., and Desaulniers, G. (2011). Enhanced branch and price and cut for vehicle routing with split deliveries and time windows. *Transportation Science*, 45(3):285–298.
- Baldacci, R., Mingozzi, A., and Roberti, R. (2011). New route relaxation and pricing strategies for the vehicle routing problem. *Operations research*, 59(5):1269–1283.
- Boland, N., Dethridge, J., and Dumitrescu, I. (2006). Accelerated label setting algorithms for the elementary resource constrained shortest path problem. *Operations Research Letters*, 34(1):58–68.
- Borůvka, O. (1926). Příspěvek k otázce ekonomické stavby elektrovodných sítí.
- Boyd, E. A. (1994). Fenchel cutting planes for integer programs. *Operations Research*, 42(1):53–64.
- Chandy, K. M. and Russell, R. A. (1972). The design of multipoint linkages in a teleprocessing tree network. *IEEE Transactions on Computers*, 100(10):1062–1066.
- Danda, S., Challa, A., Sagar, B. D., and Najman, L. (2021). A tutorial on applications of power watershed optimization to image processing. *The European Physical Journal Special Topics*, 230(10):2337–2361.
- de Aragao, M. P. and Uchoa, E. (2003). Integer program reformulation for robust branch-and-cut-and-price algorithms. In *Mathematical program in rio: a conference in honour of nelson maculan*, pages 56–61.
- Desaulniers, G. (2010). Branch-and-price-and-cut for the split-delivery vehicle routing problem with time windows. *Operations research*, 58(1):179–192.
- Esau, L. R. and Williams, K. C. (1966). On teleprocessing system design, part ii: A method for approximating the optimal network. *IBM Systems Journal*, 5(3):142–147.
- Gamvros, I., Golden, B., and Raghavan, S. (2006). The multilevel capacitated minimum spanning tree problem. *INFORMS Journal on Computing*, 18(3):348–365.
- Gavish, B. (1982). Topological design of centralized computer networks—formulations and algorithms. *Networks*, 12(4):355–377.
- Gavish, B. (1983). Formulations and algorithms for the capacitated minimal directed tree problem. *Journal of the ACM (JACM)*, 30(1):118–132.
- Gouveia, L. (1995). A $2n$ constraint formulation for the capacitated minimal spanning tree problem. *Operations research*, 43(1):130–141.
- Gouveia, L. and Lopes, M. J. (2005). The capacitated minimum spanning tree problem: On improved multistar constraints. *European Journal of Operational Research*, 160(1):47–62.
- Gouveia, L., Lucena, A., and Paixão, J. (1990). A set-partitioning approach for the capacitated minimal spanning tree problem. In *1st ORSA Telecommunications SIG Conference*, Boca Raton, FL.
- Gouveia, L. and Martins, P. (1999). The capacitated minimal spanning tree problem: An experiment with a hop-indexed model. *Annals of Operations Research*, 86(0):271–294.
- Gouveia, L. and Martins, P. (2000). A hierarchy of hop-indexed models for the capacitated minimum spanning tree problem. *Networks: An International Journal*, 35(1):1–16.
- Gouveia, L. and Martins, P. (2005). The capacitated minimum spanning tree problem: revisiting hop-indexed formulations. *Computers & operations research*, 32(9):2435–2452.
- Graham, R. L. and Hell, P. (2007). On the history of the minimum spanning tree problem. *Annals of the History of Computing*, 7(1):43–57.
- Hall, L. (1996). Experience with a cutting plane algorithm for the capacitated spanning tree problem. *INFORMS Journal on Computing*, 8(3):219–234.
- Kawatra, R. and Bricker, D. (2000). A multiperiod planning model for the capacitated minimal spanning tree problem. *European Journal of Operational Research*, 121(2):412–419.

- Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. Proceedings of the American Mathematical society, 7(1):48–50.
- Liu, X. (2015). Atypical hierarchical routing protocols for wireless sensor networks: A review. IEEE Sensors Journal, 15(10):5372–5383.
- Luo, Z., Qin, H., Cheng, T. E., Wu, Q., and Lim, A. (2021). A branch-and-price-and-cut algorithm for the cable-routing problem in solar power plants. INFORMS Journal on Computing, 33(2):452–476.
- Malik, K. and Yu, G. (1993). A branch and bound algorithm for the capacitated minimum spanning tree problem. Networks, 23(6):525–532.
- Papadimitriou, C. H. (1978). The complexity of the capacitated tree problem. Networks, 8(3):217–230.
- Pecin, D., Contardo, C., Desaulniers, G., and Uchoa, E. (2017). New enhancements for the exact solution of the vehicle routing problem with time windows. INFORMS Journal on Computing, 29(3):489–502.
- Prim, R. C. (1957). Shortest connection networks and some generalizations. The Bell System Technical Journal, 36(6):1389–1401.
- Ralphs, T. K., Ladányi, L., and Saltzman, M. J. (2003). Parallel branch, cut, and price for large-scale discrete optimization. Mathematical Programming, 98(1):253–280.
- Righini, G. and Salani, M. (2008). New dynamic programming algorithms for the resource constrained elementary shortest path problem. Networks: An International Journal, 51(3):155–170.
- Sevak, J. S., Kapadia, A. D., Chavda, J. B., Shah, A., and Rahevar, M. (2017). Survey on semantic image segmentation techniques. In 2017 International conference on intelligent sustainable systems (ICISS), pages 306–313. IEEE.
- Uchoa, E., Fukasawa, R., Lysgaard, J., Pessoa, A., De Aragao, M. P., and Andrade, D. (2008). Robust branch-cut-and-price for the capacitated minimum spanning tree problem over a large extended formulation. Mathematical Programming, 112(2):443–472.
- Uchoa, E., Toffolo, T. A., de Souza, M. C., Martins, A. X., and Fukasawa, R. (2012). Branch-and-cut and hybrid local search for the multi-level capacitated minimum spanning tree problem. Networks, 59(1):148–160.
- Wang, C., Wu, Q., Pan, K., and Shen, Z.-J. M. (2025). Layout optimization for a large-scale grid-connected solar power plant. INFORMS Journal on Computing, 37(6):1560–1586.
- Zhang, N. (1994). Facet-defining inequalities for capacitated spanning trees. PhD thesis, Princeton University.

Table EC.2 Detailed Experimental Results for Eliminating Arcs of CMST-L Instances with 50 vertices

No.Vertex	No.Arc	id	c		e		r	
			No.Remove	P.Remove(%)	No.Remove	P.Remove(%)	No.Remove	P.Remove(%)
50	2450	0	1777	72.53%	443	18.08%	1752	71.51%
		1	1711	69.84%	409	16.69%	1288	52.57%
		2	1690	68.98%	496	20.24%	1208	49.31%
		3	1756	71.67%	528	21.55%	1446	59.02%
		4	1787	72.94%	466	19.02%	883	36.04%
		5	1760	71.84%	440	17.96%	724	29.55%
		6	1742	71.10%	732	29.88%	1174	47.92%
		7	1732	70.69%	526	21.47%	1155	47.14%
		8	1696	69.22%	577	23.55%	1348	55.02%
		9	1728	70.53%	561	22.90%	1013	41.35%
		10	1688	68.90%	430	17.55%	756	30.86%
		11	1725	70.41%	443	18.08%	1424	58.12%
		12	1737	70.90%	611	24.94%	1286	52.49%
		13	1786	72.90%	457	18.65%	1115	45.51%
		14	1699	69.35%	547	22.33%	1106	45.14%
		15	1724	70.37%	471	19.22%	823	33.59%
		16	1769	72.20%	587	23.96%	1428	58.29%
		17	1775	72.45%	579	23.63%	1625	66.33%
		18	1731	70.65%	591	24.12%	1345	54.90%
		19	1697	69.27%	522	21.31%	1034	42.20%
		20	1766	72.08%	483	19.71%	894	36.49%
		21	1752	71.51%	509	20.78%	1075	43.88%
		22	1659	67.71%	506	20.65%	1192	48.65%
		23	1742	71.10%	571	23.31%	1170	47.76%
		24	1735	70.82%	401	16.37%	1117	45.59%
		25	1686	68.82%	547	22.33%	1523	62.16%
		26	1761	71.88%	515	21.02%	833	34.00%
		27	1761	71.88%	641	26.16%	1342	54.78%
		28	1735	70.82%	521	21.27%	1184	48.33%
		29	1689	68.94%	421	17.18%	733	29.92%
		30	1721	70.24%	524	21.39%	1110	45.31%
		31	1755	71.63%	464	18.94%	1391	56.78%
		32	1724	70.37%	431	17.59%	615	25.10%
		33	1735	70.82%	487	19.88%	682	27.84%
		34	1762	71.92%	588	24.00%	1147	46.82%
		35	1732	70.69%	354	14.45%	1707	69.67%
		36	1768	72.16%	507	20.69%	1402	57.22%
		37	1759	71.80%	661	26.98%	1178	48.08%
		38	1688	68.90%	564	23.02%	1484	60.57%
		39	1721	70.24%	499	20.37%	1330	54.29%
		40	1752	71.51%	581	23.71%	1571	64.12%
		41	1741	71.06%	646	26.37%	1187	48.45%
		42	1725	70.41%	623	25.43%	1698	69.31%
		43	1723	70.33%	613	25.02%	1437	58.65%
		44	1615	65.92%	605	24.69%	1698	69.31%
		45	1740	71.02%	529	21.59%	1086	44.33%
		46	1768	72.16%	464	18.94%	1698	69.31%
		47	1724	70.37%	555	22.65%	983	40.12%
		48	1763	71.96%	459	18.73%	1627	66.41%
		49	1756	71.67%	445	18.16%	1593	65.02%

Table EC.3 Detailed Experimental Results for Eliminating Arcs of CMST-L Instances with 100 vertices

No.Vertex	No.Arc	id	c		e		r	
			No.Remove	P.Remove(%)	No.Remove	P.Remove(%)	No.Remove	P.Remove(%)
100	9900	0	7000	70.71%	1811	18.29%	3906	39.45%
		1	7003	70.74%	2160	21.82%	2910	29.39%
		2	7075	71.46%	1923	19.42%	3840	38.79%
		3	6923	69.93%	2169	21.91%	6221	62.84%
		4	7041	71.12%	2115	21.36%	6863	69.32%
		5	7007	70.78%	2095	21.16%	6089	61.51%
		6	7041	71.12%	1985	20.05%	3868	39.07%
		7	6984	70.55%	1941	19.61%	6927	69.97%
		8	6855	69.24%	2386	24.10%	5521	55.77%
		9	7076	71.47%	2260	22.83%	4164	42.06%
		10	7012	70.83%	2346	23.70%	4340	43.84%
		11	7067	71.38%	1382	13.96%	2894	29.23%
		12	7024	70.95%	2528	25.54%	3373	34.07%
		13	6979	70.49%	2001	20.21%	5375	54.29%
		14	7071	71.42%	2222	22.44%	6036	60.97%
		15	6974	70.44%	2329	23.53%	4318	43.62%
		16	7048	71.19%	2135	21.57%	6639	67.06%
		17	7125	71.97%	2254	22.77%	5581	56.37%
		18	7049	71.20%	2093	21.14%	3548	35.84%
		19	6987	70.58%	1890	19.09%	6484	65.49%
		20	6926	69.96%	2465	24.90%	4143	41.85%
		21	7040	71.11%	1715	17.32%	4716	47.64%
		22	6968	70.38%	2267	22.90%	6623	66.90%
		23	6939	70.09%	2012	20.32%	6019	60.80%
		24	7016	70.87%	2263	22.86%	4251	42.94%
		25	7043	71.14%	2509	25.34%	4323	43.67%
		26	6963	70.33%	1936	19.56%	5250	53.03%
		27	6940	70.10%	2227	22.49%	4520	45.66%
		28	7061	71.32%	1962	19.82%	6812	68.81%
		29	7079	71.51%	1999	20.19%	6251	63.14%
		30	7079	71.51%	2453	24.78%	6196	62.59%
		31	7125	71.97%	2331	23.55%	4612	46.59%
		32	7009	70.80%	1956	19.76%	4943	49.93%
		33	7117	71.89%	1980	20.00%	3945	39.85%
		34	7000	70.71%	1905	19.24%	4187	42.29%
		35	7076	71.47%	1693	17.10%	3132	31.64%
		36	6957	70.27%	2356	23.80%	6477	65.42%
		37	6960	70.30%	2121	21.42%	3605	36.41%
		38	7043	71.14%	1990	20.10%	3335	33.69%
		39	7054	71.25%	1990	20.10%	5279	53.32%
		40	7028	70.99%	2240	22.63%	4136	41.78%
		41	6944	70.14%	2319	23.42%	6287	63.51%
		42	7051	71.22%	2487	25.12%	2963	29.93%
		43	6981	70.52%	1560	15.76%	4402	44.46%
		44	7039	71.10%	1883	19.02%	3410	34.44%
		45	6960	70.30%	1922	19.41%	4560	46.06%
		46	6864	69.33%	1990	20.10%	6447	65.12%
		47	6991	70.62%	2273	22.96%	5109	51.61%
		48	7010	70.81%	2044	20.65%	2308	23.31%
		49	7031	71.02%	2174	21.96%	3464	34.99%

Table EC.4 Detailed Experimental Results for Eliminating Arcs of CMST-L Instances with 150 vertices

No.Vertex	No.Arc	id	c		e		r	
			No.Remove	P.Remove(%)	No.Remove	P.Remove(%)	No.Remove	P.Remove(%)
150	22350	0	15778	70.60%	4861	21.75%	8475	37.92%
		1	15815	70.76%	4528	20.26%	11017	49.29%
		2	15797	70.68%	4635	20.74%	9951	44.52%
		3	15992	71.55%	5260	23.53%	10507	47.01%
		4	15662	70.08%	4761	21.30%	9833	44.00%
		5	15804	70.71%	4428	19.81%	6873	30.75%
		6	15777	70.59%	4435	19.84%	8611	38.53%
		7	15867	70.99%	4819	21.56%	11335	50.72%
		8	15958	71.40%	5034	22.52%	8161	36.51%
		9	16053	71.83%	4805	21.50%	11989	53.64%
		10	15966	71.44%	4413	19.74%	9658	43.21%
		11	16066	71.88%	4474	20.02%	12116	54.21%
		12	16030	71.72%	4813	21.53%	10346	46.29%
		13	15987	71.53%	4466	19.98%	11752	52.58%
		14	15677	70.14%	4463	19.97%	10077	45.09%
		15	15947	71.35%	5089	22.77%	8329	37.27%
		16	15961	71.41%	5036	22.53%	6573	29.41%
		17	15893	71.11%	4478	20.04%	8183	36.61%
		18	15884	71.07%	4750	21.25%	13234	59.21%
		19	16041	71.77%	4949	22.14%	9152	40.95%
		20	15608	69.83%	4439	19.86%	8389	37.53%
		21	15943	71.33%	4277	19.14%	11062	49.49%
		22	16071	71.91%	4045	18.10%	14375	64.32%
		23	15917	71.22%	4710	21.07%	13169	58.92%
		24	15835	70.85%	4994	22.34%	10378	46.43%
		25	16011	71.64%	5137	22.98%	14609	65.36%
		26	16005	71.61%	5015	22.44%	10990	49.17%
		27	15611	69.85%	5137	22.98%	5398	24.15%
		28	15863	70.98%	5333	23.86%	13255	59.31%
		29	15892	71.11%	4434	19.84%	14320	64.07%
		30	15908	71.18%	4180	18.70%	12727	56.94%
		31	15890	71.10%	4574	20.47%	8477	37.93%
		32	15872	71.02%	4829	21.61%	15401	68.91%
		33	16018	71.67%	4753	21.27%	9133	40.86%
		34	15950	71.36%	4735	21.19%	9281	41.53%
		35	15596	69.78%	4788	21.42%	15535	69.51%
		36	15952	71.37%	4729	21.16%	10495	46.96%
		37	15733	70.39%	5235	23.42%	12819	57.36%
		38	15950	71.36%	4363	19.52%	10093	45.16%
		39	15683	70.17%	5325	23.83%	9628	43.08%
		40	15638	69.97%	4757	21.28%	8874	39.70%
		41	15789	70.64%	4760	21.30%	11746	52.55%
		42	15595	69.78%	4763	21.31%	11990	53.65%
		43	15862	70.97%	5379	24.07%	8334	37.29%
		44	15659	70.06%	5126	22.94%	15154	67.80%
		45	15939	71.32%	5019	22.46%	11544	51.65%
		46	15814	70.76%	5349	23.93%	14563	65.16%
		47	15818	70.77%	4426	19.80%	8372	37.46%
		48	15905	71.16%	4857	21.73%	13355	59.75%
		49	15862	70.97%	4670	20.89%	10064	45.03%

Table EC.5 Detailed Experimental Results for Dichotomy and Order Labeling

instance	id	LP Cost	Dichotomy Labeling					Order Labeling				
			No.Nd Label	No.Ex Label	No.Co Label	No.Co Count	LP Time	No.Nd Label	No.Ex Label	No.Co Label	No.Co Count	LP Time
tc-80-10	1	876.01	1142	887	793	6503	1.19	1151	849	844	392790	2.56
	2	876.80	1098	1048	715	6448	0.74	1002	940	757	333113	1.4
	3	872.56	1464	1197	1040	9010	0.91	975	711	708	323030	1.61
	4	865.73	546	525	297	1545	0.63	552	533	360	112473	1.43
	5	1001.37	1608	1294	1324	9873	0.8	1601	1222	1239	763532	1.87
te-80-10	1	1646.62	1568	1639	1287	31743	1.04	1458	1619	1571	932250	1.76
	2	1619.66	680	639	326	2710	1.21	1098	1446	1315	570603	1.86
	3	1672.43	794	753	437	3987	1.21	826	898	692	277164	1.83
	4	1627.50	1097	1073	827	11897	1.19	1517	1523	1238	868992	2.37
	5	1595.04	897	1061	800	7813	1.39	771	903	662	248372	3.01
td-80-10	1	3211.14	1864	3913	2491	91708	1.8	2134	4609	3289	2345577	3.13
	2	3201.20	2710	4321	3788	108885	1.77	3070	4948	4939	3972790	3.07
	3	3197.06	2424	3605	3330	96538	1.78	2059	3042	2710	1775154	2.98
	4	3194.14	3291	5152	6146	236249	1.74	2836	4518	5062	3537748	3.35
	5	3175.08	2002	2730	2387	62824	1.59	2429	3215	3781	2314593	2.85
tc-120-10	1	885.50	3856	4775	4750	115184	1.9	4249	5725	7701	5049813	8.35
	2	742.04	4255	5488	4663	143734	2.45	4428	5618	6168	6473908	7.48
	3	709.30	3394	4060	3501	79522	2.43	2675	3089	3216	3058043	7.12
	4	720.19	4524	6270	5363	133046	2.95	4308	5793	7404	6533991	8.91
	5	743.80	6217	9098	7514	380495	2.27	4507	5961	6949	7486417	7.1
te-120-10	1	1323.23	3278	3512	2933	56456	3.88	2344	2620	2812	2073260	7.53
	2	1219.91	2842	4059	2727	68776	3.99	2444	3981	3935	2601144	6.6
	3	1187.07	2635	3646	2977	67838	2.96	2856	3778	3321	3516422	7.62
	4	1223.86	5436	10417	9943	739839	3.61	5469	10278	12161	16107679	8.86
	5	1158.80	3261	4987	3782	122201	3.17	2931	7819	8171	4956268	6.85
tc-160-10	1	1306.79	3928	5094	4086	87609	5.61	4654	5890	5499	8589067	18.57
te-160-10	1	1633.81	3581	6181	5273	135022	7.41	4035	6684	7615	7344413	15.48

Table EC.6 Detailed Experimental Results for PT-RT joining and Classical joining

PT-RT Joining								Classic Joining			
instance	id	LP Cost	No.P1 Label	No.P2 Label	No.R Label	Join Time	LP Time	No.F Label	No.B Label	Join Time	LP Time
tc-80-10	1	876.01	924	89	452	0.03	1.11	847	375	0.81	3.11
	2	876.8	1002	39	815	0.02	0.83	1157	927	0.86	2.03
	3	872.56	1036	48	964	0.01	0.84	998	1051	0.78	1.81
	4	865.73	471	21	422	0.01	0.61	538	451	0.61	1.44
	5	1001.37	1203	20	2126	0.01	0.81	1068	1921	1.0	2.39
te-80-10	1	1646.62	1719	76	2071	0.01	1.13	849	1887	0.99	2.27
	2	1619.66	775	5	1610	0.01	1.26	1004	2359	1.21	2.71
	3	1672.42	658	7	2102	0.01	1.2	676	2287	1.15	2.79
	4	1627.5	966	23	1796	0.01	1.22	1260	1595	1.15	2.71
	5	1595.04	759	47	2221	0.01	1.43	848	1603	1.04	2.46
td-80-10	1	3211.14	3365	12	7360	0.03	2.17	2651	5374	2.74	5.94
	2	3201.2	2818	50	4548	0.02	1.96	2457	4338	2.5	4.99
	3	3197.06	2349	51	6333	0.02	1.76	1718	5973	2.86	5.65
	4	3194.14	2700	454	2658	0.02	1.83	2424	2501	2.08	4.25
	5	3175.08	1921	118	2153	0.02	1.53	1215	2204	1.49	3.35
tc-120-10	1	885.5	2970	179	5924	0.05	2.19	3019	4398	4.87	9.73
	2	742.04	3563	211	5315	0.06	2.58	2269	3606	5.48	10.83
	3	709.3	3435	91	5888	0.05	2.53	3215	5652	5.86	11.32
	4	720.19	3677	166	6531	0.06	3.05	3378	7007	9.43	17.02
	5	743.8	4857	282	5485	0.05	2.32	4156	8049	8.11	17.89
te-120-10	1	1323.23	2818	347	11829	0.06	3.98	1727	5459	5.11	10.37
	2	1219.91	2677	37	4426	0.04	3.93	2269	3591	4.92	9.8
	3	1187.07	2761	58	5471	0.05	3.21	2701	7808	7.48	16.42
	4	1223.86	3849	162	8882	0.06	4.15	4342	16120	15.97	38.17
	5	1158.8	2267	54	2795	0.04	3.4	2142	3032	4.92	10.38
tc-160-10	1	1306.79	4163	157	9908	0.15	5.67	2768	7196	13.76	27.52
te-160-10	2	1633.81	3950	73	5967	0.13	7.91	3133	4758	13.68	25.56

Table EC.7 Detailed Experimental Results for Ng-arb and Q-arb pruning

		Ng-arb						Q-arb					
instance	id	IP Cost	Max.P1 Label	Max.P2 Label	Max.R Label	Pr Time	IP Time	Max.P1 Label	Max.P2 Label	Max.R Label	Pr Time	IP Time	
tc-80-10	1	888.0	8652	5115	10123	2.62	42.81	9158	4554	12887	0.2	49.95	
	2	877.0	693	389	321	0.05	0.81	1002	428	815	0.01	0.82	
	3	878.0	7260	5918	3188	1.22	21.88	6729	5745	7878	0.09	22.16	
	4	868.0	408	137	211	0.16	6.19	653	195	708	0.03	5.07	
	5	1002.0	723	450	273	0.08	0.87	1420	555	2126	0.01	0.89	
te-80-10	1	1657.0	2210	388	827	0.45	7.67	1884	773	3273	0.05	8.57	
	2	1639.0	12247	3498	14478	1.66	14.53	12318	3685	25694	0.12	16.2	
	3	1687.0	11301	1338	16012	2.18	24.41	15088	2084	33969	0.13	28.46	
	4	1629.0	637	337	341	0.19	1.65	1126	385	2951	0.02	1.69	
	5	1603.0	3008	263	1723	0.47	11.92	1772	367	6367	0.08	13.24	
td-80-10	1	3223.0	89389	127402	78839	11.63	288.62	550429	352900	129935	0.2	730.28	
	2	3205.0	916	709	766	0.85	9.57	4452	1069	7606	0.04	9.18	
	3	3212.0	28639	58195	42919	13.29	124.61	381591	8462763	95316	0.22	500.52	
	4	3203.0	9849	1366	7268	2.25	19.3	14289	2434	24561	0.08	22.18	
	5	3180.0	2390	542	1183	0.97	11.29	4292	753	4278	0.05	10.98	
tc-120-10	1	904.0	45317	225754	42799	7.26	143.43	47927	152389	74137	0.42	216.68	
	2	756.0	14783	5109	13645	13.23	197.41	37408	34378	49623	0.75	236.65	
	3	722.0	34464	51804	27792	6.99	112.74	31626	31389	51230	0.48	157.02	
	4	722.0	773	1370	536	0.65	8.29	4433	2158	20018	0.04	8.29	
	5	761.0	28802	34110	47695	9.73	158.18	32301	79811	71189	0.5	213.27	
te-120-10	1	1329.0	6048	851	5881	3.07	40.39	11808	1781	41368	0.18	41.96	
	2	1225.0	8335	749	8498	3.35	28.36	11569	1089	21022	0.28	33.42	
	3	1195.0	19691	5911	31539	23.4	180.45	41423	12274	85095	1.11	485.2	
	4	1229.0	11665	1215	13198	5.4	45.04	23065	1582	93484	0.21	99.06	
	5	1164.0	3865	655	4494	4.17	35.02	9152	1063	12371	0.18	30.06	
tc-160-10	1	1318.0	28867	28762	54798	11.77	212.55	39086	16834	91293	0.88	231.23	
te-160-10	1	1645.0	19421	3022	90624	15.1	247.61	45483	21402	74399	1.04	284.53	

Table EC.8 Detailed Experimental Results for CMST-E

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
tc-80-5	1	1099.0	0.44	0.71	0.05	1.34	30	0	0	35	0	35	0	1	0.01	0.06	0	26.67
	2	1100.0	0.97	0.1	0.08	12.62	113	0	99	101	20	101	20	1	0.01	0.08	0	38.19
	3	1073.0	0.74	0.12	0.08	9.21	95	0	103	102	20	102	20	1	0.01	0.03	0	34.81
	4	1080.0	0.99	0.11	0.07	4.68	148	0	135	81	40	81	40	1	0.01	0.04	0	30.41
	5	1287.0	1.34	0.08	0.30	10.46	194	0	156	103	60	129	140	5	0.01	0.19	0	42.95
te-80-5	1	2544.0	0.56	0.14	0.04	3.04	52	0	10	49	0	49	0	1	0.01	0.06	0	28.72
	2	2551.0	0.91	0.24	0.03	6.13	238	0	341	109	60	109	60	1	0.01	0.17	0	32.03
	3	2612.0	0.77	0.4	0.13	6.93	229	0	274	99	80	120	100	7	0.01	0.14	0	36.8
	4	2558.0	0.58	0.45	0.15	5.69	255	0	321	103	80	116	160	9	0.01	0.24	0	36.38
	5	2469.0	0.31	0.39	0.04	9.07	53	0	5	100	0	100	0	1	0.01	0.04	0	34.98
td-80-5	1	6068.0	0.23	0.18	0.03	14.69	242	0	419	119	120	119	120	3	0.01	0.19	0	41.53
	2	6019.0	0.11	1.22	0.01	9.18	301	0	364	70	60	70	60	1	0.01	0.1	0	35.1
	3	5994.0	0.12	1.23	0.01	5.2	270	0	395	76	60	76	60	1	0.01	0.09	0	31.28
	4	6012.0	0.18	0.18	0.01	8.67	117	0	64	86	20	86	20	1	0.01	0.1	0	34.67
	5	5977.0	0.20	0.81	0.02	9.26	98	0	48	100	20	100	20	1	0.01	0.07	0	35.14
tc-120-5	1	1291.0	1.54	0.65	0.03	27.26	664	0	1007	158	160	175	200	5	0.01	0.49	0	78.12
	2	1189.0	1.73	0.22	0.10	16.71	531	0	684	158	140	227	240	8	0.01	0.68	0	76.37
	3	1124.0	1.11	0.29	0.12	24.22	644	0	624	160	120	160	120	5	0.01	0.59	0	71.7
	4	1126.0	0.91	0.35	0.08	19.2	413	0	424	113	120	113	120	1	0.01	0.36	0	61.93
	5	1158.0	1.23	0.31	0.06	25.25	589	0	754	140	140	140	140	1	0.01	0.35	0	68.06
te-120-5	1	2197.0	0.44	0.51	0.04	13.75	177	0	238	114	20	114	20	1	0.01	0.3	0	56.77
	2	2134.0	0.42	1.77	0.09	21.21	570	0	745	138	180	156	220	7	0.02	0.61	0	75.71
	3	2079.0	0.37	1.37	0.04	29.29	582	0	794	161	120	161	120	1	0.01	0.41	0	71.84
	4	2158.0	0.46	1.6	0.08	26.73	546	0	787	145	160	153	160	5	0.01	0.5	0	75.07
	5	2017.0	0.62	1.5	0.04	19.84	182	0	206	135	40	135	40	1	0.01	0.26	0	61.62
tc-160-5	1	2077.0	0.66	1.34	0.04	40.25	543	0	887	316	120	316	120	1	0.01	0.78	0	162.82
te-160-5	1	2789.0	0.40	2.06	0.06	53.63	599	0	833	184	180	202	200	15	0.06	1.54	0	222.94
tc-80-10	1	888.0	1.35	2.56	0.10	42.93	8349	4666	9840	127	280	127	280	1	0.3	2.63	0	147.05
	2	877.0	0.02	0.82	0.02	0.82	577	15	321	0	0	0	0	1	0.01	0.06	0	104.43
	3	878.0	0.62	2.42	0.08	21.46	7183	5533	3188	69	200	69	200	1	0.08	1.14	0	125.3
	4	868.0	0.26	1.5	0.11	6.08	155	1	31	41	0	41	0	1	0.01	0.17	0	112.4
	5	1002.0	0.06	0.88	0.06	0.88	723	15	221	0	0	0	0	1	0.01	0.08	0	105.35
te-80-10	1	1657.0	0.63	2.13	0.00	7.25	706	15	280	45	20	45	20	1	0.02	0.39	0	111.77
	2	1639.0	1.18	2.35	0.04	14.5	11397	2826	13307	84	220	84	220	1	0.18	1.67	0	119.55
	3	1687.0	0.86	2.08	0.05	24.75	11241	1141	15360	63	280	63	280	1	0.2	1.97	0	128.47
	4	1629.0	0.09	1.26	0.00	1.64	375	11	133	4	0	4	0	1	0.01	0.18	0	105.48
	5	1603.0	0.50	2.68	0.05	11.77	1999	95	1588	32	80	32	80	1	0.02	0.45	0	115.54
td-80-10	1	3223.0	0.37	3.26	0.09	58.45	23824	2581	48563	56	260	68	460	9	15.81	11.18	0	393.27
	2	3205.0	0.12	3.6	0.01	9.41	694	28	587	29	20	29	20	1	0.02	0.89	0	113.76
	3	3212.0	0.47	3.04	0.13	28.1	16887	3125	18367	39	260	57	600	13	9.94	14.6	0	249.5
	4	3203.0	0.28	3.32	0.02	19.23	7568	688	6749	46	140	46	140	1	0.07	2.38	0	122.52
	5	3180.0	0.15	2.65	0.01	11.24	2152	77	1092	36	80	36	80	1	0.02	0.84	0	115.32
tc-120-10	1	904.0	2.05	2.44	0.08	100.31	15710	12916	24363	174	420	188	500	3	7.41	7.01	0	308.75
	2	756.0	1.85	2.92	0.35	72.67	6047	676	10048	133	300	155	640	15	1.44	12.97	0	360.33
	3	722.0	1.76	3.82	0.22	75.08	26582	21262	23792	155	320	169	400	3	3.57	7.28	0	279.82
	4	722.0	0.25	5.24	0.13	8.57	605	3	376	14	20	14	20	1	0.04	0.64	0	176.47
	5	761.0	2.26	3.73	0.28	84.51	14518	7887	37294	163	380	169	480	5	5.22	9.56	0	324.32
te-120-10	1	1329.0	0.43	5.41	0.07	39.33	5341	772	5097	42	200	42	200	1	0.11	3.01	0	209.91
	2	1225.0	0.42	5.33	0.08	28.02	6471	318	5734	73	160	73	160	1	0.09	3.14	0	198.61
	3	1195.0	0.66	4.2	0.18	44.48	10696	1325	11010	72	400	87	860	13	2.51	23.06	0	348.53
	4	1229.0	0.42	4.73	0.07	43.97	11298	1146	11069	64	240	64	240	1	0.22	5.26	0	216.69
	5	1164.0	0.45	4.8	0.07	34.32	3687	176	4330	68	160	68	160	1	0.07	4.09	0	203.02
tc-160-10	1	1318.0	0.85	7.64	0.09	157.91	15210	2562	46305	123	500	127	560	3	3.46	12.14	0	684.27
te-160-10	1	1645.0	0.68	10.07	0.12	167.36	17666	1415	90535	117	480	128	560	5	2.84	14.49	0	726.29

Table EC.9 Detailed Experimental Results for group 50c in CMST-L

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
50c	0	568.48	0.52	0.26	0.09	0.9	2505	996	1582	9	60	9	60	5	0.06	0.49	0.0	1.42
	1	540.62	1.57	0.1	0.96	0.56	5757	949	3118	14	100	14	1060	419	2.03	10.93	0.0	47.82
	2	558.66	0.28	0.09	0.28	0.09	2153	451	1476	0	0	1	40	11	0.02	0.17	0.0	0.56
	3	564.28	0.78	0.06	0.05	0.61	5349	3470	3677	7	140	7	140	3	0.03	0.49	0.0	1.03
	4	541.68	0.61	0.09	0.22	0.41	5566	1620	3490	12	100	58	400	61	0.48	2.46	0.0	8.33
	5	608.16	0.71	0.05	0.09	0.4	2958	1002	1932	3	120	4	120	11	0.06	0.49	0.0	1.38
	6	571.43	1.32	0.08	0.41	1.22	5658	2505	3811	1	180	1	940	141	1.42	17.63	0.0	39.82
	7	580.53	0.40	0.04	0.08	0.21	2294	926	1356	11	80	13	140	7	0.03	0.32	0.0	0.93
	8	616.96	0.14	0.1	0.00	0.18	3969	1341	2480	0	40	0	40	1	0.01	0.04	0.0	0.18
	9	635.48	0.29	0.07	0.04	0.2	3378	850	2140	0	60	0	60	3	0.01	0.09	0.0	0.32
	10	557.31	0.62	0.09	0.28	1.06	8763	8356	4799	12	100	12	880	53	3.78	20.26	0.0	43.6
	11	592.57	0.74	0.09	0.20	0.58	7224	2246	4404	1	80	2	220	21	0.64	2.89	0.0	9.18
	12	630.49	0.75	0.07	0.30	0.19	1580	383	1008	3	60	6	280	9	0.07	0.69	0.0	1.8
	13	541.06	0.69	0.05	0.47	0.13	2032	491	1448	8	20	69	800	123	0.22	2.89	0.0	8.84
	14	565.96	0.46	0.14	0.17	0.37	3525	1394	2214	1	60	1	100	3	0.03	0.31	0.0	0.84
	15	560.88	1.09	0.09	0.32	4.25	17289	19924	9628	2	140	2	300	21	7.2	14.15	0.0	48.43
	16	577.77	0.39	0.11	0.10	0.28	4086	2349	2334	7	80	7	140	3	0.09	0.29	0.0	1.04
	17	570.14	0.34	0.04	0.18	0.07	1929	423	1338	0	20	0	20	7	0.01	0.09	0.0	0.33
	18	605.11	0.24	0.09	0.00	0.31	3647	1338	1868	13	100	13	100	1	0.01	0.1	0.0	0.31
	19	608.21	0.45	0.07	0.01	3.9	10347	5182	5640	43	280	44	320	5	0.31	3.54	0.0	6.66
	20	561.3	1.51	0.05	0.57	0.75	6200	1879	4076	1	160	3	1180	227	1.47	16.25	0.0	37.28
	21	593.01	0.68	0.05	0.00	0.37	4943	1947	2843	1	40	1	40	1	0.02	0.11	0.0	0.37
	22	565.35	0.31	0.08	0.07	0.36	5870	1551	3385	1	60	1	100	3	0.03	0.2	0.0	0.66
	23	573.49	0.76	0.08	0.12	1.03	7480	2967	4489	8	140	8	140	11	0.35	1.54	0.0	4.81
	24	623.63	0.93	0.06	0.00	0.14	2924	1037	1824	1	20	1	20	1	0.0	0.04	0.0	0.14
	25	554.54	0.02	0.08	0.00	0.1	2621	1096	1630	8	0	8	0	1	0.0	0.02	0.0	0.1
	26	570.78	0.45	0.03	0.00	0.1	1857	316	1204	1	40	1	40	1	0.0	0.04	0.0	0.1
	27	564.58	0.34	0.06	0.00	0.2	2059	876	1401	1	60	1	60	1	0.01	0.07	0.0	0.2
	28	581.97	0.27	0.08	0.01	0.79	6061	4091	4131	2	140	2	160	3	0.1	0.68	0.0	1.47
	29	572.33	0.81	0.1	0.34	0.19	2441	949	1623	2	20	4	140	3	0.03	0.22	0.0	0.71
	30	601.98	0.41	0.06	0.00	0.14	3116	1147	1911	1	40	1	40	1	0.01	0.05	0.0	0.14
	31	582.82	0.86	0.04	0.33	0.37	4105	1792	3272	1	100	2	260	47	0.2	2.56	0.0	5.31
	32	591.41	0.92	0.04	0.33	0.36	3248	1105	1707	1	60	4	620	65	0.82	5.63	0.0	14.66
	33	562.66	0.58	0.06	0.14	0.15	2670	792	1560	1	40	1	60	3	0.01	0.13	0.0	0.36
	34	596.99	0.80	0.06	0.45	0.14	1963	462	1170	11	20	17	320	49	0.08	0.96	0.0	3.21
	35	574.19	0.95	0.06	0.51	0.22	2743	1173	1924	1	60	13	2020	217	2.21	16.23	0.0	46.25
	36	579.59	0.48	0.05	0.00	0.38	4003	1041	2232	8	100	8	100	1	0.01	0.12	0.0	0.38
	37	525.88	0.38	0.05	0.02	0.21	2512	694	1629	10	60	10	60	5	0.01	0.14	0.0	0.46
	38	595.33	0.87	0.07	0.04	1.8	15029	9033	7523	19	140	19	140	3	0.37	1.24	0.0	3.72
	39	549.04	1.59	0.08	0.86	1.52	9425	7195	6555	8	80	43	3100	253	8.24	116.79	0.0	188.66
	40	554.15	0.28	0.05	0.00	0.18	2097	1028	1224	4	60	4	60	1	0.0	0.05	0.0	0.18
	41	578.87	0.71	0.07	0.39	0.48	3178	786	1982	19	120	72	1300	91	0.65	5.59	0.0	16.56
	42	581.19	0.18	0.1	0.18	0.1	1297	382	771	0	0	36	100	7	0.04	0.22	0.0	0.95
	43	581.14	1.20	0.05	0.11	0.49	4457	2686	2686	2	120	3	160	7	0.16	0.71	0.0	2.13
	44	584.5	0.49	0.13	0.49	0.17	1991	714	1225	11	20	66	1080	73	1.23	7.18	0.0	22.21
	45	581.79	0.91	0.06	0.12	0.66	6164	2610	3824	2	140	2	200	9	0.13	0.82	0.0	2.44
	46	575.62	0.21	0.14	0.00	0.28	4086	1340	2764	1	40	1	40	1	0.02	0.06	0.0	0.29
	47	622.09	0.58	0.12	0.11	0.82	10867	3701	5196	15	60	15	80	7	0.11	0.42	0.0	1.58
	48	579.06	0.83	0.05	0.35	0.23	2091	932	1439	10	80	12	160	19	0.07	0.77	0.0	2.2
49	559.61	0.42	0.05	0.10	0.29	2746	907	2054	9	120	9	140	5	0.03	0.44	0.0	1.0	

Table EC.10 Detailed Experimental Results for group 50e in CMST-L

Root													Total					
instance	id	UB	LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
50e	0	1108.67	0.26	0.46	0.02	4.23	14341	11277	7941	10	140	10	140	3	0.37	1.71	0.0	5.77
	1	1147.73	0.45	1.66	0.02	78.35	74567	93208	31888	22	260	22	260	5	18.25	55.52	0.0	164.15
	2	1007.27	0.46	0.36	0.08	6.04	10216	5768	5471	39	180	40	300	15	1.98	8.84	0.0	33.23
	3	1084.11	0.27	0.37	0.00	15.17	18570	11640	9274	25	260	25	260	1	0.64	7.54	0.0	15.17
	4	1123.23	0.49	0.4	0.04	40.99	59680	46381	25688	12	240	22	300	7	13.21	84.38	0.0	171.97
	5	1096.21	0.33	2.2	0.14	16.78	30416	30936	14478	29	160	32	360	15	18.05	59.31	0.0	200.31
	6	1002.3	0.30	0.47	0.00	2.98	8555	5812	5398	12	80	12	80	1	0.07	0.48	0.0	2.98
	7	1038.71	0.31	0.44	0.10	6.09	12395	5939	6126	25	100	25	140	7	1.88	2.53	0.0	27.39
	8	1078.0	0.24	1.9	0.00	3.75	7619	6843	4495	14	120	14	120	1	0.06	0.61	0.0	3.75
	9	1117.05	0.95	1.94	0.44	67.28	42770	64964	22216	30	280	88	4040	193	381.02	3662.82	0.0	5698.32
	10	1115.71	0.59	2.09	0.12	37.53	35614	38830	17620	57	260	80	400	7	9.65	84.8	0.0	158.48
	11	1093.43	0.87	0.33	0.12	21.83	15115	10972	8495	44	220	54	620	11	10.15	90.49	0.0	182.58
	12	1018.23	0.38	0.45	0.00	12.68	25196	29145	13436	1	160	1	160	3	2.35	8.06	0.0	20.13
	13	1158.83	0.13	0.76	0.00	1.5	6519	3855	3942	9	40	9	40	1	0.06	0.2	0.0	1.51
	14	1093.22	0.36	0.77	0.04	14.35	17901	16347	9161	2	200	2	240	3	1.35	12.67	0.0	24.94
	15	1081.8	0.74	0.53	0.28	52.52	57491	73886	24013	60	220	203	1920	123	454.76	3052.99	0.0	6154.21
	16	1044.07	0.32	0.67	0.00	4.14	11891	13633	7255	14	140	14	140	1	0.21	1.39	0.0	4.14
	17	1058.85	0.52	2.02	0.19	24.33	26786	25837	13320	13	220	47	720	37	52.19	507.51	0.0	826.39
	18	1073.22	0.38	0.99	0.10	8.8	20676	19142	10893	13	180	14	360	5	2.66	54.42	0.0	70.24
	19	1039.36	0.93	1.68	0.30	19.73	23568	28696	11512	17	160	76	1340	35	143.02	546.99	0.0	1403.39
	20	1042.51	0.28	0.4	0.00	2.03	11298	11128	6377	24	60	24	60	1	0.15	0.61	0.0	2.04
	21	1138.7	0.38	1.63	0.00	20.55	39528	35631	20551	16	200	16	200	1	1.47	9.17	0.0	20.55
	22	1107.96	0.43	0.47	0.03	25.3	35953	40623	18184	12	280	12	300	3	3.12	18.04	0.0	36.68
	23	1020.01	0.53	0.49	0.17	152.62	138385	236755	52938	14	180	37	1180	51	267.78	1674.92	0.0	3162.57
	24	1064.9	0.19	3.07	0.00	9.1	7505	3744	5364	12	40	12	40	1	0.09	0.54	0.0	9.11
	25	1030.41	0.31	0.37	0.04	6.51	12195	6971	6643	8	200	8	220	3	0.26	2.55	0.0	8.53
	26	1121.63	0.13	0.48	0.00	4.16	6296	4522	3566	13	60	13	60	1	0.05	0.44	0.0	4.16
	27	941.54	0.25	0.29	0.00	3.37	15348	12328	8111	10	160	10	160	1	0.16	0.78	0.0	3.37
	28	1059.92	0.65	2.39	0.27	15.33	32768	31579	15290	26	160	148	1620	119	255.77	1397.36	0.0	3027.87
	29	1128.08	0.29	0.92	0.02	11.51	43910	51759	18273	2	140	2	160	3	2.96	4.77	0.0	24.54
	30	1044.71	0.22	0.43	0.00	1.6	8526	8419	4636	1	100	1	100	1	0.14	0.45	0.0	1.6
	31	1109.93	0.32	2.75	0.03	72.92	57299	74195	25055	36	200	53	280	5	21.77	128.21	0.0	322.8
	32	1135.53	0.44	0.95	0.00	8.1	18124	17944	9454	22	180	22	180	1	0.65	1.76	0.0	8.1
	33	1068.54	0.52	0.44	0.04	20.9	33189	36789	16133	11	300	13	340	3	3.26	35.21	0.0	55.7
	34	1024.63	0.56	1.58	0.09	35.0	53426	56922	22375	19	180	19	300	7	14.77	100.85	0.0	202.81
	35	1039.3	0.37	1.17	0.12	18.03	39096	24388	18492	29	260	40	860	33	107.12	213.02	0.0	1070.95
	36	1057.71	0.42	2.17	0.00	5.04	8140	6847	4799	47	100	47	100	1	0.07	0.83	0.0	5.04
	37	1001.47	0.77	0.33	0.14	19.81	31087	41340	15671	24	240	26	320	7	9.72	46.36	0.0	95.93
	38	1080.92	0.40	0.31	0.06	23.63	24798	29260	12801	19	340	19	460	9	5.77	65.0	0.0	99.48
	39	1038.41	0.69	1.13	0.42	46.64	39098	62091	16200	42	260	273	4020	154	461.55	4075.07	0.16	7200.0
	40	1046.8	0.56	1.74	0.00	12.12	16727	21635	8765	21	200	21	200	1	0.49	6.62	0.0	12.12
	41	1049.91	0.44	2.14	0.00	19.88	31643	24756	14986	17	160	17	160	1	1.37	5.9	0.0	19.89
	42	1052.06	0.61	0.49	0.02	46.37	34707	40915	18065	23	360	23	380	3	5.58	39.71	0.0	74.3
	43	1049.89	0.77	1.21	0.42	88.09	43224	104052	22249	12	160	99	1960	77	340.78	5554.02	0.23	7200.0
	44	1017.24	0.45	0.48	0.00	2.23	13921	10445	8182	12	100	12	100	1	0.13	0.74	0.0	2.23
	45	1061.87	0.30	0.48	0.00	4.5	12959	14793	7301	17	180	17	180	1	0.24	1.92	0.0	4.51
	46	1106.69	0.68	0.43	0.34	14.5	21732	32035	11462	5	260	371	9640	352	668.13	2165.39	0.04	7200.0
	47	1113.14	0.55	1.96	0.09	50.51	44771	45732	21567	12	220	22	300	7	23.05	62.46	0.0	212.05
	48	1028.04	0.63	0.39	0.27	4.25	13154	7009	7663	11	180	126	1880	61	43.76	210.31	0.0	551.65
49	1098.59	0.51	1.34	0.22	9.22	21823	17477	10149	30	140	54	1440	61	129.49	353.22	0.0	1195.31	

Table EC.11 Detailed Experimental Results for group 50r in CMST-L

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
50r	0	591.99	0.74	0.06	0.14	1.63	8699	6162	5119	8	220	9	260	13	0.53	2.77	0.0	7.36
	1	737.05	0.43	0.23	0.06	1.4	6673	5096	4076	1	120	2	140	5	0.19	1.22	0.0	3.54
	2	701.63	0.39	0.22	0.00	1.2	4721	3092	3024	14	80	14	80	1	0.03	0.34	0.0	1.2
	3	676.35	0.17	0.44	0.00	0.62	2052	2794	1384	11	40	11	40	1	0.02	0.16	0.0	0.62
	4	859.79	0.08	0.38	0.00	2.35	2267	850	1497	4	60	4	60	1	0.01	0.13	0.0	2.35
	5	958.69	0.57	0.44	0.17	7.05	22837	15843	11808	16	260	16	440	11	4.33	24.29	0.0	61.23
	6	767.16	0.83	0.22	0.25	49.47	34776	60355	17101	19	200	46	940	71	56.54	1161.9	0.0	1482.38
	7	741.73	0.78	0.2	0.13	21.18	32080	30826	16935	2	160	3	340	11	14.12	101.17	0.0	163.17
	8	684.67	0.56	0.21	0.00	0.96	5964	3175	3841	12	80	12	80	1	0.04	0.32	0.0	0.96
	9	829.18	0.38	0.33	0.08	1.01	5036	1865	2944	11	140	11	140	11	0.46	1.42	0.0	6.7
	10	850.33	0.38	0.29	0.13	1.63	5894	2241	3375	5	80	17	440	11	2.14	8.86	0.0	31.37
	11	670.77	0.17	0.22	0.13	0.28	2350	1062	1524	2	0	10	20	7	0.03	0.18	0.0	0.93
	12	715.21	0.75	0.82	0.00	4.32	9377	4291	5988	12	180	12	180	1	0.09	1.16	0.0	4.32
	13	790.46	0.26	0.39	0.05	1.92	12738	15587	6754	11	100	11	100	3	0.52	0.97	0.0	4.7
	14	786.88	0.38	0.36	0.00	11.27	28545	46600	17075	4	140	4	140	3	2.64	11.21	0.0	22.63
	15	872.36	0.35	0.28	0.07	13.04	22840	14621	12155	7	240	7	360	5	3.62	30.07	0.0	60.48
	16	702.67	0.72	0.32	0.10	5.08	18466	13521	8975	1	120	14	200	3	2.02	4.71	0.0	16.24
	17	585.5	0.60	0.14	0.10	0.54	3784	1803	2324	11	60	11	60	7	0.06	0.26	0.0	1.22
	18	690.71	0.64	0.34	0.00	1.56	4836	2224	2843	16	40	16	40	1	0.04	0.22	0.0	1.56
	19	811.37	0.34	0.28	0.05	11.31	26637	25252	13869	3	140	3	160	3	1.79	27.95	0.0	38.34
	20	879.34	0.83	0.37	0.36	15.88	20669	17699	10774	50	120	122	1480	111	157.28	1493.45	0.0	2532.23
	21	797.79	0.46	0.43	0.30	0.85	6457	3610	3790	12	40	47	1340	45	5.95	21.79	0.0	84.16
	22	688.66	0.11	0.16	0.00	0.62	3883	1361	2467	31	80	31	80	1	0.01	0.1	0.0	0.62
	23	720.74	0.92	0.14	0.18	6.82	20590	12838	10010	24	220	24	380	11	8.56	70.37	0.0	127.62
	24	809.76	0.75	0.21	0.22	9.34	20099	21922	10932	0	120	43	760	33	36.08	219.75	0.0	426.04
	25	625.11	0.34	0.19	0.07	2.2	12403	8519	5720	2	180	3	280	3	0.49	5.79	0.0	9.28
	26	853.64	0.74	0.27	0.27	7.25	23787	20647	11281	12	180	100	2200	115	235.46	851.0	0.0	2416.23
	27	694.71	0.39	0.3	0.00	0.88	4954	2383	2959	20	40	20	40	1	0.01	0.13	0.0	0.88
	28	773.76	0.52	0.38	0.04	10.29	39111	20730	16378	24	160	25	240	3	2.27	5.58	0.0	24.62
	29	1062.23	0.66	1.17	0.28	49.85	25656	63814	12486	22	100	29	540	27	232.97	1665.67	0.0	2524.97
	30	712.62	0.39	0.69	0.17	2.32	4481	1989	2381	11	80	22	200	19	1.09	4.97	0.0	19.75
	31	729.42	0.66	0.22	0.00	1.6	9824	5191	5649	23	80	23	80	1	0.08	0.56	0.0	1.6
	32	973.97	0.58	2.11	0.25	5.78	9629	6791	5912	25	100	34	620	19	10.91	87.32	0.0	181.12
	33	970.96	0.27	0.42	0.00	1.17	8333	4731	4312	11	80	11	80	1	0.07	0.2	0.0	1.17
	34	792.2	0.00	0.23	0.00	0.26	3214	1652	2084	0	20	0	20	1	0.01	0.05	0.0	0.26
	35	551.25	0.61	0.08	0.19	0.87	5308	3096	3124	7	160	7	240	13	0.32	1.64	0.0	4.73
	36	698.52	0.60	0.3	0.00	9.17	17965	12693	8680	11	220	11	220	3	1.4	4.94	0.0	17.11
	37	769.78	1.28	0.28	0.55	41.03	20504	40226	12390	19	100	26	1380	83	225.54	3688.96	0.0	4744.59
	38	657.63	0.45	0.14	0.19	1.76	4926	3385	3003	11	100	12	280	15	1.46	8.49	0.0	18.34
	39	687.32	0.66	1.49	0.14	7.5	16305	19302	8368	49	160	49	320	5	3.08	15.3	0.0	39.31
	40	649.14	0.72	0.23	0.31	0.55	4046	2359	2577	11	80	11	220	19	0.27	1.19	0.0	4.05
	41	788.53	0.58	0.16	0.07	0.75	4067	1430	2708	13	80	13	140	9	0.37	1.46	0.0	5.11
	42	535.24	0.66	0.08	0.05	0.33	3022	708	1778	2	120	2	120	3	0.02	0.19	0.0	0.66
	43	719.96	0.21	0.16	0.00	0.42	4269	1889	2962	5	60	5	60	1	0.01	0.14	0.0	0.42
	44	621.56	0.32	0.09	0.00	0.35	3519	1557	2092	2	100	2	100	1	0.01	0.1	0.0	0.35
	45	800.28	0.54	0.26	0.22	9.36	25565	18536	11125	9	180	9	700	35	27.03	67.96	0.0	285.5
	46	547.65	1.47	0.07	0.00	1.07	4395	2112	2884	8	140	8	140	1	0.04	0.58	0.0	1.07
	47	766.44	0.56	0.29	0.11	8.4	22484	25955	11155	4	180	5	340	17	14.49	36.96	0.0	116.72
	48	684.08	0.78	0.17	0.18	0.77	5009	1209	3567	3	140	3	240	39	0.53	3.61	0.0	9.2
	49	628.97	0.21	0.14	0.00	0.29	2393	1010	1522	11	20	11	20	1	0.01	0.07	0.0	0.29

Table EC.12 Detailed Experimental Results for group 100c in CMST-L

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
100c	0	1075.43	0.20	1.41	0.00	11.66	14980	6572	8170	35	160	35	160	1	0.28	1.4	0.0	11.67
	1	1102.35	0.41	2.25	0.00	22.4	21565	12143	11651	51	220	51	220	1	0.57	2.84	0.0	22.4
	2	1108.36	0.49	0.83	0.00	126.11	36282	35422	19934	67	440	67	440	3	4.49	123.65	0.0	160.45
	3	1096.08	0.23	2.59	0.00	15.79	22595	16653	12575	51	160	51	160	1	0.61	3.06	0.0	15.79
	4	1073.83	0.84	0.79	0.09	96.01	65983	58481	33421	16	340	20	860	35	69.5	214.06	0.0	681.29
	5	1106.46	0.45	3.23	0.08	28.67	36064	45052	19660	41	200	50	400	7	8.6	97.73	0.0	182.18
	6	1042.48	0.65	1.14	0.08	20.53	23357	13962	12262	3	360	9	900	31	10.65	96.48	0.0	204.84
	7	1136.44	0.92	1.29	0.18	84.62	54202	52673	24644	54	500	89	2440	81	156.1	3331.62	0.0	4364.51
	8	1104.34	0.66	1.39	0.08	143.63	48668	52174	25323	62	280	108	460	17	28.9	764.04	0.0	973.31
	9	1140.36	0.37	1.16	0.02	12.51	20815	12574	11672	4	200	5	280	5	1.1	5.37	0.0	25.92
	10	1040.94	0.41	1.1	0.11	28.22	39739	36158	20430	28	300	54	620	13	15.86	222.92	0.0	328.98
	11	1046.84	0.77	2.39	0.11	33.28	31630	17862	16893	32	300	36	800	39	15.09	75.54	0.0	242.34
	12	1100.99	0.63	1.11	0.13	17.78	22455	18226	11443	34	280	52	1280	61	12.06	92.9	0.0	243.57
	13	1076.33	0.55	0.74	0.13	49.57	48884	38254	23763	64	380	183	1740	33	64.44	256.44	0.0	848.21
	14	1052.86	0.58	1.33	0.08	93.23	61139	51588	28182	35	460	37	980	55	36.36	344.1	0.0	635.18
	15	1096.6	0.59	1.13	0.01	34.27	42122	38536	21795	34	280	34	300	3	3.0	27.08	0.0	53.99
	16	1081.68	0.50	1.0	0.08	26.67	25609	17325	14200	31	240	31	440	17	6.09	86.21	0.0	164.42
	17	1196.53	0.55	1.45	0.12	119.97	54693	64332	27773	32	480	40	2140	75	152.94	1170.8	0.0	2060.61
	18	1100.99	0.36	1.17	0.00	19.35	15240	7995	8752	32	160	32	160	1	0.21	3.04	0.0	19.35
	19	1064.04	0.20	1.49	0.02	7.85	16077	7879	7767	25	160	25	180	3	1.27	4.18	0.0	21.16
	20	1102.98	0.33	1.41	0.05	24.29	33673	27793	16350	50	260	51	400	9	9.66	52.06	0.0	135.55
	21	1082.3	0.50	0.74	0.01	73.71	44853	49395	25255	52	400	52	420	9	8.08	147.65	0.0	208.88
	22	1026.59	0.35	2.09	0.04	11.9	10961	5291	6880	28	140	31	280	13	3.64	8.01	0.0	44.36
	23	1088.23	0.36	1.04	0.04	38.44	38808	31590	21810	25	300	26	500	11	16.02	399.48	0.0	503.52
	24	1095.76	0.57	1.49	0.13	39.69	32190	33844	17157	47	300	51	860	17	41.58	242.63	0.0	503.43
	25	1088.68	0.59	0.84	0.02	75.98	69396	53541	30991	52	360	52	380	7	11.74	47.69	0.0	157.9
	26	1058.18	0.68	3.02	0.10	72.43	40460	60078	20468	43	340	72	1040	35	57.33	612.7	0.0	1002.96
	27	1067.8	0.54	0.97	0.10	22.25	25737	20873	14274	26	320	34	740	19	21.53	45.44	0.0	204.14
	28	1071.54	0.72	2.26	0.18	73.09	29698	29247	16578	38	580	42	2820	87	65.73	660.02	0.0	1448.09
	29	1136.18	0.55	1.26	0.04	27.85	28420	16910	14901	34	300	41	440	31	6.59	60.29	0.0	133.25
	30	1036.52	0.43	1.14	0.09	25.93	31471	21850	15840	41	320	50	880	41	24.82	50.21	0.0	281.39
	31	1062.34	0.45	0.88	0.11	16.22	18026	14482	9635	50	260	56	720	25	12.39	198.92	0.0	299.95
	32	1059.19	0.90	1.17	0.10	126.76	72102	72942	36672	35	460	37	1920	83	231.64	1395.28	0.0	2792.03
	33	1105.18	0.32	0.93	0.00	24.82	23319	11331	11389	27	240	27	240	3	1.48	29.3	0.0	45.21
	34	1071.75	0.52	2.34	0.03	61.19	41099	40626	19269	55	360	65	560	7	16.99	70.22	0.0	210.65
	35	1128.93	0.32	1.03	0.00	22.78	20943	17365	11872	35	400	35	400	1	0.85	5.33	0.0	22.79
	36	1123.29	0.48	0.97	0.09	33.5	32813	32202	16584	20	280	23	920	27	42.38	451.77	0.0	716.23
	37	1114.46	0.65	2.06	0.01	99.47	61426	53497	26983	36	420	36	420	3	13.39	38.95	0.0	170.66
	38	1123.32	0.50	1.37	0.07	28.23	35020	31304	16887	7	340	12	780	15	22.5	106.78	0.0	261.18
	39	1095.43	0.26	2.19	0.00	21.22	16985	11949	9234	44	260	44	260	1	0.48	3.59	0.0	21.22
	40	1087.76	0.50	1.13	0.10	51.41	29327	32442	15407	44	360	73	1680	41	49.6	537.07	0.0	898.65
	41	1070.24	0.67	0.99	0.06	39.66	24875	16789	12973	17	360	18	460	15	2.55	23.42	0.0	85.73
	42	1004.47	0.57	1.43	0.02	20.15	21236	14962	13192	35	240	35	300	9	1.98	16.38	0.0	47.84
	43	1059.29	0.53	2.62	0.08	92.9	55340	51328	24977	65	360	67	860	21	48.06	409.98	0.0	814.21
	44	1098.47	0.52	1.12	0.03	31.68	34849	29349	18289	12	340	12	380	3	3.27	23.51	0.0	55.2
	45	1135.27	0.55	2.6	0.12	74.02	57885	87008	23577	35	280	47	700	25	95.59	107.32	0.0	704.71
	46	1120.81	0.63	0.82	0.15	68.59	50338	68533	23863	46	360	99	1440	63	225.59	2060.45	0.0	3416.86
	47	1132.81	0.64	0.64	0.13	19.74	26340	24674	12353	32	320	34	1840	41	42.02	126.51	0.0	452.44
	48	1061.25	0.33	1.02	0.00	19.82	25319	24649	13830	46	320	46	320	1	0.89	5.23	0.0	19.82
	49	1034.51	0.43	1.2	0.05	24.58	29074	25880	16743	14	300	40	1000	33	33.88	123.64	0.0	353.15

Table EC.13 Detailed Experimental Results for group 100e in CMST-L

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
100e	0	2167.85	0.21	4.01	0.04	53.35	46649	49727	25653	18	340	19	600	9	9.56	99.03	0.0	232.12
	1	2007.67	0.38	2.78	0.13	70.75	52860	50334	26413	23	320	127	1580	29	465.81	839.74	0.09	7200.0
	2	2111.53	0.38	2.76	0.14	124.25	81110	116884	38280	50	400	115	1560	33	479.82	1023.97	0.06	7200.0
	3	1987.9	0.34	3.01	0.13	49.81	56724	68279	26663	12	280	78	2380	107	324.4	2036.9	0.0	4404.17
	4	2031.29	0.34	5.51	0.07	119.94	86143	95767	46053	74	420	75	1460	44	582.88	889.45	0.01	7200.0
	5	2137.38	0.41	4.48	0.14	121.2	50990	63021	27481	34	380	121	1860	50	518.62	4315.57	0.07	7200.0
	6	2032.0	0.54	2.68	0.32	33.17	37241	30264	19367	13	260	51	1640	27	989.83	537.83	0.26	7200.0
	7	2209.25	0.26	5.42	0.04	110.98	62497	85094	30188	29	360	35	720	17	140.27	171.88	0.0	613.81
	8	2030.22	0.45	3.98	0.21	83.02	62349	67795	30862	16	420	246	4200	100	1000.43	3877.33	0.09	7200.0
	9	2022.63	0.46	3.2	0.22	68.66	39419	33301	19833	52	380	162	4380	72	1604.69	2493.19	0.1	7200.0
	10	2000.73	0.33	4.3	0.03	299.4	88339	100606	40446	42	480	56	820	15	201.6	898.58	0.0	1668.52
	11	2212.79	0.49	4.88	0.32	57.45	43504	64682	23612	42	320	67	720	8	317.35	166.81	0.26	7200.0
	12	1924.29	0.53	1.99	0.28	216.63	91471	119043	47448	46	420	57	2420	77	1048.37	4088.21	0.18	7200.0
	13	2138.08	0.55	3.41	0.28	76.39	51759	44633	26821	31	440	73	1540	26	462.23	772.16	0.19	7200.0
	14	2056.77	0.86	3.99	0.64	52.18	39872	34847	21508	10	320	82	2420	35	549.03	895.66	0.46	7200.0
	15	1959.48	0.33	4.66	0.14	57.66	37810	29217	19087	35	300	118	1560	35	728.6	452.05	0.07	7200.0
	16	2088.47	0.40	4.98	0.21	68.46	39881	45421	20590	21	280	111	2040	49	1936.25	1011.53	0.12	7200.0
	17	1960.29	0.57	4.12	0.28	160.51	91038	73359	44376	15	380	47	3080	71	1070.11	3704.29	0.17	7200.0
	18	2046.15	0.50	3.94	0.17	270.35	93923	109352	41571	105	500	253	1580	37	1365.38	2769.8	0.1	7200.0
	19	2177.98	0.44	3.66	0.25	46.32	36538	43229	18934	21	320	131	2520	53	1824.44	992.42	0.18	7200.0
	20	1932.49	0.27	3.87	0.02	128.9	79168	72375	36411	39	380	40	480	9	82.15	166.54	0.0	449.5
	21	2126.71	0.25	3.53	0.06	71.7	50722	49273	26094	30	340	38	1000	13	217.19	568.55	0.0	1181.78
	22	2012.08	0.35	2.12	0.08	61.89	45740	35809	21982	13	340	27	960	25	195.45	567.28	0.0	1220.95
	23	2045.21	0.44	2.76	0.26	68.63	44425	38569	20979	35	300	82	2500	57	1065.29	2598.5	0.17	7200.0
	24	2018.63	0.38	4.03	0.14	51.38	47725	44226	25329	32	340	101	1160	27	725.95	1515.54	0.07	7200.0
	25	1945.32	0.66	3.32	0.34	221.73	74197	72347	36316	47	500	101	2500	52	1500.6	3514.39	0.21	7200.0
	26	2091.22	0.25	3.19	0.01	58.6	41578	28279	19202	25	300	25	360	3	8.72	35.83	0.0	95.16
	27	1967.48	0.59	3.28	0.30	87.54	56689	47521	27753	40	400	85	3260	63	1498.81	1908.02	0.19	7200.0
	28	2097.35	0.62	3.76	0.40	107.95	83767	80299	36990	46	320	112	1780	37	1790.2	1326.13	0.33	7200.0
	29	2105.64	0.76	2.47	0.52	116.06	61233	72963	30171	4	300	82	1660	38	1554.08	3063.5	0.44	7200.0
	30	1947.71	0.24	2.4	0.05	46.62	35982	34456	19767	7	340	19	1080	17	148.72	498.76	0.0	956.46
	31	2017.15	0.64	5.25	0.46	43.82	23773	29777	12793	64	260	167	3840	96	2345.37	1300.33	0.35	7200.0
	32	2174.5	0.32	4.3	0.05	305.69	74638	69544	34984	69	400	95	980	19	408.31	1655.68	0.0	2804.45
	33	2131.95	0.74	2.73	0.42	81.3	60407	54741	30947	5	300	225	2820	80	1258.78	2818.43	0.32	7200.0
	34	2057.77	0.28	3.72	0.00	78.12	79697	89318	34927	25	300	25	300	1	13.65	29.51	0.0	78.13
	35	2159.75	0.29	2.5	0.00	297.33	97161	239559	42545	28	300	28	300	1	83.43	115.8	0.0	297.34
	36	1998.03	0.57	3.23	0.31	136.62	55926	53720	26122	41	440	165	2260	31	1389.28	1592.53	0.23	7200.0
	37	2018.32	0.28	3.26	0.08	62.56	39186	31033	19635	59	360	139	1740	43	738.21	490.25	0.0	4422.4
	38	2120.07	0.28	2.51	0.16	24.24	18603	11243	9756	2	280	4	600	7	71.48	31.77	0.13	7200.0
	39	2031.09	0.23	2.45	0.06	46.21	41179	45970	21078	6	340	18	640	9	98.39	88.39	0.0	345.46
	40	2031.24	0.26	4.12	0.19	31.43	20999	14588	12593	48	160	360	3880	80	2167.29	247.52	0.1	7200.0
	41	2048.91	0.27	3.85	0.06	41.75	27499	21129	14380	26	200	51	620	19	210.83	100.68	0.0	995.44
	42	1875.66	0.33	4.28	0.08	54.8	36649	36406	21041	24	300	30	1580	57	1344.74	435.52	0.0	6999.66
	43	2223.27	0.53	4.03	0.28	286.34	85455	174896	38728	27	320	160	1020	33	1415.73	2546.34	0.22	7200.0
	44	2152.7	0.32	4.28	0.04	158.76	71258	110432	36271	56	320	59	420	9	102.81	605.4	0.0	884.38
	45	2153.81	0.30	4.1	0.08	72.12	73508	93969	34278	38	260	113	1440	59	1378.54	1142.52	0.0	5628.13
	46	2129.14	0.57	5.32	0.24	133.4	59049	68703	27693	71	440	125	2540	53	2053.95	1786.52	0.16	7200.0
	47	2047.15	0.27	6.18	0.04	84.63	51662	36438	22445	51	280	85	620	17	206.81	139.7	0.0	817.71
	48	2041.83	0.39	2.41	0.12	106.82	65706	66639	31866	33	480	38	1340	19	1786.29	531.31	0.07	7200.0
	49	2066.72	1.02	3.3	0.75	60.52	48790	46876	27539	13	280	72	1560	37	1466.53	1215.88	0.66	7200.0

Table EC.14 Detailed Experimental Results for group 100r in CMST-L

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
100r	0	1594.36	0.38	3.81	0.11	46.68	28661	24575	15271	44	360	250	2760	105	1747.87	1491.68	0.0	7137.01
	1	1886.7	0.50	2.66	0.25	198.91	84742	125611	42062	68	240	130	1580	36	1624.07	2057.15	0.15	7200.0
	2	1476.98	0.21	3.13	0.02	84.04	60889	60333	24936	55	360	55	420	9	138.33	69.19	0.0	632.59
	3	1166.95	0.59	2.76	0.14	191.24	54530	50537	27001	26	400	78	1900	77	1177.54	3564.09	0.0	6697.02
	4	1155.41	0.44	1.78	0.13	20.9	21370	16905	11931	31	260	46	1200	27	65.8	94.63	0.0	319.17
	5	1202.53	0.23	2.1	0.01	28.21	18668	13785	10763	6	260	6	260	5	2.33	5.96	0.0	36.52
	6	1601.89	0.29	3.14	0.04	21.7	25313	15215	14297	7	200	24	480	29	77.84	111.77	0.0	491.64
	7	1107.75	0.51	1.42	0.09	31.95	30826	26601	15687	22	380	28	840	23	94.87	277.9	0.0	572.57
	8	1326.98	0.30	2.82	0.07	38.77	45376	43529	20852	33	340	56	860	37	553.47	366.8	0.0	1583.93
	9	1496.46	0.63	3.28	0.33	82.22	56605	49047	30086	61	380	147	3640	69	1868.36	2277.07	0.2	7200.0
	10	1493.36	0.73	1.71	0.37	988.75	157032	372822	60016	23	280	26	900	22	1991.24	1856.63	0.27	7200.0
	11	1793.7	0.30	3.02	0.04	46.74	35779	25945	19534	14	300	14	480	5	29.47	55.4	0.0	159.93
	12	1706.71	0.46	2.05	0.15	41.25	35602	34581	20480	24	340	456	6900	316	1813.45	2069.18	0.01	7200.0
	13	1369.53	0.33	1.74	0.00	22.29	36882	25543	16379	31	200	31	200	1	3.1	4.25	0.0	22.29
	14	1245.24	0.33	1.67	0.11	37.09	33476	31864	15317	33	300	50	1700	55	210.11	376.92	0.0	1094.16
	15	1614.99	0.30	3.62	0.03	37.85	36854	30430	19933	28	300	28	480	3	15.72	41.04	0.0	110.24
	16	1132.19	0.31	1.29	0.00	24.66	28595	24894	14055	17	300	17	300	1	3.66	4.81	0.0	24.67
	17	1315.93	0.46	1.57	0.12	24.63	24074	15993	13225	21	320	49	1180	47	57.24	110.29	0.0	342.3
	18	1724.02	0.34	2.66	0.06	91.82	60532	55353	29711	13	400	28	720	13	142.64	499.18	0.0	911.13
	19	1111.8	0.41	1.49	0.02	14.8	13928	10195	7264	19	240	19	260	3	1.29	6.12	0.0	22.47
	20	1577.96	0.90	2.21	0.70	45.49	32163	26053	16379	30	380	267	6380	176	1856.56	1435.74	0.58	7200.0
	21	1501.88	0.41	2.37	0.12	94.17	67215	89853	33299	40	340	55	1320	55	734.25	1268.42	0.0	2995.52
	22	1140.39	0.20	1.62	0.00	5.08	12898	8404	8547	25	60	25	60	1	0.38	1.32	0.0	5.08
	23	1232.63	0.35	2.79	0.00	22.9	19287	14838	10574	23	220	23	220	1	1.5	4.38	0.0	22.9
	24	1507.25	0.62	2.95	0.30	221.67	79552	83366	33757	38	280	69	2080	53	1460.03	2375.73	0.17	7200.0
	25	1595.2	0.28	2.96	0.02	95.81	74219	63079	32534	32	300	37	500	3	82.58	121.37	0.0	355.84
	26	1368.11	0.45	3.42	0.04	68.29	47415	46834	23974	12	340	12	420	9	30.22	95.05	0.0	209.82
	27	1535.43	0.22	1.77	0.00	33.45	33039	32660	18668	15	300	15	300	1	3.65	15.04	0.0	33.45
	28	1116.7	0.42	2.95	0.18	22.23	25399	17445	14070	5	220	116	2600	131	704.95	1556.44	0.0	3734.17
	29	1210.07	0.36	2.75	0.00	35.6	28771	18854	14319	56	280	56	280	1	2.29	10.1	0.0	35.6
	30	1264.27	0.40	2.23	0.03	32.56	33834	31530	16618	33	280	34	400	3	12.81	39.5	0.0	83.82
	31	1484.86	0.24	3.59	0.08	61.05	24325	18650	12382	48	340	68	840	23	533.47	477.62	0.0	1777.8
	32	1436.27	0.44	3.08	0.13	23.23	35431	28848	19049	22	240	77	1960	97	726.87	271.6	0.0	1804.26
	33	1592.71	0.58	2.13	0.16	131.13	62453	83255	32987	31	460	142	2820	88	1757.91	2501.22	0.01	7200.0
	34	1521.74	0.33	2.13	0.07	28.61	26485	20222	14423	30	260	41	780	19	28.99	90.9	0.0	227.38
	35	1777.04	0.34	2.6	0.10	26.71	31342	25048	17078	37	200	56	620	29	109.62	180.01	0.0	499.25
	36	1190.45	0.63	1.26	0.16	47.47	39865	33652	19583	30	360	115	3700	186	1592.07	2846.68	0.02	7200.0
	37	1621.93	0.38	2.01	0.06	47.78	38602	31064	20362	11	380	28	920	23	72.96	190.84	0.0	489.22
	38	1674.09	0.29	2.07	0.09	57.22	47711	38621	23271	46	360	190	2660	101	1033.86	1408.14	0.0	6026.37
	39	1311.14	0.31	2.21	0.00	28.5	23738	13851	13051	30	260	30	260	1	1.55	5.61	0.0	28.51
	40	1502.42	0.55	3.8	0.34	79.57	37141	47242	17351	67	340	270	3580	89	1686.0	1630.15	0.25	7200.0
	41	1192.53	0.47	1.6	0.11	8.13	15667	8515	8857	3	140	13	1080	33	28.34	66.58	0.0	186.5
	42	1805.55	0.58	3.4	0.34	64.43	39502	38324	21015	63	320	212	5100	132	1956.96	650.64	0.22	7200.0
	43	1562.93	0.29	2.66	0.08	42.23	37279	28809	19813	22	320	44	760	15	39.57	144.57	0.0	313.81
	44	1746.46	0.29	2.96	0.10	252.38	88134	144022	38514	21	360	38	1160	29	1052.23	1406.48	0.0	4102.67
	45	1497.78	0.56	2.77	0.23	37.16	40858	32430	18842	54	280	259	4800	158	1757.0	2096.19	0.11	7200.0
	46	1154.47	0.63	1.38	0.07	179.12	58464	47243	31430	22	440	26	1000	33	192.99	1108.53	0.0	1663.16
	47	1405.6	0.58	2.27	0.18	77.25	52758	52647	26286	42	340	142	4120	119	1817.42	2193.98	0.03	7200.0
	48	1977.6	0.35	5.36	0.06	80.84	53723	60300	24979	2	380	111	1020	33	267.4	248.87	0.0	1081.09
	49	1695.6	0.37	2.05	0.09	45.67	47422	34497	23165	2	320	46	1500	39	740.97	718.49	0.0	4675.2

Table EC.15 Detailed Experimental Results for group 150c in CMST-L

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
150c	0	1547.97	0.36	5.73	0.02	114.32	37944	23210	19842	6	360	18	540	5	19.66	53.86	0.0	269.41
	1	1634.42	0.35	7.56	0.07	127.19	75358	63123	38968	45	240	110	1300	43	623.09	90.27	0.0	1805.89
	2	1621.4	0.69	4.03	0.18	383.49	92002	91769	43010	67	520	120	2060	43	876.0	3505.01	0.1	7200.0
	3	1578.88	0.51	6.99	0.15	266.53	50791	50833	26685	67	600	158	3660	66	870.84	2604.45	0.04	7200.0
	4	1626.89	0.58	5.38	0.21	199.11	49723	45509	24831	31	460	62	5000	127	934.31	2630.09	0.12	7200.0
	5	1653.69	0.35	5.13	0.00	157.29	47856	44273	25190	81	380	81	380	1	12.58	21.38	0.0	157.3
	6	1549.0	0.32	5.58	0.04	112.29	45956	34136	24762	29	360	41	680	5	54.8	64.44	0.0	369.63
	7	1594.66	0.62	4.34	0.24	269.0	72203	75410	39569	48	540	88	2580	64	933.77	3259.49	0.15	7200.0
	8	1584.62	0.30	8.04	0.02	292.16	77344	72209	41359	43	640	59	1020	11	122.9	431.14	0.0	1098.9
	9	1548.47	0.63	3.31	0.24	289.3	51252	39468	30299	32	620	93	4460	105	510.15	1785.58	0.13	7200.0
	10	1662.4	0.40	4.82	0.00	228.4	86593	70542	45506	45	520	45	520	1	20.29	44.7	0.0	228.41
	11	1535.34	0.41	5.02	0.04	168.76	49439	36290	27538	16	480	26	900	25	118.6	155.5	0.0	781.13
	12	1596.73	0.61	3.94	0.13	228.43	72394	66759	37151	46	580	97	3880	98	1769.36	1512.34	0.05	7200.0
	13	1617.85	0.38	5.8	0.11	191.86	43726	34102	20954	26	480	47	2920	64	1312.5	2471.5	0.02	7200.0
	14	1586.82	0.51	6.38	0.08	406.14	165353	383637	69760	55	320	56	840	31	1262.79	950.61	0.0	5199.65
	15	1587.63	0.53	6.49	0.15	323.71	74727	65935	36484	72	620	160	3500	86	1317.7	1981.76	0.05	7200.0
	16	1539.45	0.42	6.75	0.01	286.82	82273	81511	42113	13	540	13	700	9	92.48	124.96	0.0	577.99
	17	1638.97	0.35	4.12	0.10	100.23	31330	22833	17682	10	360	134	3140	125	365.58	428.83	0.0	2366.33
	18	1621.33	0.35	5.7	0.03	189.35	52923	48045	27793	17	480	72	1220	81	220.04	419.66	0.0	1526.0
	19	1672.75	0.41	4.85	0.09	145.72	71782	81093	35442	21	320	51	1420	63	968.92	2121.2	0.0	4977.59
	20	1519.63	0.39	4.57	0.03	156.86	46478	30207	25542	56	400	149	1480	51	194.87	318.58	0.0	1302.01
	21	1609.34	0.47	27.29	0.12	303.95	76765	75884	41463	65	440	115	3820	79	1434.62	1744.61	0.03	7200.0
	22	1605.97	0.26	5.08	0.06	103.63	38274	38473	21559	28	340	78	940	23	198.21	464.53	0.0	1171.66
	23	1608.46	0.72	6.73	0.28	489.4	146515	166779	69507	47	620	104	1880	39	1924.36	1314.14	0.22	7200.0
	24	1624.84	0.58	7.43	0.19	448.16	106978	110745	50959	59	600	99	1920	41	1400.27	2700.46	0.11	7200.0
	25	1582.06	0.49	6.97	0.09	268.49	69073	67979	39215	42	500	131	3360	77	1234.54	2844.62	0.01	7200.0
	26	1556.01	0.26	6.53	0.01	227.39	108445	141852	52446	35	380	35	440	3	143.22	24.11	0.0	432.21
	27	1653.48	0.28	6.11	0.02	230.42	77619	73795	37413	33	440	33	600	11	70.13	163.71	0.0	582.85
	28	1568.84	0.31	4.54	0.06	115.18	34032	26504	19932	42	360	48	2340	55	198.62	469.7	0.0	1726.02
	29	1571.36	0.30	5.43	0.05	152.04	49351	40691	27455	37	380	60	780	13	220.41	135.21	0.0	941.75
	30	1584.15	0.55	4.45	0.21	374.45	80135	81489	39453	50	480	94	2660	60	1372.22	2491.83	0.12	7200.0
	31	1598.0	0.53	2.89	0.07	333.97	83460	85836	37806	34	520	76	1700	53	653.29	1693.87	0.0	4060.16
	32	1595.89	0.53	4.58	0.14	361.99	105225	111580	52482	88	500	111	2500	67	1397.74	1823.87	0.06	7200.0
	33	1639.54	0.40	4.99	0.00	196.15	55429	41507	29278	51	480	51	480	1	10.31	39.66	0.0	196.15
	34	1519.33	0.39	4.64	0.02	283.43	92509	77506	42480	34	480	34	700	13	184.3	563.08	0.0	1294.24
	35	1623.23	0.50	5.87	0.13	201.74	81796	83509	36739	41	440	172	3320	80	1625.41	1603.56	0.07	7200.0
	36	1522.81	0.37	4.27	0.09	352.3	56624	63309	27221	50	540	83	2440	87	505.62	1833.03	0.0	4071.01
	37	1572.69	0.44	3.48	0.04	277.85	65759	71185	33877	63	580	63	1040	31	209.47	617.99	0.0	1665.63
	38	1515.91	0.85	24.78	0.27	1640.47	206245	321469	107979	55	720	57	1140	11	2422.89	632.08	0.24	7200.0
	39	1583.58	0.40	5.63	0.06	116.93	40454	33478	22205	22	400	56	1340	45	172.5	291.37	0.0	1181.26
	40	1667.61	0.37	4.51	0.05	312.5	61355	57892	30339	110	540	158	1300	41	342.15	443.38	0.0	2313.66
	41	1603.81	0.46	5.61	0.08	227.38	57999	54378	29232	21	500	24	1560	39	262.45	385.69	0.0	1812.8
	42	1622.17	0.62	8.32	0.14	563.48	108237	145300	48786	52	600	134	1800	30	1119.03	3135.55	0.05	7200.0
	43	1603.59	0.49	6.66	0.05	207.58	72017	62090	32589	68	460	70	900	21	133.21	182.91	0.0	897.83
	44	1609.33	0.25	5.98	0.00	115.01	32313	26825	18975	71	340	71	340	1	4.51	14.68	0.0	115.02
	45	1626.41	0.52	8.5	0.16	1636.26	183438	258156	80975	35	480	42	900	9	2815.11	262.54	0.12	7200.0
	46	1634.05	0.35	6.36	0.03	106.62	39241	40346	21082	24	320	24	1020	23	116.69	141.82	0.0	759.09
	47	1601.93	0.54	5.64	0.18	1152.74	153346	301771	70172	46	580	55	2060	32	2664.21	750.59	0.1	7200.0
	48	1590.43	0.29	3.1	0.04	129.28	36822	23776	19332	30	420	37	860	15	46.34	76.82	0.0	434.34
	49	1657.83	0.28	4.98	0.00	141.98	49284	50205	25503	38	280	38	320	3	22.98	27.02	0.0	187.78

Table EC.16 Detailed Experimental Results for group 150e in CMST-L

instance	id	UB	Root										Total					
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
150e	0	3002.45	0.20	10.0	0.02	204.8	96035	81807	46981	20	460	35	780	13	102.31	218.66	0.0	778.09
	1	3102.37	0.41	7.07	0.26	215.19	71278	59479	37842	5	440	63	3820	86	938.85	1895.91	0.2	7200.0
	2	3054.29	0.66	8.1	0.49	344.18	78907	95199	42075	36	480	140	2560	49	1395.69	2319.42	0.44	7200.0
	3	3035.46	0.66	6.2	0.53	177.51	62000	60257	31384	31	400	178	4540	93	878.04	1808.06	0.49	7200.0
	4	3020.73	0.68	7.47	0.55	227.83	47536	44936	25866	44	540	209	6140	96	511.97	2077.97	0.49	7200.0
	5	3092.38	0.20	6.8	0.05	329.09	67955	69032	35291	27	580	121	4880	122	898.73	1479.24	0.01	7200.0
	6	3099.78	0.39	8.07	0.12	462.72	106635	118663	53519	27	660	97	2580	37	1001.91	2484.34	0.06	7200.0
	7	3104.73	0.52	7.2	0.33	307.74	78352	111164	39613	17	420	119	2200	47	1021.57	3212.75	0.27	7200.0
	8	2980.83	0.16	7.08	0.03	304.24	63557	62620	33628	18	420	50	1400	27	261.71	1373.16	0.0	2789.33
	9	2992.97	0.27	6.7	0.04	315.98	96741	101965	49710	34	520	41	1180	17	261.31	472.91	0.0	1748.96
	10	3043.01	0.24	8.57	0.05	351.03	102989	102097	48985	56	540	81	2540	58	1330.01	2055.42	0.01	7200.0
	11	3046.96	0.70	8.15	0.47	514.58	137318	125349	65704	42	520	43	1380	54	695.62	4199.02	0.43	7200.0
	12	3017.4	0.44	7.59	0.28	255.45	59903	64860	32620	35	480	235	3660	75	1010.09	2302.22	0.22	7200.0
	13	3081.28	0.34	7.29	0.15	330.87	94033	98918	48677	36	520	204	3360	67	1338.41	1500.33	0.1	7200.0
	14	3065.38	0.21	7.78	0.04	285.8	73135	75313	34575	11	500	39	1400	25	199.74	447.44	0.0	1792.63
	15	3068.14	0.65	7.64	0.42	345.17	130605	129595	64193	33	500	169	2700	50	1587.82	1526.01	0.37	7200.0
	16	2880.52	0.39	7.4	0.26	226.61	63836	51650	32119	2	500	204	4900	97	1010.81	1698.71	0.21	7200.0
	17	3107.66	0.21	6.65	0.04	348.47	76816	79133	39885	17	620	78	1440	23	269.11	646.49	0.0	2457.68
	18	3052.38	0.14	8.2	0.01	111.83	43562	33844	21793	12	280	13	500	3	23.97	52.51	0.0	267.15
	19	2890.86	0.27	11.76	0.11	402.29	87158	92237	42728	81	480	181	2220	44	1651.59	1376.23	0.07	7200.0
	20	2992.85	0.17	9.67	0.00	139.69	64732	55354	33362	38	260	38	260	1	11.08	23.25	0.0	139.69
	21	3135.19	0.21	7.86	0.05	213.53	60405	52170	31121	22	480	184	3280	67	685.83	1045.12	0.0	4724.59
	22	3083.99	0.45	7.28	0.30	215.28	88169	84325	45931	36	440	212	3200	64	1364.75	1321.25	0.26	7200.0
	23	2982.46	0.53	8.87	0.33	415.88	87083	145400	42072	46	500	126	2140	40	1046.81	3627.63	0.27	7200.0
	24	3061.48	0.14	8.89	0.01	224.12	77204	73071	41379	23	460	52	680	7	54.92	119.54	0.0	656.4
	25	3017.53	0.45	7.1	0.37	176.99	83250	91796	43035	15	300	35	1000	16	566.74	268.45	0.33	7200.0
	26	3002.97	0.29	9.0	0.17	253.13	93876	73340	44556	32	460	123	3320	73	1067.21	1299.14	0.13	7200.0
	27	2861.28	0.44	7.3	0.28	278.92	81657	85153	44942	20	480	105	3780	60	845.53	2894.08	0.24	7200.0
	28	2938.71	0.37	9.1	0.21	233.14	71064	65120	36442	19	500	124	3740	85	885.24	2589.04	0.16	7200.0
	29	3129.11	0.47	8.88	0.29	323.51	83169	82477	43972	40	560	119	3340	57	788.13	2735.51	0.24	7200.0
	30	3103.76	0.52	10.23	0.32	640.46	110281	149699	47620	46	440	99	1240	17	1843.0	2057.19	0.27	7200.0
	31	3013.19	0.59	7.63	0.43	213.58	81998	67579	40354	38	420	247	3500	68	1397.45	1670.51	0.39	7200.0
	32	3122.05	0.19	7.8	0.05	197.03	97405	89696	49690	33	440	79	2080	37	656.1	770.01	0.0	3455.61
	33	3161.72	0.54	7.8	0.37	368.98	128257	141436	53903	52	500	77	2240	31	1738.79	1152.99	0.33	7200.0
	34	3101.62	0.19	9.21	0.05	179.75	55950	51236	29959	60	380	195	3480	95	597.94	1018.85	0.0	4625.42
	35	3076.24	0.95	7.44	0.75	568.88	123110	134002	62831	91	640	122	1880	31	1125.42	2668.43	0.71	7200.0
	36	2935.38	0.16	9.7	0.00	167.47	58716	54755	31333	44	360	44	360	1	9.7	21.86	0.0	167.48
	37	2964.26	0.18	20.55	0.11	322.67	148415	153770	87165	26	280	56	1400	18	3399.19	147.42	0.09	7200.0
	38	3164.24	0.43	8.54	0.29	292.41	81508	70578	39452	51	580	120	1980	33	1220.45	1002.54	0.25	7200.0
	39	2938.38	0.30	7.25	0.16	223.84	70937	68817	38214	33	500	113	4300	90	1024.61	1873.23	0.11	7200.0
	40	3032.14	0.77	7.1	0.55	288.89	93978	100862	44355	23	520	129	3600	52	968.97	2217.92	0.5	7200.0
	41	2951.29	0.46	6.78	0.30	302.78	72723	70237	37810	29	520	94	3380	64	697.69	3155.74	0.25	7200.0
	42	3081.22	1.05	7.78	0.96	131.8	42885	38095	23132	12	300	93	2240	46	252.02	576.08	0.91	7200.0
	43	2957.07	0.75	7.45	0.58	252.04	87402	93644	47908	40	440	147	2080	35	1392.12	2364.66	0.51	7200.0
	44	3056.82	0.66	8.35	0.46	424.86	103401	101834	51691	39	580	103	1920	46	898.42	3444.24	0.41	7200.0
	45	3122.27	0.77	8.22	0.58	256.6	88313	83219	44004	52	540	133	1320	16	2170.83	1334.38	0.54	7200.0
	46	2986.84	0.53	9.38	0.28	304.42	118082	124933	54868	75	440	240	1800	35	1267.99	2587.21	0.22	7200.0
	47	3082.7	0.53	10.55	0.41	222.14	59828	58866	31424	22	480	107	3520	70	1430.38	1439.1	0.36	7200.0
	48	3041.69	0.62	8.38	0.41	561.41	116501	133438	51355	36	560	59	3100	49	810.61	2535.49	0.36	7200.0
	49	3106.09	0.16	9.06	0.05	169.5	57191	48272	29333	35	380	123	2540	81	957.6	2012.02	0.0	5954.02

Table EC.17 Detailed Experimental Results for group 150r in CMST-L

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
150r	0	2374.98	0.21	10.87	0.09	119.99	57866	29830	23668	57	260	139	2580	73	1759.61	424.5	0.01	7200.0
	1	2092.1	0.65	7.42	0.34	371.02	87942	133686	42983	35	560	174	2720	56	1231.99	2667.73	0.28	7200.0
	2	2214.91	0.34	5.09	0.07	208.66	63116	45016	32690	2	420	22	1560	37	220.89	734.04	0.0	2289.14
	3	2148.01	0.42	9.28	0.22	138.61	65608	53430	33406	42	320	102	920	21	1654.75	573.8	0.17	7200.0
	4	2206.27	0.31	7.29	0.17	93.41	47904	36088	27183	17	240	58	960	13	338.26	220.97	0.14	7200.0
	5	2603.15	0.53	9.4	0.37	164.98	59580	48585	32844	18	400	92	4360	83	859.85	2536.94	0.28	7200.0
	6	2440.13	0.54	10.14	0.31	423.84	105604	94981	48295	52	500	111	2180	49	980.17	2902.88	0.26	7200.0
	7	2016.71	0.65	7.86	0.34	380.84	111467	200820	59276	62	300	138	2060	35	1644.81	2610.32	0.27	7200.0
	8	2448.62	0.32	6.43	0.14	236.6	82861	77742	40921	12	420	121	3880	87	1180.45	1337.27	0.08	7200.0
	9	2010.31	0.24	6.49	0.09	139.01	57236	44125	28811	39	300	76	2160	79	1689.26	1936.68	0.0	7056.55
	10	2225.49	0.35	5.01	0.05	191.36	61439	45429	31372	33	420	69	1200	39	222.47	403.3	0.0	1673.66
	11	1930.25	0.57	5.4	0.22	219.68	64003	57134	34789	12	460	111	3800	117	1150.34	1838.5	0.15	7200.0
	12	2234.97	0.25	6.67	0.01	278.87	88865	88862	42605	46	520	47	540	5	71.39	107.7	0.0	475.7
	13	1917.15	0.27	10.28	0.05	205.14	36883	25269	20453	60	460	72	1380	25	185.36	393.74	0.0	1500.98
	14	2187.82	0.25	7.67	0.07	143.06	50453	34743	25627	32	320	133	1740	35	423.35	377.27	0.0	3066.21
	15	2444.03	0.31	8.41	0.03	302.39	78112	70655	40294	63	460	77	1000	11	204.6	488.94	0.0	1504.7
	16	2628.28	0.37	11.95	0.21	198.76	61951	45564	30843	49	380	184	2280	46	1234.47	907.88	0.17	7200.0
	17	2439.29	0.37	7.61	0.21	261.31	83259	88978	37364	60	460	130	2220	28	806.92	1718.25	0.15	7200.0
	18	1842.65	0.40	4.68	0.16	176.97	65577	59183	33209	41	420	82	1860	44	496.01	390.64	0.12	7200.0
	19	2390.24	0.47	6.07	0.24	199.5	62946	59365	34413	15	480	157	4660	109	1012.28	2011.97	0.18	7200.0
	20	2421.68	0.33	7.63	0.09	154.35	54914	40021	27502	38	380	387	4520	140	1053.95	2042.16	0.04	7200.0
	21	2169.73	0.34	7.98	0.05	305.83	81332	82017	38562	27	520	54	1780	61	1314.81	1683.32	0.0	6287.42
	22	1728.01	0.75	3.38	0.37	194.78	63036	60198	34331	11	400	45	3100	67	1459.29	1460.94	0.27	7200.0
	23	1950.94	0.67	122.46	0.39	931.32	120461	97360	67606	47	580	120	1900	36	2042.84	800.33	0.32	7200.0
	24	2109.77	0.29	7.46	0.07	148.92	51456	40699	27017	36	400	83	1860	35	326.73	437.6	0.0	2065.3
	25	1726.51	0.24	8.36	0.01	160.89	52185	47026	26278	64	400	64	440	3	21.72	40.43	0.0	211.38
	26	2034.09	0.30	6.48	0.05	234.72	58583	52548	30346	64	520	85	2060	35	293.42	734.98	0.0	2543.27
	27	2917.83	0.33	13.15	0.11	369.37	90553	81827	45117	80	520	107	2060	41	1044.51	677.48	0.08	7200.0
	28	1941.1	0.46	7.49	0.17	432.31	73291	82382	41562	66	600	191	2420	60	891.08	1704.59	0.12	7200.0
	29	1712.2	0.52	8.09	0.07	386.64	100618	95019	44820	72	560	125	2060	38	1794.96	1650.45	0.01	7200.0
	30	1915.28	0.16	7.14	0.02	102.91	38643	28442	20409	13	360	29	540	15	50.34	40.78	0.0	317.89
	31	2419.96	0.27	13.01	0.05	301.39	84907	64304	43451	67	520	100	1560	27	2656.47	539.24	0.0	7173.92
	32	1513.74	0.37	5.32	0.03	199.39	56859	48217	30340	13	480	38	600	9	46.44	111.27	0.0	441.75
	33	2364.41	0.29	6.88	0.08	245.24	80064	74551	41095	33	440	52	940	15	176.48	312.44	0.05	7200.0
	34	2193.56	0.44	5.25	0.22	217.64	55033	41099	29343	21	540	65	4180	64	978.59	2378.48	0.15	7200.0
	35	1687.81	0.47	5.34	0.14	338.03	75357	74039	38155	50	620	83	2860	52	1050.07	2513.54	0.03	7200.0
	36	2119.82	0.41	7.28	0.18	171.22	49145	33876	25215	34	480	210	5480	118	1080.16	1107.54	0.12	7200.0
	37	1905.47	0.35	5.99	0.03	231.19	62168	49095	32566	17	540	20	1000	19	153.92	330.17	0.0	1155.42
	38	2214.65	0.43	6.46	0.13	325.48	96461	101309	47325	41	480	43	2200	36	923.79	1063.25	0.08	7200.0
	39	2253.01	0.72	7.81	0.45	274.34	75877	83961	37894	50	460	113	2920	53	1370.53	1595.2	0.36	7200.0
	40	2333.04	0.20	9.43	0.00	154.78	69342	63467	37609	66	260	66	360	5	49.99	51.41	0.0	323.09
	41	2028.24	0.31	7.07	0.12	100.46	29996	27310	16986	33	320	176	4160	89	2066.58	372.14	0.04	7200.0
	42	2000.46	0.33	8.01	0.06	150.62	52081	54464	26529	20	400	30	1000	49	202.9	197.24	0.0	1097.95
	43	2462.77	0.51	9.83	0.25	281.88	86847	77762	44587	18	460	148	2560	43	1500.33	2195.93	0.18	7200.0
	44	1625.35	0.43	9.15	0.08	427.47	100155	103056	52462	71	620	209	3140	86	1877.2	1134.78	0.01	7200.0
	45	2062.27	0.48	5.28	0.22	195.98	62582	53831	32868	17	480	84	4700	68	1065.96	1757.79	0.14	7200.0
	46	1719.94	0.37	4.83	0.05	269.95	80927	65676	39380	80	500	122	1780	37	611.42	1126.96	0.0	3490.55
	47	2345.5	0.38	8.06	0.16	154.5	54832	44282	26242	15	400	314	4440	89	1427.98	1090.06	0.11	7200.0
	48	1829.49	0.29	4.49	0.09	113.01	39843	30994	20621	16	380	171	3760	161	1029.89	620.43	0.0	5008.29
	49	2176.43	0.37	7.32	0.11	241.63	93546	86892	45681	32	520	74	1320	20	2311.06	476.77	0.08	7200.0

Table EC.18 Detailed Experimental Results for CMST-F

instance	id	UB	Root									Total						
			LP gap (%)	LP Time	LPC gap (%)	LPC Time	No.P1 Label	No.P2 Label	No.R Label	No. RCI	No. SRI	No. RCI	No. SRI	No. Node	Join Time	Pr Time	Opt gap (%)	IP Time
q-6-8	1	21103.63	1.47	1.39	0.60	5.08	14062	6607	5850	39	205	500	3094	265	13.48	0.83	0	746.97
	2	21855.3	1.24	1.13	0.00	3.39	11401	5048	5017	32	175	32	175	1	0.07	0.01	0	9.41
	3	18776.61	0.69	1.48	0.11	4.03	14541	7375	5887	15	233	15	304	5	0.26	0.02	0	20.14
	4	22061.06	0.94	2.1	0.27	15.35	15137	8349	5749	25	158	72	533	51	2.42	0.16	0	158.15
	5	16836.02	1.17	0.91	0.20	3.16	13530	7034	5685	18	174	20	410	18	0.99	0.08	0	42.24
	6	22999.88	0.66	1.95	0.00	10.31	10928	4198	4839	70	141	70	141	1	0.07	0.01	0	16.87
	7	18782.55	1.29	0.93	0.41	5.05	13593	6559	5914	26	223	104	780	57	2.44	0.18	0	128.1
	8	18114.08	0.84	1.4	0.05	4.91	13167	5855	5634	12	227	12	227	3	0.14	0.02	0	13.08
	9	27817.95	1.07	0.91	0.00	1.85	10080	4156	4467	29	133	29	133	1	0.06	0.01	0	7.69
	10	16811.35	0.89	0.69	0.16	7.16	12623	5532	5563	3	213	6	279	13	0.44	0.04	0	37.47
	11	23885.42	0.63	0.97	0.03	3.47	13378	6766	5544	2	220	2	220	3	0.16	0.02	0	14.28
	12	23460.52	1.07	1.57	0.21	6.51	13275	6954	5461	57	184	89	300	9	0.52	0.04	0	37.27
	13	21209.38	0.57	0.8	0.07	2.28	12787	5833	5386	1	189	1	216	7	0.27	0.03	0	16.54
	14	19499.13	0.65	0.94	0.00	4.97	9946	3459	4484	53	128	53	128	1	0.08	0.01	0	11.85
	15	23914.31	1.48	2.03	0.55	11.09	12061	5328	5255	66	174	89	330	15	0.78	0.06	0	66.34
	16	17805.83	0.81	0.8	0.08	3.95	14569	7320	6026	14	267	14	267	3	0.18	0.02	0	14.26
	17	17409.64	1.14	0.74	0.29	3.98	15784	7994	6572	8	257	22	568	23	1.34	0.08	0	60.73
q-8-10	1	19353.37	1.12	0.74	0.00	8.28	51248	57003	21013	26	333	26	333	1	0.85	0.02	0	15.27
	2	20133.23	1.58	1.03	0.19	24.15	57883	76319	22925	61	333	66	366	11	8.9	0.06	0	148.3
	3	17566.56	0.81	0.43	0.18	6.67	48550	47400	20525	5	169	10	371	9	3.81	0.03	0	56.88
	4	20478.58	1.21	1.38	0.02	23.06	59767	71243	23201	53	259	53	259	3	1.81	0.02	0	41.05
	5	16188.87	1.02	1.26	0.10	10.54	46486	44563	18726	8	143	11	157	9	2.84	0.03	0	49.0
	6	20977.9	1.32	1.47	0.37	16.35	59666	69761	23319	44	250	182	960	53	38.14	0.19	0	564.74
	7	17581.59	1.11	0.64	0.09	13.27	53265	59422	21647	55	292	55	311	3	1.59	0.01	0	35.46
	8	17141.24	0.74	0.42	0.01	9.36	54128	56312	22105	10	217	10	228	5	1.96	0.02	0	31.39
	9	24606.39	0.68	1.12	0.00	4.38	30429	22970	13713	13	101	13	101	1	0.22	0.01	0	11.15
	10	15933.54	0.63	1.44	0.00	6.48	46005	44543	18797	1	155	1	155	1	0.5	0.01	0	13.9
	11	21740.49	1.45	0.6	0.30	13.43	56280	69709	22176	24	288	34	601	35	20.53	0.12	0	304.38
	12	21484.17	1.54	1.1	0.50	16.06	51574	60654	21270	38	225	50	816	39	24.53	0.13	0	381.3
	13	19550.31	1.07	0.83	0.40	6.44	52441	54315	21809	16	290	19	777	39	16.46	0.13	0	269.08
	14	18361.39	1.00	1.71	0.00	12.39	51555	52552	20681	35	217	35	217	1	0.61	0.01	0	20.28
	15	21905.66	1.47	1.41	0.23	16.2	53629	74059	21512	34	307	42	428	13	10.15	0.05	0	174.12
	16	16824.79	1.13	0.38	0.07	11.81	52769	56666	20728	0	265	0	265	3	1.27	0.01	0	25.37
	17	16575.76	1.81	0.66	0.62	12.94	59732	66688	23631	31	318	283	2850	163	104.56	0.6	0	1776.11
q-10-12	1	18545.12	1.03	5.39	0.17	55.86	174881	318292	67829	32	257	33	327	7	35.79	0.06	0	259.67
	2	19203.59	1.87	6.54	0.02	114.59	202429	529557	72190	14	335	14	335	3	32.25	0.03	0	189.15
	3	17053.19	0.95	9.15	0.06	78.64	202917	417937	77012	0	321	0	326	5	37.38	0.04	0	248.57
	4	19586.08	1.19	6.61	0.23	78.84	194248	390877	73316	62	282	64	433	5	33.66	0.06	0	240.7
	5	16136.5	1.22	6.92	0.00	35.76	143595	239066	57913	22	117	22	117	1	5.36	0.02	0	43.9
	6	19666.72	0.45	6.92	0.00	20.22	129231	158026	56762	2	142	2	142	1	2.96	0.01	0	29.21
	7	17110.43	1.37	6.94	0.00	100.8	203633	470089	73787	4	349	4	349	1	18.18	0.02	0	109.78
	8	16833.12	1.19	6.79	0.00	46.97	180672	352171	71050	10	274	10	274	1	7.95	0.02	0	55.85
	9	22830.01	0.74	7.2	0.00	33.15	131710	221770	53038	7	151	7	151	1	4.7	0.02	0	40.81
	10	15886.12	1.44	7.87	0.41	80.0	197112	397664	75142	13	347	13	797	17	139.85	0.14	0	950.63
	11	20513.55	1.59	6.35	0.06	99.48	186397	414421	69534	38	278	39	308	3	28.78	0.04	0	185.33
	12	20320.18	1.07	5.38	0.00	39.16	149980	261075	60813	36	159	36	159	1	4.89	0.01	0	48.3
	13	18648.96	0.86	8.2	0.17	49.89	189135	376515	73481	1	252	1	363	9	46.64	0.07	0	312.63
	14	17914.83	1.88	9.04	0.25	126.16	212297	512037	76389	13	315	13	593	25	165.36	0.17	0	1154.74
	15	20842.83	1.00	5.71	0.01	65.85	149442	321026	58034	33	247	33	247	3	14.2	0.03	0	109.39
	16	16472.06	1.61	6.79	0.10	121.44	207202	519261	75987	13	377	13	399	3	39.06	0.04	0	236.61
	17	16084.24	1.68	7.21	0.31	108.56	220768	481003	82234	39	354	46	880	37	316.83	0.32	0	2112.71
q-10-14	1	17863.14	0.49	93.25	0.00	289.83	410991	879945	157652	3	191	3	191	1	50.6	0.03	0.0	298.96
	2	18533.58	1.50	89.03	0.69	714.44	593290	2169219	182511	23	298	36	656	11	962.88	0.18	0.43	7200.0
	3	16662.96	0.75	164.2	0.00	490.32	592872	1616319	213039	1	206	1	206	1	69.82	0.02	0.0	500.3
	4	18999.33	0.66	170.35	0.03	748.73	619819	1759833	208657	3	266	3	274	3	231.2	0.05	0.0	1942.49
	5	15910.11	0.34	59.87	0.00	85.71	315250	531275	112376	3	72	3	72	1	10.25	0.01	0.0	94.64
	6	19098.14	0.92	98.49	0.00	777.96	716276	2401983	235474	0	297	0	297	1	120.56	0.02	0.0	787.82
	7	16944.32	1.76	69.56	0.30	1888.02	803586	3920736	214325	32	421	34	559	6	1238.91	0.11	0.12	7200.0
	8	16557.91	1.45	159.96	0.28	581.55	633187	1841010	207113	0	296	0	535	13	898.88	0.18	0.0	7200.0
	9	21812.04	0.18	75.99	0.00	103.37	286510	528023	120171	1	96	1	96	1	14.66	0.01	0.0	111.79
	10	15633.2	0.73	110.41	0.00	290.51	513138	1143631	173663	4	200	4	200	1	37.73	0.02	0.0	300.62
	11	19575.29	0.92	124.84	0.00	360.45	443023	1079700	156033	10	147	10	147	1	42.67	0.02	0.0	369.18
	12	19731.24	2.22	49.39	0.24	1992.56	700290	3642606	224028	24	495	32	609	7	1314.07	0.13	0.1	7200.0
	13	18134.35	0.92	137.96	0.22	715.74	646323	2082136	240977	0	306	0	427	9	633.91	0.12	0.0	4904.03
	14	17649.1	1.56	115.29	0.00	2905.57	818690	3756423	219235	14	337	14	337	1	400.07	0.05	0.0	2915.77
	15	19782.54	0.06	97.51	0.00	118.24	268251	455450	113263	1	19	1	19	1	13.1	0.01	0.0	127.92
	16	16173.62	1.15	96.5	0.01	2138.59	883515	4604803	200977	4	401	4	401	3	543.16	0.06	0.0	4822.78
	17	15964.11	1.54	129.78	0.20	1220.48	762717	2603896	221351	19	533	21	686	9	922.4	0.15	0.0	7200.0